

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**COURSE REPORT
INTERNSHIP
(Course ID: CO3335)**

SEMESTER 253 ACADEMIC YEAR 2025–2026

- MAJOR: Computer Science**
- TRAINING PROGRAM: High-quality**

- COMPANY / INTERNSHIP UNIT:**
Công ty TNHH Amazon Web Services Việt Nam

- DIRECT SUPERVISOR AT THE COMPANY:**
Lữ Hoàn Thiện

- FACULTY SUPERVISOR / REPORT GRADER:**
Vũ Ngọc Tú

- STUDENT:**
FULL NAME: Hoàng Đức Hiếu Anh STUDENT ID: 2352030

HO CHI MINH CITY, AUGUST 2026

INTERNSHIP REPORT

Table of Contents

1	Worklog	1
1.1	Week 1	1
1.2	Week 2	4
1.3	Week 3	6
1.4	Week 4	8
1.5	Week 5	11
1.6	Week 6	13
1.7	Week 7	15
1.8	Week 8	17
2	Proposal	30
3	Blogs Posted	41
3.1	Blog 1	41
3.2	Blog 2	42
3.3	Blog 3	48
3.4	Blog 4	51
4	Events Participated	57
4.1	Event 1: FCAJ Community Day – June 2026	57
4.2	Event 2: Enterprise Cloud Architectures and Industry Applications	59
4.3	Event 3: FCAJ x Agentic AI Build Week 2026	60
5	Workshop	62
5.1	Overview	64
5.2	Architecture	67
5.3	Prerequisites	69
5.4	Source Code Preparation	73
5.5	AWS Infrastructure	76
5.6	Database Deployment	80
5.7	Backend Deployment	84
5.8	Frontend Deployment	88
5.9	Monitoring and Logging	91
5.10	End-to-End Testing	95
5.11	Security and Cost	98
5.12	Troubleshooting	103
5.13	Clean-up	107
6	Self-Evaluation	112
7	Sharing and Feedback	114

1 Worklog

Internship Worklog

The AI-Powered Internship Application Tracker was developed by a five-member team. This internship worklog records the assigned contribution of one member over 8 weeks, from 08/06/2026 to 30/07/2026, rather than claiming ownership of the whole system. The schedule follows weekly reporting blocks that start on Monday; the final week is shortened to match the internship end date.

Individual Role in the Team

My primary responsibility was Cloud/DevOps and system integration. I prepared container and Kubernetes deployment paths, supported AWS architecture and CI/CD, validated the production runtime, collected technical evidence, and maintained deployment documentation. Backend, frontend, chat, and AI features are described only when I reviewed their architecture, integrated them into the deployment pipeline, or validated them with the teammates responsible for those components.

Week	Time	Individual Focus
1	08/06/2026 - 14/06/2026	Reviewed the repository and mapped service dependencies for later deployment work
2	15/06/2026 - 21/06/2026	Reviewed concurrency and idempotency requirements that affect multi-replica deployment
3	22/06/2026 - 28/06/2026	Contributed deployment requirements for the outbox, SQS, and asynchronous workers
4	29/06/2026 - 05/07/2026	Standardized containers, Docker Compose, CI quality gates, and smoke-test paths
5	06/07/2026 - 12/07/2026	Prepared local Kubernetes deployment, scaling controls, and observability resources
6	13/07/2026 - 19/07/2026	Prepared AWS IAM/OIDC, ECR, and CI image-publishing foundations
7	20/07/2026 - 26/07/2026	Integrated EKS workloads, managed AWS services, ALB routing, and rollout automation
8	27/07/2026 - 30/07/2026	Validated the AWS runtime, analyzed issues and costs, collected evidence, and completed documentation

Project-wide architecture and results are retained as context, while each weekly page separates my contribution from the work delivered by the other four team members.

1.1 Week 1

Week 1 - Project Analysis and Architecture Baseline

Objectives

Within the five-member team, my first-week objective was to understand the parts of the source code that affected deployment, identify service boundaries and dependencies, and document an architecture baseline for the later Cloud/DevOps work.

My Contributions

Status	My assigned contribution	Evidence basis
Completed	Reviewed the monorepo structure and identified the main runtime services.	Source tree, README.md, docker-compose.yml, and commit 4e939cf.
Completed	Mapped the frontend, FastAPI backend, chat service, AI service, PostgreSQL, Redis, and DynamoDB dependencies.	backend/, frontend/, chat-service/, ai_service/, Compose files, and service configuration.
Completed	Reviewed the initial request flow between browser, API, chat, and storage layers.	Frontend API clients, backend routers, chat Socket.IO server, and backend storage services.
Completed	Checked the local startup model and documented separate service startup commands.	Commits d3c1168 and e43cc04.

Technical Implementation

The source repository is a monorepo with separate service responsibilities:

- **frontend/**: React and Vite application for Candidate and HR users.
- **backend/**: FastAPI API, SQLAlchemy models, Alembic migrations, authentication, jobs, applications, documents, and AI orchestration routes.
- **chat-service/**: Node.js, Express, Socket.IO, Redis adapter, and DynamoDB-backed chat persistence.
- **ai_service/**: AI parsing, CV/job normalization, reranking, and later SageMaker adapter code.
- **docker-compose.yml**: local dependencies and multi-service startup path.

Initial architecture baseline:

Flow summary. The browser uses the React/Vite frontend, which calls the FastAPI backend for application functions and the Socket.IO service for realtime chat. The backend relies on PostgreSQL, document storage, and the AI service, while chat uses Redis pub/sub and DynamoDB tables.

The baseline also identified the first cloud migration direction: keep long-running backend and chat services containerized, move durable relational data to PostgreSQL/RDS, use DynamoDB for chat records, use Redis for realtime fan-out, and prepare the application for containerization and CI/CD.

Problems and Solutions

Problem	Root cause	Resolution	Status
The first source snapshot contained broad application code and generated dependency content.	The initial commit added the complete baseline in one large commit.	Later work separated documentation, scripts, and runtime responsibilities into clearer paths.	Completed
The project originally contained Prisma-related Node database artifacts while the backend used SQLAlchemy/PostgreSQL.	Database strategy changed toward FastAPI, SQLAlchemy, Alembic, and PostgreSQL.	Commit ab60f7d removed Prisma artifacts and documented the EC2/RDS direction.	Completed
Local startup needed to support more than one developer workflow.	A single startup command was useful for demos, but separate terminal commands were easier for debugging.	Commits d3c1168 and e43cc04 documented both startup approaches.	Completed

Testing, Build and Deployment Results

Area	Result	Evidence
Source inspection	Completed	<code>git show --stat 4e939cf</code> shows backend, frontend, Docker, migrations, tests, and docs added in the first commit.

Area	Result	Evidence
Local startup	Partially completed	Startup scripts and README instructions exist in history, but no saved terminal log was found.
Tests	Partially completed	Test files exist under <code>backend/tests</code> ; no Week 1 test output artifact was found.
Build	Partially completed	Dockerfiles and Vite project files exist; no Week 1 build log artifact was found.
Deployment	Planned	Week 1 only defined the deployment direction; no AWS deployment was expected yet.

Evidence

Commits and Pull Requests

Commit	Description	Evidence	Pull Request
4e939cf	Initial application baseline with backend, frontend, Docker Compose, migrations, and tests.	View commit	Evidence pending
5ab2924	Added AI CV matching and AWS RDS configuration.	View commit	Evidence pending
ab60f7d	Removed Prisma artifacts and documented EC2/RDS deployment direction.	View commit	Evidence pending
d3c1168	Added one-command full-stack startup.	View commit	Evidence pending
e43cc04	Documented separate service startup commands.	View commit	Evidence pending

Test Logs

Evidence pending: no saved Week 1 local or CI test log was found in the source repository.

Build Logs

Evidence pending: no saved Week 1 Docker or frontend build log was found in the source repository.

Deployment Logs

Not applicable for Week 1. Deployment planning started in this phase, but cloud deployment was planned for later weeks.

Weekly Results

By the end of Week 1, I had produced the service and dependency map needed for my deployment scope and an initial containerization/AWS deployment direction. The application code and feature baseline remained the shared output of the five-member team.

Lessons Learned

The main lesson was that deployment planning depends on clear runtime boundaries. Separating API, chat, AI, relational data, realtime pub/sub, and document storage early made later Docker, Kubernetes, and AWS work easier to reason about.

Next Week Plan

The next week should focus on data integrity and concurrency: database constraints, duplicate request handling, idempotency keys, versioned workflow commands, and tests that prove concurrent writes return controlled conflicts instead of inconsistent data.

1.2 Week 2

Week 2 - Concurrency Control and Data Integrity

Objectives

Within the five-member team, my Week 2 objective was to review the concurrency and data-integrity requirements that would affect multi-replica deployment. I supported the responsible backend and chat members by checking that idempotency, version checks, and database constraints were suitable for later Kubernetes scaling.

My Contributions

Status	My assigned contribution	Evidence basis
Completed	Analyzed concurrent writes across multiple backend pods.	<code>docs/architecture/concurrency-policy.md</code> .
Completed	Contributed deployment-oriented review notes to the optimistic and pessimistic UI policies.	<code>docs/architecture/concurrency-policy.md</code> and <code>docs/frontend/CMD_MUTATION_POLICY.md</code> .
Completed	Reviewed the backend member's database-backed idempotency path for candidate apply requests.	<code>backend/app/services/idempotency_service.py</code> , <code>backend/app/routers/jobs.py</code> , and <code>migration/0004_idempotency_records.py</code> .
Completed	Reviewed version-gated workflow commands using <code>expectedVersion</code> for multi-pod behavior.	<code>backend/app/services/optimistic_concurrency.py</code> , <code>backend/app/services/workflow_commands.py</code> , and <code>HR/Candidate routers</code> .
Completed	Coordinated integration validation for chat retry and duplicate-write behavior with the chat member.	<code>chat-service/repositories/chatRepository.js</code> and <code>chat-service/tests/chatRepository.concurrency.test.js</code> .

Technical Implementation

Concurrency control was implemented at the database and API-contract level:

- PostgreSQL unique constraints protect duplicate user, company, application, and idempotency records.
- Idempotency-Key is accepted for candidate application submission and selected workflow commands.
- Reusing the same idempotency key with the same payload replays the stored response.
- Reusing the same idempotency key with a different payload returns 409 with `idempotency_key_reused`.
- Mutable HR job and application commands require `expectedVersion`.
- Stale version updates return controlled 409 Conflict responses.
- Chat retries use deterministic client message IDs and DynamoDB conditional write behavior to avoid duplicate logical messages.

Conflict flow:

Flow summary. A client submits an application with an idempotency key. The backend reserves the key in PostgreSQL, creates the application for a new request, returns the stored response for an exact replay, and returns HTTP 409 when the payload or business state conflicts.

Problems and Solutions

Problem	Root cause	Resolution	Status
Duplicate registration or company creation could surface as uncontrolled database errors.	Unique constraints existed or were needed, but error translation had to be explicit.	Integrity errors are rolled back and translated into 409 Conflict.	Completed
Candidate apply can be retried by browser/network clients.	A second request may reach another backend pod before the first response is received.	Scoped idempotency records persist request hash and response data.	Completed



Problem	Root cause	Resolution	Status
HR workflow updates can become stale when two users act on the same job/application.	Plain updates do not know whether the client saw the latest row version.	Commands require <code>expectedVersion</code> and use conditional updates.	Completed
Chat retries can duplicate realtime messages.	Socket retries and REST retries can re-send the same logical message.	Chat repository uses deterministic message identity and skips duplicate broadcast on replay.	Completed

Testing, Build and Deployment Results

Area	Result	Evidence
Backend concurrency tests	Partially completed	Test sources exist: <code>backend/tests/test_apply_idempotency.py</code> , <code>backend/tests/test_apply_idempotency_postgres.py</code> , <code>backend/tests/test_optimistic_concurrency.py</code> , and <code>backend/tests/test_optimistic_concurrency_postgres.py</code> . Current-run output is pending.
Chat concurrency tests	Partially completed	Test source exists: <code>chat-service/tests/chatRepository.concurrency.test.js</code> . Current-run output is pending.
API conflict behavior	Implemented	Source contains explicit 409 handling for stale versions, duplicate apply, and idempotency-key reuse.
Deployment	Planned	This week focused on correctness before deployment.

Evidence

Commits and Pull Requests

Commit	Description	Evidence	Pull Request
0f9e7f2	Added backend workflow outbox, idempotency records, optimistic concurrency helpers, async processing foundation, and tests.	View commit	Evidence pending
5326cb0	Hardened chat repository concurrency and added chat concurrency tests.	View commit	Evidence pending
77faab4	Integrated workflow and processing contracts into frontend API helpers and UI flows.	View commit	Evidence pending
5041f84	Fixed failing optimistic concurrency test assertions.	View commit	Evidence pending

Test Logs

Evidence pending: attach actual output from commands such as:

```
cd backend
python -m pytest tests/test_apply_idempotency.py tests/test_optimistic_concurrency.py
python -m pytest -m postgres tests/test_apply_idempotency_postgres.py
→ tests/test_optimistic_concurrency_postgres.py
```

Build Logs

Not applicable for the core Week 2 task. No dedicated build artifact was found for this phase.

Deployment Logs

Not applicable for Week 2. The feature was designed to be safe before scaling the backend in Kubernetes.

Weekly Results

My Week 2 result was a deployment-focused review of the team's database-backed concurrency policy and its behavior with multiple replicas. The backend and chat implementations themselves were owned by the teammates responsible for those components.

Lessons Learned

Service load balancing does not serialize writes. Correctness must be enforced at the storage and transaction boundary, then reflected clearly through API response codes and UI behavior.

Next Week Plan

Build the transactional outbox and asynchronous processing layers so committed domain changes can trigger reliable downstream work without keeping the main API request open.

1.3 Week 3

Week 3 - Transactional Outbox and Asynchronous Processing

Objectives

Within the five-member team, my Week 3 objective was to define the deployment and integration requirements for the transactional outbox and asynchronous workers. I focused on production transport, worker lifecycle, retries, configuration, and observability while the backend member owned the business implementation.

My Contributions

Status	My assigned contribution	Evidence basis
Completed	Reviewed the <code>outbox_events</code> schema from a deployment and recovery perspective.	<code>backend/alembic/versions/0007_outbox_events.py</code> .
Completed	Validated the lifecycle states and retry model needed for operations.	Migration check constraint and <code>backend/app/services/outbox.py</code> .
Completed	Reviewed <code>deduplication_key</code> and <code>claim/retry</code> indexes for multi-worker execution.	Migration <code>0007_outbox_events.py</code> .
Completed	Defined runtime requirements for the outbox dispatcher and admin operations.	<code>backend/app/workers/outbox_dispatcher.py</code> and <code>backend/app/workers/outbox_admin.py</code> .
Completed	Documented SQS Standard as the production event transport for the team.	<code>docs/architecture/adr/ADR-001-production-event-transport.md</code> .
Completed	Coordinated integration requirements for <code>async_processing_jobs</code> and the document/AI worker.	Migration <code>0008_async_processing_jobs.py</code> , <code>processing_jobs.py</code> , and <code>processing_worker.py</code> .
Partially completed	Assessed downstream consumer readiness beyond the contract.	Source docs state downstream notification/search/analytics consumers were not implemented in this phase.

Technical Implementation

The outbox design keeps domain mutation and event creation in the same PostgreSQL transaction. If the business write fails, no event is inserted. If the API pod dies after commit, the outbox row remains `PENDING` and can be claimed later by the dispatcher.

Flow summary. The backend commits the business change and an outbox row in PostgreSQL. The dispatcher claims pending events and publishes them to SQS; successful events are marked published, temporary failures are retried with backoff, and terminal failures are marked dead. Consumers deduplicate messages by event ID.

Asynchronous processing uses a separate database-backed queue because document parsing and AI matching are user-visible jobs with status, retry behavior, source-version guards, and result payloads. The worker claims jobs, applies lease-based retry logic, and persists results back to PostgreSQL.

Problems and Solutions

Problem	Root cause	Resolution	Status
A committed workflow change could lose its follow-up event if publishing failed after the database commit.	Direct publish inside request handling is not transactional with PostgreSQL.	Insert a canonical outbox event in the same DB transaction, then dispatch asynchronously.	Completed
SQS Standard can redeliver or reorder messages.	Standard queues provide at-least-once delivery, not exactly-once delivery.	The event envelope includes <code>eventId</code> ; consumers must deduplicate by <code>eventId</code> .	Completed
Dispatcher failure can leave events half-processed.	A worker can die after claiming or publishing an event.	Lease expiry, retry counters, DEAD status, and admin requeue/cleanup commands were added.	Completed
AI and document work can take too long for API request latency.	Parsing CVs, extracting text, and reranking candidates are long-running operations.	Added <code>async_processing</code> jobs and <code>backend-processing-worker</code> .	Completed
Real downstream consumers were not part of this implementation slice.	I focused the week on producer reliability and the consumer contract.	I kept downstream consumers as future work until runtime evidence is added.	Partially completed

Testing, Build and Deployment Results

Area	Result	Evidence
Outbox unit/integration test coverage	Partially completed	Test sources exist: <code>test_outbox_events.py</code> , <code>test_outbox_postgres.py</code> , <code>test_outbox_publishers.py</code> , and <code>test_outbox_admin.py</code> . Current-run output is pending.
Async processing tests	Partially completed	Test sources exist: <code>test_processing_jobs.py</code> and <code>test_processing_jobs_postgres.py</code> . Current-run output is pending.
Kubernetes deployment readiness	Implemented	<code>k8s/app/backend-outbox-dispatcher.yaml</code> and <code>k8s/app/backend-processing-worker.yaml</code> exist.
SQS provisioning	Implemented	<code>scripts/aws/provision-outbox-sqs.ps1</code> and <code>.sh</code> exist. Runtime queue-name evidence still needs a screenshot or CLI log.

Evidence

Commits and Pull Requests



Commit	Description	Evidence	Pull Request
0f9e7f2	Added backend workflow outbox, async processing jobs, dispatcher, worker, and tests.	View commit	Evidence pending
be84e4e	Added Kubernetes deployments, SQS provisioning scripts, validation gates, and alerts for async workers.	View commit	Evidence pending
4970eba	Documented phase 2 workflow operations, event transport ADR, and consumer contract.	View commit	Evidence pending
14173c6	Merged workflow, outbox, async processing, and infrastructure changes.	View commit	Evidence pending

Test Logs

Evidence pending: attach actual output from commands such as:

```
cd backend
python -m pytest tests/test_outbox_events.py tests/test_outbox_publishers.py
→ tests/test_processing_jobs.py
python -m pytest -m postgres tests/test_outbox_postgres.py
→ tests/test_processing_jobs_postgres.py
```

Build Logs

Evidence pending: no dedicated Week 3 image build log was found.

Deployment Logs

Evidence pending: add dispatcher or worker rollout logs after deployment, for example `kubectl rollout status deployment/backend-outbox-dispatcher -n internship`.

Weekly Results

My Week 3 result was the SQS transport decision and a documented set of deployment requirements for the outbox dispatcher and processing worker. The outbox schema, backend services, and document/AI business logic remained team deliverables.

Lessons Learned

Idempotency exists at multiple layers. HTTP idempotency prevents duplicate command execution, the transactional outbox prevents lost domain events, and consumers still need event-level deduplication.

Next Week Plan

Standardize container builds, CI quality gates, smoke tests, security scans, and infrastructure validation so the system can be deployed repeatedly with less manual checking.

1.4 Week 4

Week 4 - Containerization and CI Quality Gates

Objectives

Week 4 was the first week centered on my primary Cloud/DevOps assignment in the five-member team. My objective was to make the teammates' backend, frontend, chat, and AI components repeatable through containers, Docker Compose, CI quality gates, smoke tests, and security checks.

My Contributions

Status	My assigned contribution	Evidence basis
Completed	Standardized Docker build paths for backend, chat, frontend, and AI service.	backend/Dockerfile, chat-service/Dockerfile, frontend/Dockerfile, and Dockerfile.ai-service.
Completed	Kept local Compose as the parity environment for dependencies and smoke tests.	docker-compose.yml, docker-compose.ai-service.yml, and scripts/ci/smoke-test.sh.
Completed	Consolidated GitHub Actions into the main CI/CD workflow.	.github/workflows/cicd.yml and commit e09e84e.
Completed	Added required quality/security gates.	scripts/ci/repository_quality.py, scripts/ci/infrastructure.py, scripts/ci/security_scan.py, .gitleaks.toml, and .pre-commit-config.yaml.
Completed	Fixed ShellCheck SC2155 in the smoke test script.	Commit 98f3420.

Technical Implementation

The final CI workflow defines validation jobs for backend, chat service, frontend, Docker images, infrastructure, smoke checks, and security scanning. The workflow uses Python 3.12, Node.js 22, shell validation, Docker image builds, Trivy/Gitleaks images, and actionlint/kubeconform support through the infrastructure script.

Relevant workflow checks include:

```
python -m ruff format --check .
python -m ruff check .
python -m mypy app tests
python -m pytest -m "not postgres"
npm run check --prefix chat-service
npm run lint --prefix chat-service
npm run format:check --prefix chat-service
npm run test:ci --prefix chat-service
npm run test --prefix frontend
npm run build --prefix frontend
docker build -t internship-backend:ci ./backend
docker build -t internship-chat:ci ./chat-service
docker build -f Dockerfile.ai-service -t internship-ai:ci .
```

Problems and Solutions

Problem	Root cause	Resolution	Status
Multiple workflows made the deployment path harder to control.	Separate CI/deploy workflows split validation and deployment behavior.	Consolidated workflows into .github/workflows/cicd.yml.	Completed
ShellCheck flagged SC2155 in the smoke test script.	Assignment and command substitution were combined in a way ShellCheck warns about.	Commit 98f3420 split the logic to satisfy ShellCheck.	Completed
.env files can contain UTF-8 BOM and break shell parsing.	Windows editors may add BOM at the beginning of files.	scripts/ci/smoke-test.sh strips the BOM when preparing smoke-test env input.	Completed
Docker-based infra validation may hang or fail when Docker is unavailable.	Local Windows/Docker setups vary by machine.	Infrastructure scripts support local CLI binaries and bounded Docker fallback behavior.	Completed

Testing, Build and Deployment Results

Area	Result	Evidence
Backend CI commands	Implemented	Commands are defined in <code>.github/workflows/cicd.yml</code> . Current-run output is pending.
Chat CI commands	Implemented	Commands are defined in <code>.github/workflows/cicd.yml</code> and <code>scripts/chat-service.sh</code> . Current-run output is pending.
Frontend CI commands	Implemented	Commands are defined in <code>.github/workflows/cicd.yml</code> and <code>scripts/frontend.sh</code> . Current-run output is pending.
Infrastructure/security checks	Implemented	<code>scripts/ci/infrastructure.py</code> , <code>scripts/check-infra.ps1</code> , <code>scripts/check-infra.sh</code> , and <code>scripts/ci/security_scan.py</code> exist.
Compose smoke test	Implemented	<code>scripts/ci/smoke-test.sh</code> exists and ends with <code>Compose smoke test passed</code> on success. Current-run output is pending.

Evidence

Commits and Pull Requests

Commit	Description	Evidence	Pull Request
b4ef947	Added required pipeline gates, security scanning, pre-commit config, and infrastructure validation scripts.	View commit	Evidence pending
e09e84e	Consolidated workflows into a single <code>cicd.yml</code> pipeline.	View commit	Evidence pending
29de145	Resolved CI failures for pytest, frontend build, and infrastructure checks.	View commit	Evidence pending
98f3420	Fixed ShellCheck SC2155 in the smoke test script.	View commit	Evidence pending
c2ad7cc	Consolidated and optimized the GitHub Actions workflow.	View commit	Evidence pending

Test Logs

Evidence pending: attach CI or local output for backend, chat, frontend, and smoke-test jobs.

Build Logs

Evidence pending: attach Docker build and Vite build output from GitHub Actions or a local run.

Deployment Logs

Not applicable for Week 4. This phase prepared deployable artifacts and gates, while AWS/EKS deployment continued in later weeks.

Weekly Results

My Week 4 result was a shared CI/CD backbone and repeatable container validation path that all five members could use. Feature code remained with the corresponding component owners; my responsibility was packaging, automation, and integration validation.

Lessons Learned

CI should fail early on repeatable checks: formatting, typing, tests, build, security, shell syntax, Dockerfile quality, and infrastructure manifest validation. Making these checks explicit reduced the amount of manual inspection needed before deployment.

Next Week Plan

Move from Docker Compose to local Kubernetes with kind, Kubernetes manifests, migration/init jobs, probes, HPA/PDB configuration, and observability resources.

1.5 Week 5

Week 5 - Local Kubernetes Deployment

Objectives

Within the five-member team, my Week 5 objective was to move the integrated application from Compose-only development to a local Kubernetes environment using kind. I owned the deployment manifests and automation, while teammates supplied and checked their component-specific runtime configuration.

My Contributions

Status	My assigned contribution	Evidence basis
Completed	Added kind cluster configuration.	<code>k8s/cluster/kind-config.yaml</code> and commit 8dae02c.
Completed	Added Kubernetes resources for backend, chat, PostgreSQL, Redis, DynamoDB Local, migration job, and chat initialization job.	<code>k8s/app/*.yaml</code> .
Completed	Added local deployment automation for PowerShell and shell users.	<code>scripts/k8s/deploy-local.ps1</code> and <code>scripts/k8s/deploy-local.sh</code> .
Completed	Added HPA and PDB definitions for app services.	<code>k8s/app/autoscaling.yaml</code> .
Completed	Added observability resources for Prometheus, Grafana, Loki, Tempo, Alloy, OTel collector, ServiceMonitors, and PrometheusRules.	<code>k8s/observability/*</code> and <code>observability/grafana/*</code> .

Technical Implementation

The local Kubernetes path uses kind for the cluster, in-cluster PostgreSQL/Redis/DynamoDB Local for dependencies, and Kubernetes manifests under `k8s/app` for the workload layer.

Flow summary. The local kind cluster contains the application workloads and local data services in the `internship` namespace. A separate monitoring namespace runs Prometheus, Grafana, Loki, and Tempo; Prometheus collects metrics from the backend, chat service, and outbox dispatcher.

The deployment scripts build local images, load them into kind, apply app manifests, run migration/init jobs, wait for rollouts, and print port-forward commands for backend, chat, and optional AI service checks.

Problems and Solutions

Problem	Root cause	Resolution	Status
Compose did not prove Kubernetes readiness.	Services could work locally while still missing probes, Services, Jobs, or ConfigMaps.	Added Kubernetes manifests and local deployment scripts.	Completed
Fresh local images might not be used by kind.	Building an image is not enough; kind nodes need the image loaded.	Scripts run <code>kind load docker-image</code> and restart deployments.	Completed
Migrations and chat table creation need ordered startup.	API/chat pods should not assume databases are initialized.	Added <code>backend-migrate</code> and <code>chat-init</code> Jobs.	Completed



Problem	Root cause	Resolution	Status
Local observability required multiple supporting components.	Metrics, logs, and traces need separate platform services.	Added Helm-backed Prometheus/Grafana/Loki/Tempo/Alloy resources and local runbook guidance.	Completed

Testing, Build and Deployment Results

Area	Result	Evidence
Kubernetes manifests	Implemented	k8s/app includes Deployments, Services, Jobs, ConfigMaps, Secret examples, ingress, HPA, and PDB.
Local deployment scripts	Implemented	scripts/k8s/deploy-local.ps1 and .sh automate cluster setup, image load, jobs, rollouts, and resource display.
Observability resources	Implemented	k8s/observability and observability/grafana include dashboards, datasources, rules, and collectors.
Runtime validation logs	Partially completed	Commands are scripted, but current report does not include saved kubectl output.

Evidence

Commits and Pull Requests

Commit	Description	Evidence	Pull Request
8dae02c	Added the first Kubernetes configuration for backend, chat, dependencies, jobs, namespace, and kind config.	View commit	Evidence pending
7dbe7c6	Added Kubernetes/EKS deployment platform, autoscaling, ingress, service accounts, observability, and deploy scripts.	View commit	Evidence pending
4f37d4d	Added Kubernetes deployment workflows.	View commit	Evidence pending
7407fda	Added tracing and chat initialization support.	View commit	Evidence pending
3b6f346	Added local terminal runbook and environment examples.	View commit	Evidence pending

Test Logs

Evidence pending: attach actual local Kubernetes output from commands such as:

```
kubectl cluster-info
kubectl get nodes
kubectl get pods -n internship
kubectl get services -n internship
kubectl rollout status deployment/backend -n internship
kubectl rollout status deployment/chat-service -n internship
```

Build Logs

Evidence pending: attach output from local image builds and `kind load docker-image` commands.

Deployment Logs

Evidence pending: attach output from `scripts/k8s/deploy-local.ps1` or `scripts/k8s/deploy-local.sh`.

Weekly Results

My Week 5 result was the local Kubernetes deployment path, including manifests, automation, scaling controls, and observability resources. Functional verification of backend, frontend, chat, and AI behavior was coordinated with the four component owners.

Lessons Learned

Local Kubernetes validation is more than checking that pods start. Jobs, readiness probes, image freshness, DNS, service ports, ingress, HPA metrics, and observability scraping all need explicit checks.

Next Week Plan

Prepare the AWS foundation: AWS CLI validation, region selection, GitHub OIDC, IAM deployment role, ECR repositories, and image build/push workflow.

1.6 Week 6

Week 6 - AWS Foundation, IAM, OIDC and ECR

Objectives

Within the five-member team, my Week 6 objective was to prepare the shared AWS deployment foundation: region configuration, GitHub OIDC authentication, IAM deployment permissions, ECR image repositories, and CI jobs that publish the team's service images without long-lived AWS credentials.

My Contributions

Status	My assigned contribution	Evidence basis
Completed	Standardized production region as <code>ap-southeast-1</code> .	<code>PROJECT_CONTEXT.md</code> , <code>.github/workflows/cicd.yml</code> , and deployment scripts.
Completed	Added GitHub OIDC smoke workflow.	Commit <code>2777b56</code> .
Completed	Added ECR build and push workflow for application images.	Commit <code>9f9fcf3</code> .
Completed	Aligned AWS deployment workflow with production architecture.	Commit <code>7eb1a76</code> .
Completed	Added EKS access smoke test workflow.	Commit <code>bfacd3a</code> .

Technical Implementation

The CI/CD path uses GitHub Actions OIDC to assume the AWS deployment role instead of storing long-lived AWS access keys. ECR image URIs are based on AWS account, region, repository name, and immutable Git SHA tags.

Flow summary. GitHub Actions obtains an OIDC token and assumes the scoped deployment role. That role permits SHA-tagged backend and chat images to be pushed to ECR and allows the deployment script to update the EKS workloads without long-lived AWS credentials.

The source repository includes deployment scripts for both EKS application deployment and frontend deployment. I keep sensitive runtime values such as `DATABASE_URL`, `REDIS_URL`, and `SECRET_KEY` in environment variables or Kubernetes Secrets, not in published documentation.

Problems and Solutions

Problem	Root cause	Resolution	Status
AWS access must not depend on static access keys.	Static credentials are high-risk for CI/CD.	GitHub OIDC was introduced for role assumption.	Completed



Problem	Root cause	Resolution	Status
ECR push requires authorization permissions.	The GitHub deployment role needs ECR token and repository permissions.	ECR build/push workflow and later permission verification commits were added.	Partially completed
Deployment needs deterministic image names.	Manually passing image URIs is error-prone.	Workflow builds/pushes SHA-tagged images and deployment scripts consume those values.	Completed
EKS access can be blocked by limited permissions.	Deployment role may lack cluster or Kubernetes RBAC permissions.	EKS access smoke workflow and later rollout tolerance were added.	Partially completed

Testing, Build and Deployment Results

Area	Result	Evidence
OIDC workflow	Implemented	.github/workflows/aws-oidc-smoke-test.yml existed in commit 2777b56; later consolidated into CI/CD.
ECR build/push workflow	Implemented	Commit 9f9fcf3 added AWS ECR build/push workflow.
Production deploy workflow	Implemented	Commit 7eb1a76 added scripts/ci/deploy-eks-pipeline.sh and scripts/ci/deploy-frontend.sh.
AWS CLI/runtime logs	Partially completed	I did not find local aws sts, aws ecr, or policy simulation output in the local evidence archive.

Evidence

Commits and Pull Requests

Commit	Description	Evidence	Pull Request
2777b56	Added AWS OIDC smoke test workflow.	View commit	Evidence pending
9f9fcf3	Added ECR build and push workflow.	View commit	Evidence pending
7eb1a76	Aligned AWS deployment with production architecture.	View commit	Evidence pending
bfacd3a	Added EKS access smoke test workflow.	View commit	Evidence pending
404ba6c	Reran deployment path with ECR verify permission.	View commit	Evidence pending

Test Logs

Evidence pending: attach actual output from:

```
aws sts get-caller-identity
aws ecr describe-repositories --repository-names internship-backend internship-chat
aws ecr describe-images --repository-name internship-backend
aws ecr describe-images --repository-name internship-chat
```

Build Logs

Evidence pending: attach GitHub Actions or terminal logs showing Docker image build, tag, push, and digest output.



Deployment Logs

Evidence pending: attach OIDC, ECR, or EKS smoke workflow logs. Do not include secrets or database URLs.

Weekly Results

My Week 6 result was the cloud deployment identity and image-delivery foundation used by the whole team. The teammates remained responsible for their service code and image correctness; I owned the OIDC, IAM, ECR, and deployment-workflow integration.

Lessons Learned

AWS CI/CD work depends on both IAM and runtime evidence. A workflow file can be correct structurally, but I only mark authentication, image push, or cluster access as verified when the corresponding AWS/Actions log is available.

Next Week Plan

Deploy and validate the managed AWS runtime: EKS cluster, namespace, RDS PostgreSQL, ElastiCache Redis, DynamoDB tables, SQS queue/DLQ, IRSA runtime role, and ALB ingress.

1.7 Week 7

Week 7 - Amazon EKS and Managed AWS Services

Objectives

Within the five-member team, my Week 7 objective was to integrate the production AWS runtime: EKS, managed databases, runtime IAM, SQS transport, public ALB routing, and safe deployment automation. Teammates supported service-level configuration and functional checks for the components they owned.

My Contributions

Status	My assigned contribution	Evidence basis
Completed	Added production deployment pipeline logic for EKS, ALB, and CloudFront support.	Commit 899ff3b.
Completed	Added no-domain ALB ingress path for /api, /chat, and /socket.io.	k8s/eks/ingress-alb-no-domain.yaml.
Completed	Added EKS deployment script behavior for secrets, config, migration/init jobs, rollouts, health checks, and optional AI service.	scripts/k8s/deploy-eks.sh.
Completed	Added rollout-only workflow mode for existing workloads.	Commits 51bceee and f81e086.
Completed	Hardened public ingress deployment around AWS Load Balancer Controller readiness and ALB health checks.	Commits 036a516 and 8272c4a.

Technical Implementation

The target production environment is centered on EKS namespace `internship`. Backend and chat are long-running Kubernetes Deployments. PostgreSQL, Redis, DynamoDB, SQS, S3, CloudFront, and SageMaker are managed AWS services outside the cluster.

Flow summary. CloudFront forwards application traffic through the public ALB to the backend and chat services. The backend uses RDS, S3, and the SQS outbox queue; chat uses ElastiCache and DynamoDB; the processing worker calls the AI service and SageMaker, while the dispatcher publishes events to SQS.

Runtime variables such as `DATABASE_URL`, `REDIS_URL`, `OUTBOX_QUEUE_URL`, and `AWS_REGION` are treated as deployment inputs or Kubernetes secret/config values. The report does not include their secret values.

Problems and Solutions

Problem	Root cause	Resolution	Status
Public ingress needed to work before a custom domain was available.	Domain and certificate readiness can lag behind backend deployment.	Added <code>ingress-alb-no-domain.yaml</code> with path-based ALB routing.	Completed
AWS Load Balancer Controller may not be ready when ingress is applied.	Webhook endpoints can be unavailable during controller rollout.	Public ingress script waits and retries before applying ingress.	Completed
Health checks can fail briefly while target groups converge.	ALB registration and pod readiness are eventually consistent.	Public deploy script retries ALB health checks.	Completed
EKS rollout may run with limited permissions.	IAM/RBAC permissions may not allow every optional inspection command.	Rollout script tolerates selected limited-permission cases without hiding actual failures.	Completed

Testing, Build and Deployment Results

Area	Result	Evidence
EKS deployment script	Implemented	<code>scripts/k8s/deploy-eks.sh</code> validates required variables, applies resources, waits for jobs, rolls out deployments, and performs health smoke checks through port-forwarding.
Public ingress script	Implemented	<code>scripts/aws/deploy-public-ingress.sh</code> waits for controller readiness and polls <code>/api/health/ready</code> and <code>/chat/health/ready</code> .
Rollout mode	Implemented	<code>scripts/aws/rollout-eks-workloads.sh</code> supports restart/scale behavior for backend, chat, dispatcher, and processing worker.
Managed AWS resource proof	Partially completed	<code>PROJECT_CONTEXT.md</code> lists current resource names, but screenshots/CLI logs should be attached as evidence.

Evidence

Commits and Pull Requests

Commit	Description	Evidence	Pull Request
899ff3b	Added production AWS deployment pipeline, EKS scripts, ALB no-domain ingress, and CloudFront helper.	View commit	Evidence pending
51bceee	Added EKS rollout workflow mode.	View commit	Evidence pending
f81e086	Tolerated limited EKS permissions during rollout.	View commit	Evidence pending
036a516	Waited for ALB webhook before public ingress.	View commit	Evidence pending
8272c4a	Retried ALB health checks during public deploy.	View commit	Evidence pending

Test Logs

Evidence pending: attach actual output from:

```
aws eks describe-cluster --name internship-prod --region ap-southeast-1
aws rds describe-db-instances --db-instance-identifier internship-prod-postgres --region
→ ap-southeast-1
aws elasticache describe-replication-groups --replication-group-id internship-prod-redis
→ --region ap-southeast-1
aws sqs get-queue-attributes --queue-url <OUTBOX_QUEUE_URL> --attribute-names All
kubectl get serviceaccount -n internship
kubectl describe serviceaccount internship-runtime -n internship
```

Build Logs

Evidence pending: attach EKS deployment workflow logs that show the image tags selected for backend, chat, and optional AI service.

Deployment Logs

Evidence pending: attach kubectl rollout status, kubectl get pods -n internship, and ALB health-check output.

Weekly Results

My Week 7 result was an integrated AWS deployment path for the team's services: EKS workload rollout, ALB routing, managed-service configuration, and operational deployment controls. This does not imply individual ownership of the application features running on that infrastructure.

Lessons Learned

Production Kubernetes deployment depends on more than manifests. IAM, controller readiness, target group convergence, network routing, health endpoints, and secret handling must all be validated separately.

Next Week Plan

Run the full build, push, deploy, frontend, SageMaker, and operational-validation path; then document final completed items, partial items, and cost-control actions.

1.8 Week 8

Week 8 - End-to-End Deployment and Operational Validation

Objectives

During Week 8, I finalized my Cloud/DevOps contribution to the five-member project. I integrated and validated the production deployment model for the teammates' services across Amazon EKS, Application Load Balancer routing, S3 and CloudFront frontend delivery, AWS-managed data services, event-driven processing, and cost tracking.

I used the Week 08 AWS evidence directory as the source of truth for this section. The cost figures come from the July 1-28, 2026 AWS cost summary, the Billing credits screenshot, and the Cost Explorer overview screenshot. The finalized monthly bill and the AWS Pricing Calculator estimate are still unavailable.

My Contributions

Status	My assigned contribution	Evidence basis
Completed	Integrated the AI member's SageMaker-oriented service into the deployment path while preserving worker-facing routes.	Source references from the project implementation and <code>sagemaker-health.png</code> .
Completed	Fixed deploy-app image propagation and deterministic ECR image selection.	Source references from the CI/CD workflow implementation.
Completed	Removed frontend Kubernetes resources from the current runtime path so frontend delivery uses S3 and CloudFront instead of an EKS frontend pod.	Source references from the deployment scripts and architecture notes.
Completed	Verified Amazon RDS PostgreSQL and Amazon ElastiCache Valkey runtime status from AWS CLI screenshots.	<code>RDS-health.png</code> , <code>REDIS-health.png</code> .

Status	My assigned contribution	Evidence basis
Completed	Verified the SageMaker endpoint runtime status from AWS CLI screenshot evidence.	sagemaker-health.png.
Partially completed	Verified EKS workloads and ALB runtime state from AWS CLI screenshots.	EKS workloads were running, but the ALB target health screenshot showed mixed unhealthy targets.
Completed	Added CloudFront, S3, DynamoDB, and SQS runtime evidence.	Sanitized screenshots: Cloudfront-evidence.png, S3-evidenc.png, DynamoDB-evidenc.png, SQS-evidence.png.
Partially completed	Added AWS cost evidence for July 1-28, 2026.	July cost summary, total-cost-july.png, and Cost-evidence.png; Pricing Calculator remains blocked.

Technical Implementation

The final deployment model separates static frontend hosting from long-running service workloads. Backend, chat, worker, and AI services run on EKS. Static browser assets are intended to be served through S3 and CloudFront, while API, chat, and Socket.IO paths route through the ALB.

Flow summary. The production entry point is CloudFront: static frontend requests go to S3, while API and chat paths go through the ALB to EKS. Application data flows to RDS, Redis, and DynamoDB; asynchronous CV processing uses the worker, AI service, and SageMaker; outbox events reach SQS and the notification Lambda.

The CI/CD workflow supports deployment modes including `validate`, `deploy-app`, `deploy-public`, `deploy-frontend`, `rollout`, and `full`. Backend, chat, and AI images are tagged with the GitHub commit SHA. The frontend deployment path builds the Vite app, publishes static files to S3, and invalidates CloudFront when frontend deployment is enabled.

AWS Runtime Evidence

The runtime evidence below uses only files that I placed in the Week 08 AWS evidence directory. I reviewed the following files:

- ALB-target-health.png
- ALB-target.png
- Cloudfront-evidence.png
- Cost-evidence.png
- DynamoDB-evidenc.png
- eks-deployment-rollout-success.png
- eks-runtime-pods-2026-07-28.png
- lambda.png
- Lambda-evidence.png
- lambda-cloudwatch-evidence.png
- RDS-health.png
- REDIS-health.png
- S3-evidenc.png
- sagemaker-health.png
- SQS-evidence.png
- total-cost-july.png

I did not find `.txt`, `.log`, `.json`, `.yaml`, or `.yml` evidence files in this directory, so I did not include long CLI log excerpts.

Runtime Evidence Summary

AWS service	Evidence status	Runtime status	Evidence
Amazon EKS	Available	Partially verified: nodes, pods, jobs, and deployments were healthy; cluster ACTIVE status was not directly captured	eks-runtime-pods-2026-07-28.png, eks-deployment-rollout-success.png
Application Load Balancer	Available	Partially operational: load balancer was active and target groups existed, but target health was mixed and included unhealthy targets	ALB-target-health.png, ALB-target.png
Amazon CloudFront	Available	Partially verified: I captured monitoring metrics for requests, data transfer, and distribution error rates	Cloudfront-evidence.png
Amazon S3	Available	Verified frontend bucket objects after account-suffix redaction	S3-evidenc.png
Amazon RDS	Available	Available	RDS-health.png
Amazon ElastiCache	Available	Available	REDIS-health.png
Amazon DynamoDB	Available	Verified: chat tables and Lambda dedupe table are Active and on-demand	DynamoDB-evidenc.png
Amazon SQS	Available	Verified: main queue and DLQ exist, show zero visible/in-flight messages, and use SSE-SQS	SQS-evidence.png
AWS Lambda	Available	Partially verified: function configuration, source view, and CloudWatch log group exist; invocation result still not verified	lambda.png, Lambda-evidence.png, lambda-cloudwatch-evidence.png
Amazon SageMaker	Available	InService	sagemaker-health.png

Amazon EKS

Evidence status: Available **Runtime status:** Partially verified

I verified two Ready worker nodes, running application pods, completed migration/init jobs, and ready deployments in the internship namespace. The visible workloads include ai-service, backend, backend-outbox-dispatcher, backend-processing-worker, and chat-service.

The same evidence confirms deployment readiness:

- ai-service: 1/1 ready
- backend: 2/2 ready
- backend-outbox-dispatcher: 1/1 ready
- backend-processing-worker: 1/1 ready
- chat-service: 2/2 ready

I did not capture an `aws eks describe-cluster` output showing the EKS cluster status as ACTIVE. For that reason, I classify EKS as partially verified instead of claiming that full cluster-level evidence was captured.

Application Load Balancer

Evidence status: Available **Runtime status:** Partially operational

I captured an internet-facing Application Load Balancer in active state. However, the target health evidence still shows mixed target status:

- One backend target on port 8000 was healthy.
- Another backend target on port 8000 was unhealthy with failed health checks.
- Chat targets on port 3000 were unhealthy with timeout or failed health-check reasons.

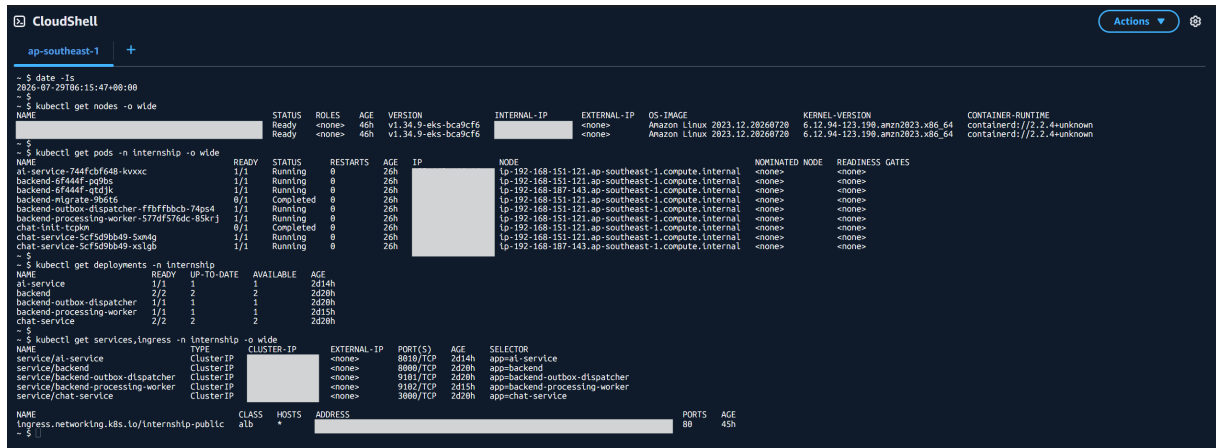


Figure 1: Amazon EKS runtime pods and services



Figure 2: Amazon EKS deployment rollout success

Because of the unhealthy targets, I do not classify the ALB as fully healthy. The load balancer was deployed, active, and associated with target groups, but the target groups still require follow-up validation and remediation.

Amazon CloudFront and S3

Evidence status: Available **Runtime status:** Partially verified

I captured CloudFront monitoring metrics for distribution EQIGYNECXDY8, including requests, data transfer, and error-rate graphs over the selected three-day window. This confirms that the distribution was receiving telemetry.

I captured the frontend S3 bucket object listing after redacting the bucket name. The visible objects include assets/, favicon.svg, and index.html, which verifies that static frontend assets were present in S3.

These screenshots support frontend hosting evidence, but they do not replace a full CloudFront behavior/origin configuration export or a browser smoke test through the CloudFront URL.

Amazon RDS

Evidence status: Available **Runtime status:** Available

I verified the PostgreSQL database instance internship-prod-postgres in status available. The captured output also includes PostgreSQL engine metadata, encrypted storage, non-public accessibility, and allocated storage.

Amazon ElastiCache

Evidence status: Available **Runtime status:** Available

I verified the ElastiCache replication group internship-prod-redis with Valkey engine status available. The captured evidence also confirms encryption at rest and in transit.

Amazon DynamoDB

Evidence status: Available **Runtime status:** Verified Active

I verified four DynamoDB tables in Active status:

- ChatGroups
- ChatMessages
- ChatUsers
- InternshipLambdaEventDedupe

```

$ aws elbv2 describe-load-balancers \
  --region ap-southeast-1 \
  --query 'LoadBalancers[0].Name,LoadBalancerScheme,State,Type,Scheme,DNSName' \
  --output table
DescribeLoadBalancers
+-----+-----+-----+-----+-----+-----+
| DNS | Name | Scheme | State | Type |
+-----+-----+-----+-----+-----+
| k8s-internshpublic-48181358ad | k8s-internshpublic-48181358ad | internet-facing | active | application |
+-----+-----+-----+-----+-----+

$ for arn in $(aws elbv2 describe-target-groups \
  --region ap-southeast-1 \
  --query 'TargetGroups[0].TargetGroupArn' \
  --output text); do
  aws elbv2 describe-target-health \
    --region ap-southeast-1 \
    --target-group-arn $arn \
    --query 'TargetHealthDescriptions[0].TargetId,TargetPort,State,Reason' \
    --output table
done
DescribeTargetHealth
+-----+-----+-----+-----+
| Port | Reason | State | Target |
+-----+-----+-----+-----+
| 8080 | Target.FailedHealthChecks | unhealthy | [REDACTED] |
| 8080 | None | healthy | [REDACTED] |
+-----+-----+-----+-----+

DescribeTargetHealth
+-----+-----+-----+-----+
| Port | Reason | State | Target |
+-----+-----+-----+-----+
| 3000 | Target.TimedOut | unhealthy | [REDACTED] |
| 3000 | Target.FailedHealthChecks | unhealthy | [REDACTED] |
+-----+-----+-----+-----+

```

Figure 3: Application Load Balancer target health

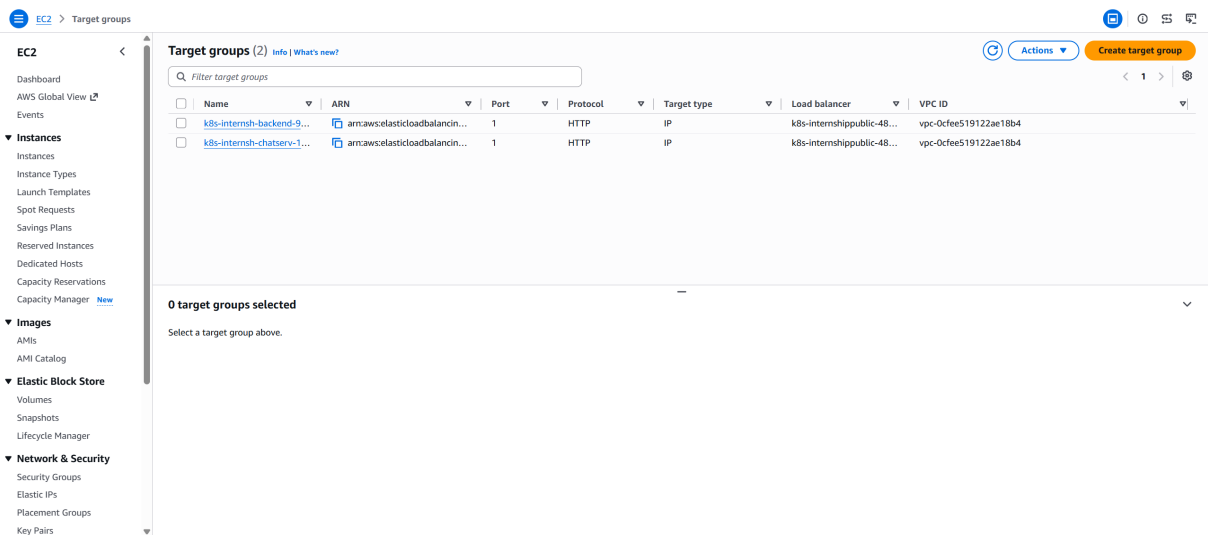


Figure 4: Application Load Balancer target groups

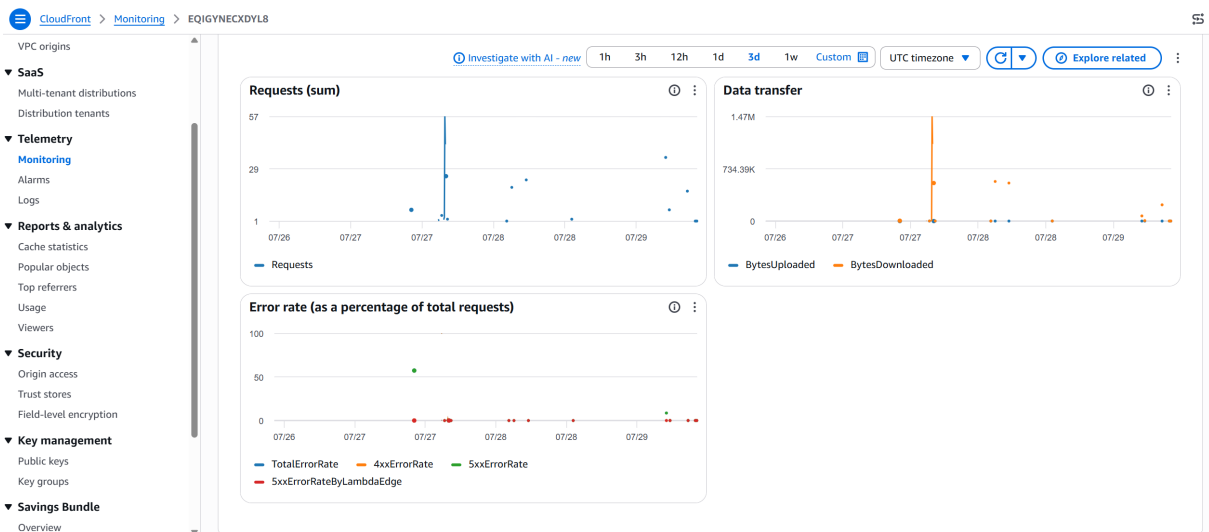


Figure 5: CloudFront monitoring evidence

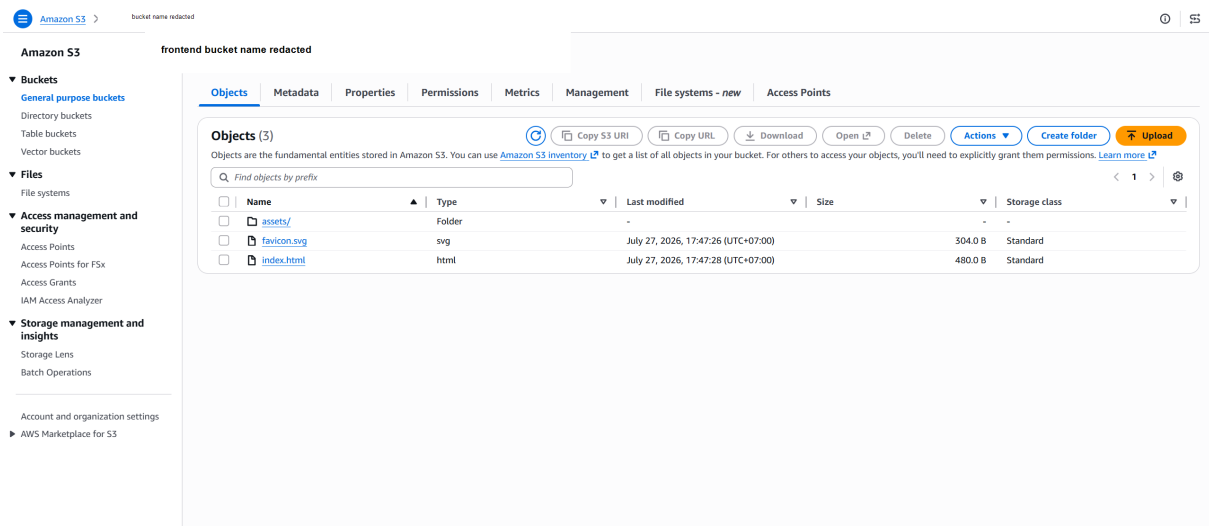


Figure 6: S3 frontend bucket object evidence

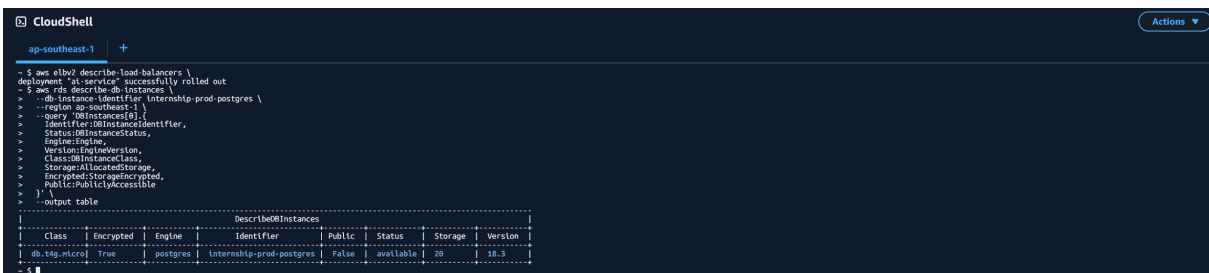


Figure 7: Amazon RDS PostgreSQL health

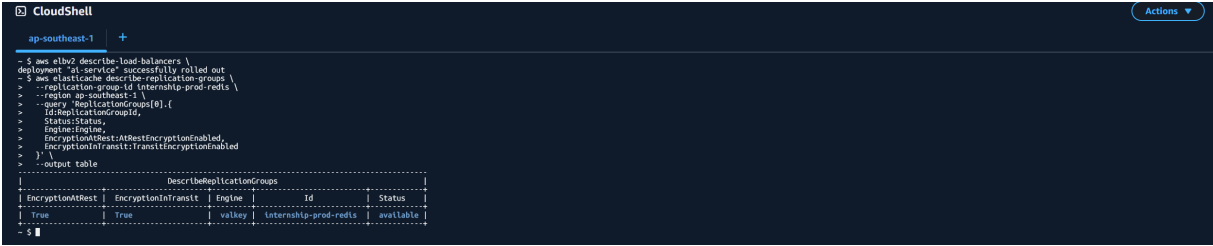


Figure 8: Amazon ElastiCache Valkey health

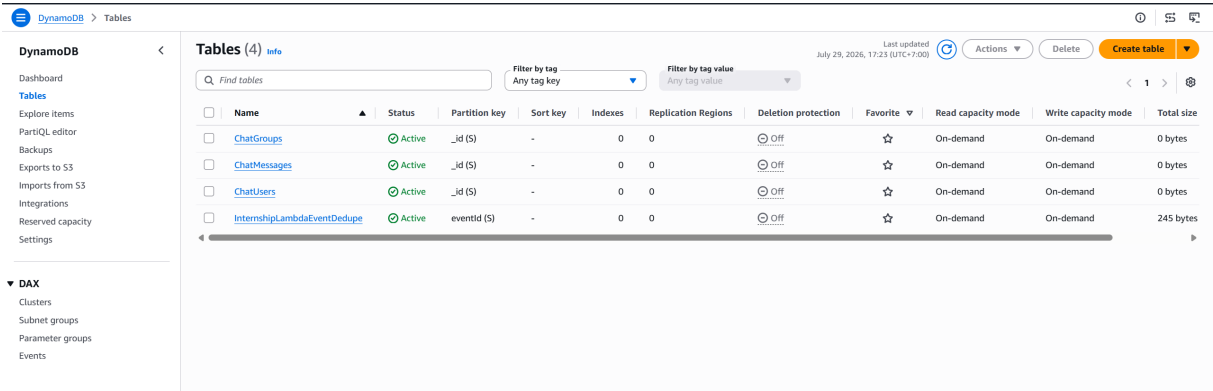


Figure 9: DynamoDB table status evidence

The captured table list also shows on-demand read and write capacity mode. This verifies that the AWS DynamoDB tables existed and were active at the time of evidence collection.

Amazon SQS

Evidence status: Available **Runtime status:** Verified available

I verified two Amazon SQS standard queues:

- internship-prod-outbox
- internship-prod-outbox-dlq

Both queues had zero visible messages and zero in-flight messages in the captured evidence. The queues use Amazon SQS managed server-side encryption.

AWS Lambda

Evidence status: Available **Runtime status:** Partially verified

I verified a Lambda function named `internship-outbox-handler` with a Python runtime and environment configuration. This confirms that the function existed in the selected AWS region.

The screenshot was sanitized before publication by redacting account-scoped ARNs, the IAM role ARN account segment, the bucket value containing an account identifier, and the sender email address.

Additional Lambda evidence captures the function source view and a CloudWatch log group named `/aws/lambda/internship-outbox-handler`. These items confirm that the function code view and log group existed.

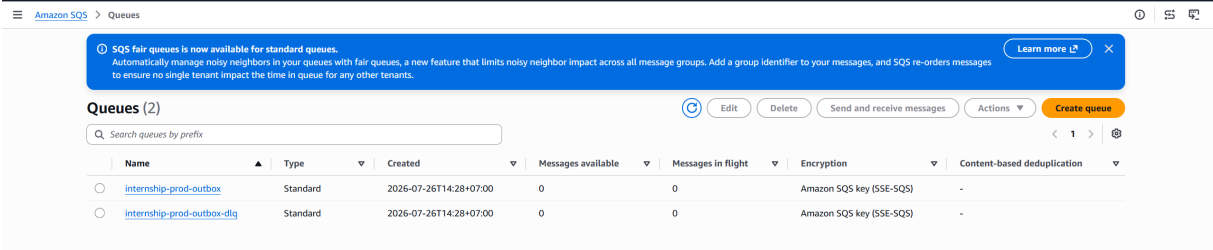


Figure 10: SQS queue evidence

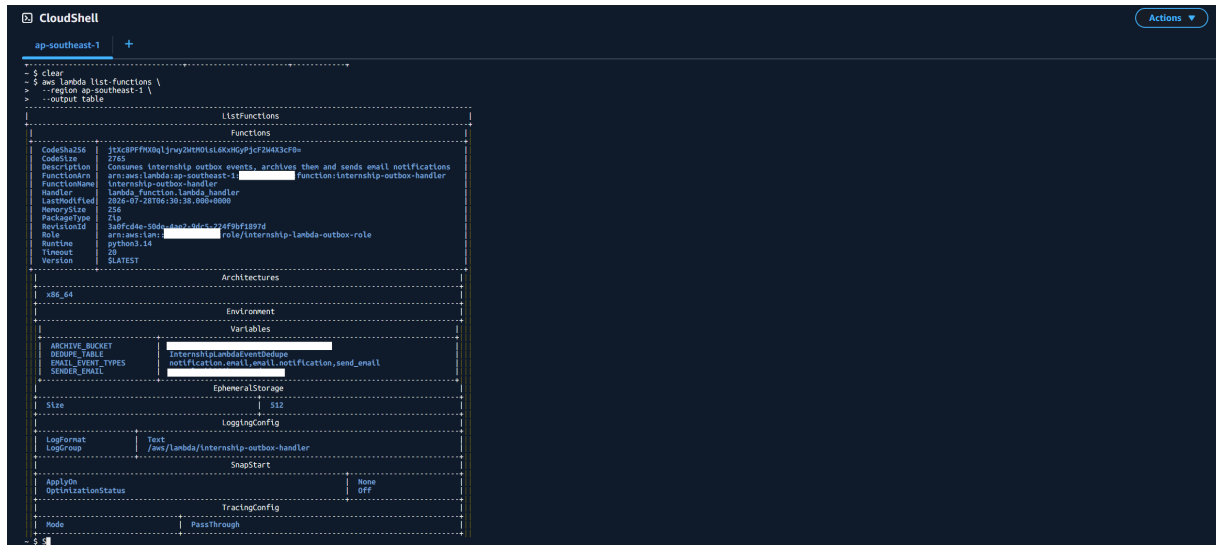


Figure 11: AWS Lambda function configuration

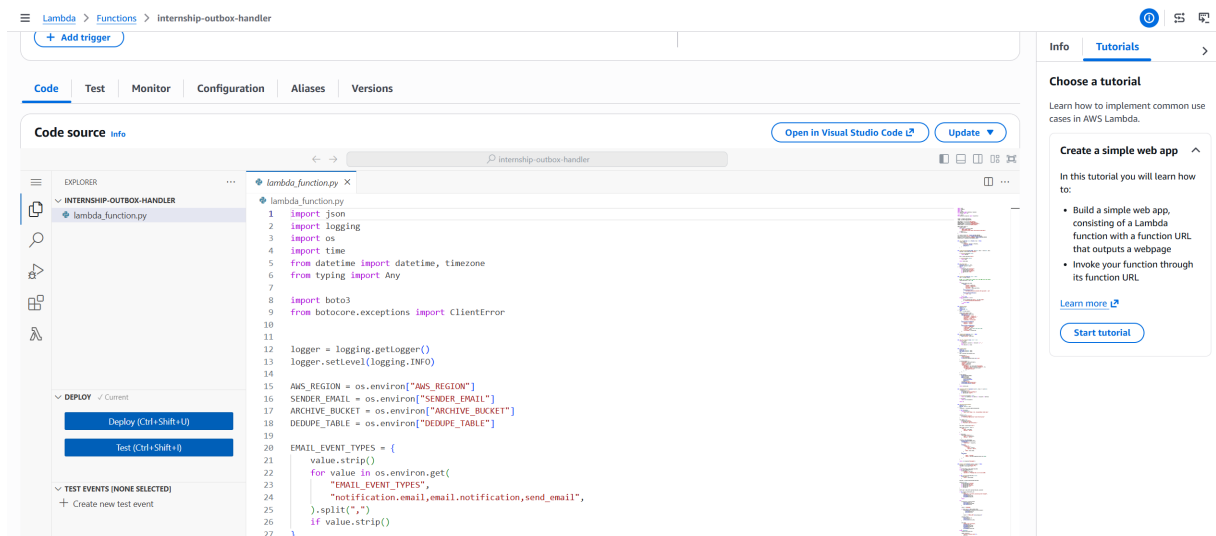


Figure 12: AWS Lambda source evidence

They do not prove a successful invocation, SQS trigger execution, or SES delivery result, so I keep Lambda runtime execution as partially verified.

Amazon SageMaker

Evidence status: Available **Runtime status:** InService

I verified the SageMaker endpoint `internship-qwen3-4b` in status `InService`. This is sufficient to show that the endpoint existed and was serving at the time of evidence collection.

AWS Cost Evidence

Status: Partially available

The July 1-28, 2026 AWS cost summary reported a month-to-date spend of **\$94.92**. I also included a Billing and Cost Management credits screenshot stored as `total-cost-july.png` and a Cost Explorer overview screenshot stored as `Cost-evidence.png`.

The credits screenshot records AWS credits rather than a grouped Cost Explorer service chart. The Cost Explorer screenshot is useful as a billing-console overview, but it uses a different report configuration and is not a Pricing Calculator estimate. Therefore, I keep the service-level cost breakdown tied to the July 1-28 cost summary.

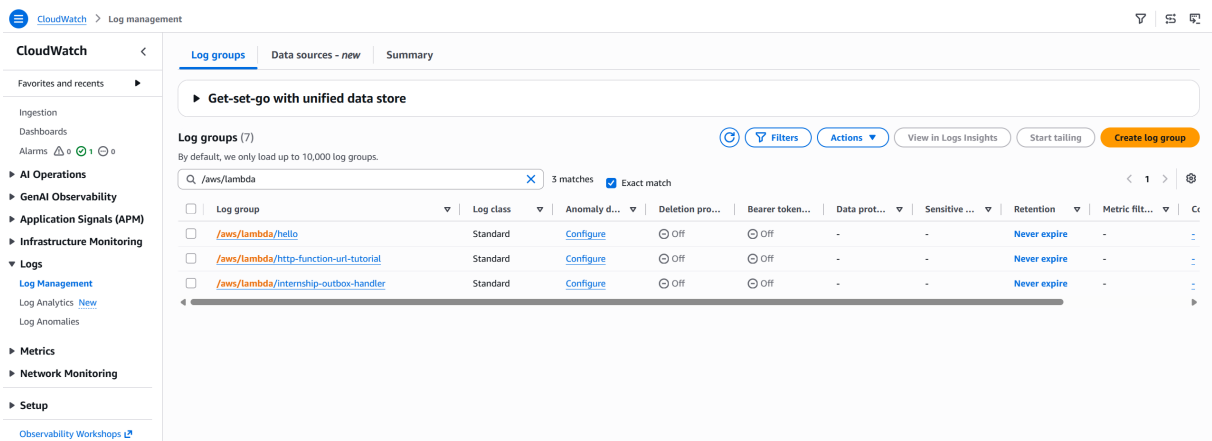


Figure 13: AWS Lambda CloudWatch log group evidence

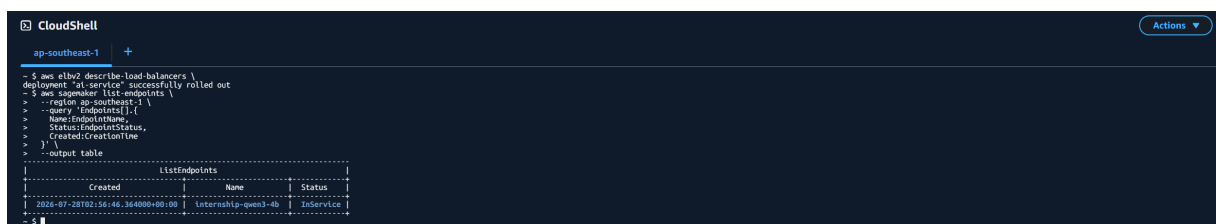


Figure 14: Amazon SageMaker endpoint status

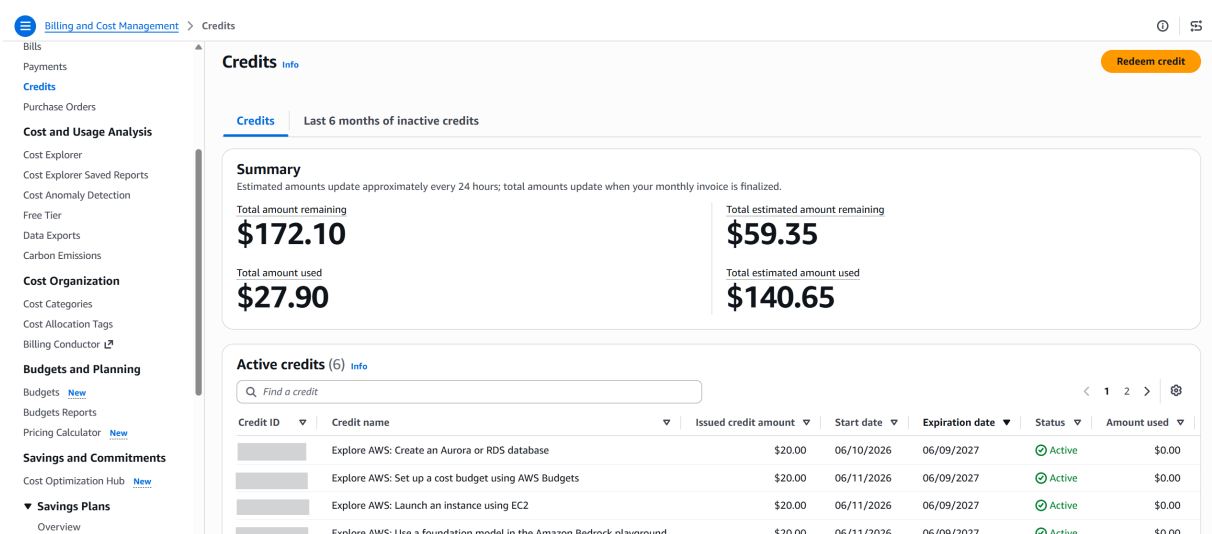


Figure 15: AWS Billing credits summary for July 2026

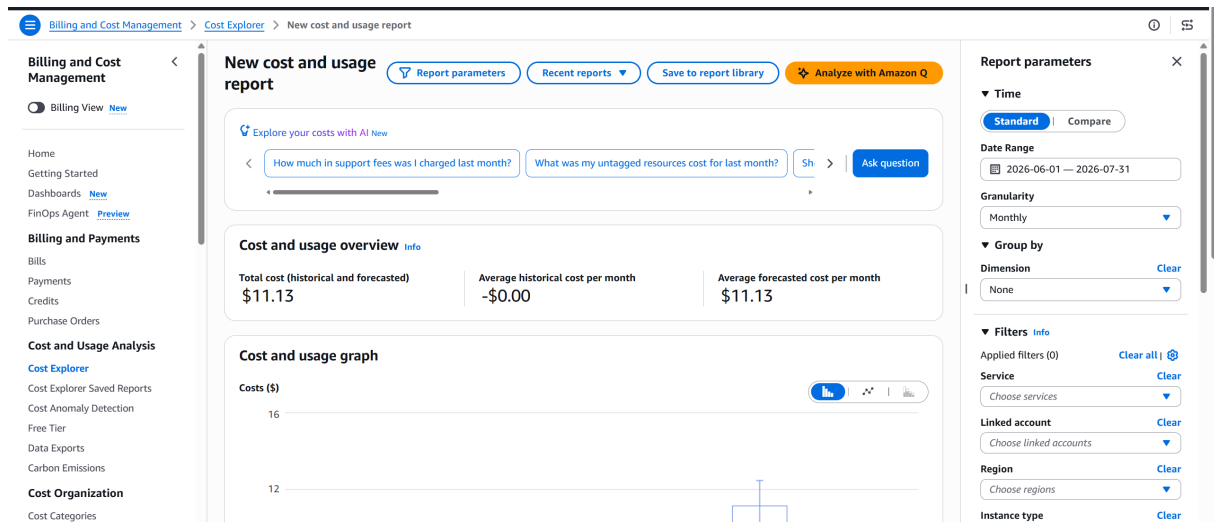


Figure 16: AWS Cost Explorer overview evidence

July 1-28 cost summary

Metric	Reported value
Total spend, July 1-28	\$94.92
Highest daily spend	\$31.83 on July 28
AWS credits total amount used	\$27.90
AWS credits total estimated amount used	\$140.65
AWS credits total amount remaining	\$172.10
AWS credits total estimated amount remaining	\$59.35

Daily cost trend

Period	Daily spend
July 1-8	Approximately \$2.20/day
July 9-25	Approximately \$1.60/day
July 26	\$5.61
July 27	\$11.60
July 28	\$31.83

The July 28 daily cost was the largest reported daily spend in the July 1-28 period and was much higher than the earlier baseline.

Top cost drivers

Rank	Service	Month-to-date cost	Share of total
1	Amazon RDS	\$29.69	31.3%
2	Amazon SageMaker	\$23.45	24.7%
3	Amazon VPC	\$14.11	14.9%
4	Amazon EC2 - Compute	\$12.42	13.1%
5	EC2 - Other	\$7.68	8.1%
6	Amazon EKS	\$5.64	5.9%

These six services account for approximately 98% of the July 1-28 reported spend.

Cost anomalies

Five active anomalies were reported around July 26-28 in ap-southeast-1:

Service	Anomaly impact	Reported context
Amazon EBS	+\$6.02, +15,050% versus expected	NAT Gateway activity in ap-southeast-1
Amazon EC2 - Compute	+\$2.56, +400% versus expected	New t3.medium instances in ap-southeast-1
Amazon RDS	+\$1.12, +138% versus expected	New db.t4g.micro instance activity in ap-southeast-1
Amazon VPC	+\$0.47, +98% versus expected	VPC/NAT Gateway infrastructure in ap-southeast-1
AWS Secrets Manager	+\$0.01, +50% versus expected	Minor secret-management cost tied to new infrastructure

Cost evidence	Status	Reported value
July 1-28 cost summary	Available	\$94.92
AWS Billing credits screenshot	Available	\$27.90 total amount used; \$140.65 total estimated amount used
Cost Explorer overview screenshot	Available	Included as console evidence; not used as the final service-level breakdown
AWS Bills finalized monthly invoice	Pending	Not available
AWS Pricing Calculator	Blocked	Not available
Actual month-to-date cost	Reported for July 1-28	\$94.92
Estimated monthly cost	Not calculated	N/A

Testing, Build and Deployment Results

Area	Result	Evidence
EKS workload rollout	Partially verified	I captured successful rollout output for all listed deployments.
EKS runtime pods and services	Partially verified	I captured Ready nodes, Running pods, Completed jobs, ready deployments, and services/ingress.
ALB deployment	Partially operational	I captured ALB state active, but target health still includes unhealthy targets.
RDS PostgreSQL	Verified available	I captured RDS status available.
ElastiCache Valkey	Verified available	I captured ElastiCache status available.
SageMaker endpoint	Verified InService	I captured SageMaker endpoint status InService.
Lambda function listing	Partially verified	Screenshots confirm function configuration, source view, and CloudWatch log group; invocation and trigger health are not verified.
CloudFront and S3 frontend delivery	Partially verified	CloudFront monitoring and S3 object listing screenshots are available; full behavior/origin export and browser smoke proof remain pending.
DynamoDB and SQS	Verified available	DynamoDB tables are Active; SQS main queue and DLQ are listed with zero messages and SSE-SQS.

Area	Result	Evidence
Cost evidence	Partially available	July 1-28 cost summary, credits screenshot, and Cost Explorer overview are available; finalized bill and Pricing Calculator remain pending/blocked.

Problems and Solutions

ALB target groups were not fully healthy

Problem: I captured the load balancer in active state, but target health still included unhealthy backend and chat targets.

Impact: API or chat traffic through the public ALB could fail intermittently or return gateway errors until every required target group becomes healthy.

Current resolution: I classify the ALB as partially operational rather than fully healthy.

Next action: Collect updated target health evidence after fixing health-check failures or deployment readiness issues.

AWS cost evidence is available but not complete

Problem: A July 1-28 cost summary and credits screenshot are available, but the evidence set still does not include a finalized monthly bill or AWS Pricing Calculator estimate.

Impact: I can show the July 1-28 month-to-date spend and major cost drivers, but I cannot conclude the final monthly bill or steady-state monthly cost yet.

Current resolution: I classify cost evidence as partially available, report only the July 1-28 figures, and keep monthly estimate fields pending.

Next action: Collect the finalized AWS Bills view and build an AWS Pricing Calculator estimate for the production architecture.

I added runtime evidence for several AWS services

Problem: The first Week 8 evidence set did not include CloudFront, S3, DynamoDB, or SQS screenshots.

Impact: Without screenshots or sanitized CLI output, I could not classify those services as runtime evidenced.

Current resolution: Sanitized screenshots were added for CloudFront monitoring, the S3 frontend bucket object list, DynamoDB table status, and SQS queue status.

Next action: For final production evidence, add CloudFront behavior/origin configuration export and a browser smoke test through the CloudFront URL.

Lambda screenshot required sanitization

Problem: The Lambda screenshot contained account-scoped ARNs, an IAM role ARN, a bucket name containing an account identifier, and an email address.

Impact: Embedding the screenshot directly would publish sensitive or semi-sensitive operational values.

Current resolution: I sanitized and embedded the screenshot as Lambda deployment evidence. It confirms the function listing/configuration, but not invocation success.

Next action: Collect a separate invocation, trigger, or CloudWatch event-detail screenshot if Lambda runtime execution must be marked fully verified.

Weekly Results

My Week 8 contribution produced meaningful AWS runtime evidence for the core compute and data path:

- EKS workloads were running and deployments had rolled out.
- I verified RDS PostgreSQL as available.
- I verified ElastiCache Valkey as available.
- CloudFront monitoring and S3 frontend object evidence were added.
- DynamoDB tables and SQS queues were verified from AWS console screenshots.
- SageMaker endpoint evidence showed InService.
- Sanitized Lambda evidence confirms the function, source view, and CloudWatch log group, while invocation proof remains pending.

The broader end-to-end application remained a team result. In my scope, I also kept the remaining gaps explicit: ALB target health was not fully healthy, CloudFront behavior/origin export and browser smoke proof were still pending, and cost evidence still needed a finalized bill and Pricing Calculator estimate.

Remaining Work

- ☒ Add the July 1-28 AWS cost summary.
- ☒ Add the AWS Billing credits screenshot.
- ☒ Add a Cost Explorer overview screenshot.
- ☐ Capture the finalized AWS Billing summary.
- ☐ Build an AWS Pricing Calculator estimate. This is currently blocked.
- ☒ Capture CloudFront monitoring evidence.
- ☒ Capture S3 frontend bucket object evidence.
- ☒ Capture Amazon DynamoDB table status evidence.
- ☒ Capture Amazon SQS queue and message-count evidence.
- ☐ Capture updated ALB target health after every required target is healthy.
- ☐ Capture CloudFront behavior/origin configuration or browser smoke proof.
- ☒ Add a sanitized Lambda screenshot.

Lessons Learned

Implementation evidence and runtime evidence answer different questions. A deployment script proves that a path exists in source control, but screenshots, AWS CLI output, and health checks prove what actually ran in AWS.

Cost reporting needs the same discipline. The July 1-28 summary is useful month-to-date evidence, but the final monthly bill and steady-state monthly estimate still require separate billing and Pricing Calculator evidence.

2 Proposal

Internship Application Platform

An AI-integrated internship application management platform deployed on AWS

Project Summary

Internship Application Platform is a centralized platform that helps students, candidates, and companies manage the internship recruitment process in one system.

For Candidates, the system supports profile creation, internship opportunity discovery, application status tracking, CV and document management, direct communication with HR, and AI-based evaluation of the fit between a CV and a Job Description.

For HR teams and companies, the system supports company profile management, job posting, applicant review, recruitment status updates, direct communication with Candidates, and AI-assisted CV analysis, comparison, and ranking.

The project is designed as a multi-service architecture, including:

- A React and Vite frontend.
- A FastAPI REST backend.
- A chat service using Node.js, Socket.IO, Redis, and DynamoDB.
- An AI service for CV analysis, Job Description analysis, and candidate ranking.
- PostgreSQL for business data.
- Amazon S3 for CV and document storage.
- Docker and Kubernetes for standardized deployment environments.
- Observability for metrics, logs, and distributed traces.
- GitHub Actions for CI/CD and security checks.

The production architecture is planned for AWS using Amazon EKS, Amazon ECR, Amazon RDS for PostgreSQL, Amazon DynamoDB, Amazon ElastiCache, Amazon S3, Amazon CloudFront, Application Load Balancer, and suitable monitoring services.

Problem Statement

Current Problems

During the internship search process, students often manage application information with spreadsheets, personal notes, email, or several separate platforms. This approach creates several problems:

- Job and company information is scattered.
- It is difficult to track the status of each application.
- CVs, transcripts, certificates, and related documents are stored in multiple places.
- Candidates have limited support for evaluating how well their CV matches a Job Description.
- Communication with HR is not directly connected to the application record.
- Candidates can easily miss deadlines, interview schedules, or document requests.

For HR teams and companies, manual recruitment workflows also create limitations:

- CV reading and classification takes a significant amount of time.
- It is difficult to compare multiple candidates using consistent criteria.
- Recruitment status may not be updated consistently.
- Candidate data may be stored insecurely.
- Communication between HR and Candidates is scattered across multiple channels.
- There is no centralized tool for tracking recruitment activity and effectiveness.

Proposed Solution

Internship Application Platform provides a centralized system for both Candidates and HR.

Candidates can:

- Register and manage their accounts.
- Update personal profiles.
- Search and view job information.

- Submit applications.
- Track recruitment status.
- Upload and manage CVs, certificates, and transcripts.
- Communicate directly with HR.
- Use AI to analyze CV fit against a job.

HR teams and companies can:

- Create and manage company profiles.
- Post and edit job openings.
- View submitted applications.
- Update candidate recruitment status.
- Communicate directly with Candidates.
- Use AI to extract information, calculate fit scores, and support candidate ranking.

Long-running tasks such as document reading, CV analysis, and reranking are processed asynchronously through workers so that the main API is not blocked during processing.

Benefits and Value

For Candidates, the platform helps:

- Manage the full application process in one place.
- Reduce the risk of losing documents and job information.
- Track the status of each application clearly.
- Identify relevant skills and missing skills.
- Communicate with companies more conveniently.

For HR teams, the platform helps:

- Reduce manual CV processing time.
- Centralize candidate information and recruitment status.
- Standardize the resume evaluation process.
- Support data-driven decision-making.
- Improve recruitment tracking and coordination.

For the development team, the project provides an opportunity to apply full-stack development, Cloud architecture, AI integration, realtime communication, Kubernetes, CI/CD, observability, and system security knowledge.

Solution Architecture

The original proposal targeted a multi-service AWS architecture with Kubernetes for containerized services, managed databases, object storage, realtime chat, observability, and CI/CD. In the final implementation, I kept that direction and adjusted two important areas after completing the deployment work:

- The React frontend is no longer a Kubernetes workload. It is built with Vite, stored in a private S3 bucket, and delivered by CloudFront.
- Backend, chat, worker, outbox dispatcher, and the SageMaker adapter remain on EKS because they are long-running processes or need Kubernetes rollout, health check, and scaling controls.

Final implemented architecture

Flow summary. CloudFront serves the private S3 frontend and routes API, chat, and Socket.IO traffic to the ALB. EKS hosts the backend, chat, dispatcher, worker, and AI services, which use RDS, Redis, DynamoDB, S3, SQS, Lambda, SES, and SageMaker. GitHub Actions deploys images, workloads, frontend assets, and CloudFront changes through OIDC-based AWS access.

Architecture explanation

CloudFront is the single public entry point for users. Static assets use the default CloudFront behavior and are read from the private frontend S3 bucket through CloudFront Origin Access Control. Dynamic API and realtime routes are sent to the public Application Load Balancer:

- `/api/*` is rewritten by the ALB Ingress and routed to the FastAPI backend.

- `/chat/*` is rewritten and routed to the chat service.
- `/socket.io/*` is routed to the chat service for Socket.IO transport.

The EKS cluster hosts only services that need long-running runtime behavior: backend, chat, outbox dispatcher, processing worker, and the AI adapter. PostgreSQL is the transactional database for users, jobs, applications, workflow state, async processing jobs, outbox records, and idempotency records. DynamoDB stores permanent chat entities and Lambda event deduplication state. Redis is used only for Socket.IO pub/sub between chat replicas. SQS decouples committed business events from notification processing, and Lambda performs short event-driven work such as deduplication, S3 archive, and SES email delivery.

Component responsibility table

Component	Final responsibility	Implementation evidence
React/Vite frontend	Candidate and HR user interface, built to static assets	<code>frontend/package.json</code> , <code>scripts/ci/deploy-frontend.sh</code>
CloudFront	HTTPS public entry point and route dispatcher	<code>scripts/aws/ensure-cloudfront.sh</code> , Week 8 distribution <code>EQIGYNECXDYL8</code>
Frontend S3 bucket	Private storage for <code>frontend/dist</code> assets	Private frontend bucket from the Week 8 deployment evidence; account suffix redacted
Application Load Balancer	Public routing to EKS services	<code>k8s/eks/ingress-alb-no-domain.yaml</code>
FastAPI backend	Auth, jobs, applications, uploads, dashboards, processing job APIs	<code>backend/app/main.py</code> , backend routers
Chat service	REST chat APIs and Socket.IO realtime channel	<code>chat-service/server.js</code>
Processing worker	Claims asynchronous processing jobs and writes results	<code>backend/app/workers/processing_worker.py</code> , <code>k8s/app/backend-processing-worker.yaml</code>
Outbox dispatcher	Publishes committed PostgreSQL outbox events to SQS	<code>backend/app/workers/outbox_dispatcher.py</code> , ADR-001
AI service	Stable worker-facing adapter for SageMaker	<code>ai_service/app.py</code> , <code>k8s/app/ai-service.yaml</code>
RDS PostgreSQL	Transactional business data and reliable worker queues	Alembic migrations through <code>0008_async_processing_jobs.py</code>
DynamoDB	Chat persistence and Lambda event deduplication	Chat table names in Kubernetes config and Week 8 runtime evidence
Redis	Socket.IO pub/sub between chat pods	<code>chat-service/lib/redis.js</code> , Kubernetes config
SQS and DLQ	At-least-once event delivery and failed-message isolation	ADR-001 and Week 8 queue evidence
Lambda and SES	Event notification, archive, and email delivery	Lambda configuration and CloudWatch evidence from Week 8
GitHub Actions OIDC	CI/CD without long-lived AWS access keys	<code>.github/workflows/cicd.yml</code>

Design rationale

The frontend was moved from EKS to S3 and CloudFront because it is static after build time. This reduces Kubernetes workload count, removes the need for a frontend Deployment, Service, HPA and PDB, and lets CloudFront handle caching and SPA delivery.

The backend and chat service remain in EKS because they are long-running APIs with health probes, replicas, HPA, PDB, and rolling deployment needs. The processing worker also remains in EKS because CV parsing, job parsing, and matching jobs can run longer than a short Lambda-style task and need queue leases, retries, and controlled concurrency.

The AI service isolates SageMaker-specific inference logic from the worker contract. The worker continues to call stable routes such as `/parse-job`, `/parse-cv`, and `/match-applications`, while the AI adapter handles SageMaker endpoint invocation, timeout, retry, and response normalization.

The transactional outbox is used because committing application data and publishing an event are separate failure domains. Events are first inserted in PostgreSQL in the same transaction as the business mutation. A dispatcher later sends them to SQS. This provides at-least-once delivery, and Lambda uses DynamoDB conditional writes to deduplicate eventId.

Proposal vs final implementation

Area	Original proposal	Final implemented architecture
Frontend hosting	Could be served through Kubernetes or AWS static hosting	Implemented as private S3 plus CloudFront
Public entry point	ALB and/or CloudFront expected	CloudFront is the user-facing entry point; ALB is an origin for API/chat/socket paths
Backend runtime	Kubernetes on EKS	EKS Deployment backend, 2 replicas, HPA 2-5
Chat runtime	Node.js and Socket.IO with Redis/DynamoDB	EKS Deployment chat-service, 2 replicas, ALB stickiness, Redis pub/sub
AI runtime	AI service expected	EKS ai-service adapter invokes SageMaker endpoint internship-qwen3-4b
Event processing	Asynchronous events expected	PostgreSQL outbox, SQS, Lambda, DynamoDB dedupe, S3 archive and SES
Deployment	CI/CD expected	GitHub Actions workflow dispatch modes with OIDC and ECR image verification
Runtime proof	Design-stage assumption	I collected runtime evidence for CloudFront, EKS workloads, RDS, Redis, DynamoDB, SQS, Lambda configuration, and the SageMaker endpoint

AWS Services Used

AWS service	Role in the system	Reason for selection
Amazon EKS	Runs the backend, worker, chat service, and container workloads	Supports Kubernetes, scaling, rolling updates, and multi-service management
Amazon ECR	Stores container images	Integrates directly with EKS and GitHub Actions
Application Load Balancer	Receives and routes requests to services	Supports routing, health checks, and Kubernetes Ingress integration
Amazon RDS for PostgreSQL	Stores users, companies, jobs, applications, document metadata, and processing jobs	Suitable for relational data and transactions
Amazon DynamoDB	Stores chat users, conversations, and messages	Suitable for chat data with high read/write traffic
Amazon ElastiCache	Provides Redis pub/sub for Socket.IO and temporary data	Helps multiple chat-service instances synchronize events
Amazon S3	Stores CVs, certificates, transcripts, and frontend assets	Provides scalability, high durability, and presigned URL support
Amazon CloudFront	Distributes frontend and static assets	Improves access speed and supports HTTPS
Amazon SQS	Receives asynchronous events from the outbox dispatcher	Reduces direct coupling between services
Amazon SageMaker	Provides an AI inference endpoint in production	Supports managed model endpoints and scalability

AWS service	Role in the system	Reason for selection
Amazon CloudWatch	Collects AWS logs, metrics, and alarms	Supports operations and incident investigation
AWS IAM	Manages access between workloads and AWS resources	Supports the least privilege principle

Component Design

Frontend

The frontend is developed with React and Vite and provides interfaces for Candidates and HR. Static files can be built and stored on Amazon S3, then distributed through Amazon CloudFront. The frontend communicates with the backend and chat service through endpoints exposed by the Application Load Balancer.

Backend API

The backend is developed with FastAPI and is responsible for user authentication, Candidate and HR management, company and job management, application and document metadata management, presigned URL generation, processing job creation, dashboard data, and business APIs.

Processing Worker

The processing worker performs long-running tasks such as CV content extraction, Job Description analysis, CV analysis, matching score calculation, and candidate reranking. Lease, retry, and attempt-limit mechanisms are used to reduce failures when a worker stops during processing.

Chat Service

The chat service uses Node.js, Express, and Socket.IO to support realtime communication. DynamoDB stores users, groups, and messages; the Redis adapter synchronizes events across multiple pods; sticky sessions help maintain Socket.IO connections when multiple replicas are running.

AI Service

The AI service analyzes CVs and Job Descriptions, converts text into structured data, and supports fit scoring. The AI service should include schema validation, timeout, limited retry, health checks, log control to avoid exposing CV data, and fallback behavior when the model is unavailable.

Data Storage

PostgreSQL stores relational data that requires transactions. DynamoDB stores chat data. Amazon S3 stores large files. Redis supports realtime event distribution. Using multiple storage technologies allows each data group to be managed with a technology that matches its access pattern.

Observability

The system collects metrics for performance and health monitoring, logs for event and error investigation, and distributed traces to follow requests across services. Prometheus, Grafana, Loki, OpenTelemetry, and Tempo are used in Kubernetes environments; Amazon CloudWatch is used for AWS resource and log monitoring.

Technical Implementation

Implementation Phases

Phase	Main activities
1. Analysis and design	Analyze the Candidate-HR problem, define functional and non-functional requirements, design the database, design the multi-service architecture, and identify required AWS services
2. Business platform foundation	Build authentication, Candidate/HR profiles, company management, job management, application tracking, and document management
3. Realtime and AI integration	Build the chat service, integrate Socket.IO, store chat data in DynamoDB, integrate Redis adapter, build the AI service, analyze CV/JD content, and implement matching and reranking
4. Reliability improvements	Build processing workers, retry, lease, idempotency, optimistic concurrency, transactional outbox, validation, and error handling

Phase	Main activities
5. Containerization and Kubernetes	Write Dockerfiles, build Docker Compose, create a local Kubernetes cluster with kind, deploy Deployment, Service, Ingress, HPA, PDB, health probes, and migration jobs
6. Observability and CI/CD	Collect metrics, logs, and traces, build dashboards, set up alerts, build GitHub Actions, run automated tests, smoke tests, and security scans
7. AWS deployment and completion	Build images, push to Amazon ECR, deploy workloads on Amazon EKS, connect RDS, DynamoDB, ElastiCache, and S3, deploy frontend on S3/CloudFront, run end-to-end tests, and complete the documentation

Technical Requirements

Component	Technology or requirement
Frontend	React, Vite, Tailwind CSS
Backend	Python, FastAPI, SQLAlchemy, Alembic
Chat service	Node.js, Express, Socket.IO
AI service	Python, FastAPI, Qwen-compatible model or SageMaker endpoint
Relational database	PostgreSQL
Chat database	DynamoDB
Cache/pub-sub	Redis or Amazon ElastiCache
Object storage	Amazon S3
Containers	Docker
Container orchestration	Kubernetes, kind, and Amazon EKS
CI/CD	GitHub Actions
Monitoring	Prometheus, Grafana, and CloudWatch
Logs	Loki and CloudWatch Logs
Tracing	OpenTelemetry and Tempo
Security	JWT, IAM Role, OIDC, least privilege, and secret management

Timeline and Milestones

I adjusted the plan to 8 weeks, from 08/06/2026 to 30/07/2026, to match the worklog timeline.

Week	Time	Milestone	Expected deliverable
1	08/06/2026 - 14/06/2026	Requirements analysis, architecture, backend foundation, and authentication	Project scope, high-level architecture, registration, login, and initial migration
2	15/06/2026 - 21/06/2026	Candidate-HR platform, applications, documents, and S3	Company, jobs, application flow, secure upload, and document download
3	22/06/2026 - 28/06/2026	Frontend and REST API integration	Candidate/HR interfaces, protected routes, and API integration
4	29/06/2026 - 05/07/2026	Realtime chat	Realtime communication between Candidate and HR using Socket.IO, Redis, and DynamoDB
5	06/07/2026 - 12/07/2026	AI integration and reliable processing	CV/JD analysis, matching, worker, idempotency, concurrency, and outbox



Week	Time	Milestone	Expected deliverable
6	13/07/2026 - 19/07/2026	Docker Compose	Full stack runs locally with smoke test procedure
7	20/07/2026 - 26/07/2026	Kubernetes and observability	System runs on kind with metrics, logs, traces, and alerts
8	27/07/2026 - 30/07/2026	CI/CD, AWS, and reporting	End-to-end testing, AWS deployment preparation, and completed documentation

Budget Estimation

Exact monthly cost should be calculated with AWS Pricing Calculator and then compared with representative billing data for the project AWS account in `ap-southeast-1`. For this report, the Week 8 AWS evidence is treated as the source of truth for current cost status.

Week 8 AWS cost evidence

The Week 8 evidence reports July 1-28, 2026 month-to-date AWS spend and a Billing and Cost Management credits screenshot. The finalized monthly bill and AWS Pricing Calculator estimate are still pending.

Cost evidence	Observed value	Interpretation
July 1-28, 2026 total spend	\$94.92	Current month-to-date spend from the Week 8 cost summary
Highest daily spend	\$31.83 on July 28	Largest daily cost in the July 1-28 period
Credits total amount used	\$27.90	Billing credits screenshot evidence
Credits total estimated amount used	\$140.65	Billing credits screenshot evidence
Credits total amount remaining	\$172.10	Billing credits screenshot evidence
Credits total estimated amount remaining	\$59.35	Billing credits screenshot evidence

The top Week 8 cost drivers are Amazon RDS (\$29.69), Amazon SageMaker (\$23.45), Amazon VPC (\$14.11), Amazon EC2 - Compute (\$12.42), EC2 - Other (\$7.68), and Amazon EKS (\$5.64). Together they account for approximately 98% of the reported July 1-28 spend.

Cost assumptions

Area	Evidence-based assumption	Estimate status
Environment	Production deployment in <code>ap-southeast-1</code>	Verified
Public traffic	CloudFront fronts browser traffic and routes API/chat traffic to ALB	Verified
Frontend	Static assets are stored in a private S3 bucket and delivered by CloudFront	Verified
Kubernetes	One EKS cluster runs backend, chat, worker, dispatcher, and AI adapter workloads	Verified
Worker nodes	Two Ready Kubernetes worker nodes were captured in the evidence snapshot	Instance type and EBS size still required
RDS PostgreSQL	Two private, encrypted PostgreSQL db. <code>t4g.micro</code> instances with 20 GiB storage each	Verified
ElastiCache / Valkey	One available Redis-compatible replication group with encryption at rest and in transit	Node type/count still required

Area	Evidence-based assumption	Estimate status
DynamoDB	Chat and Lambda dedupe tables use on-demand billing and are near empty in the snapshot	Verified
SQS	Main outbox queue and DLQ have zero visible messages in the snapshot	Verified
Lambda	Outbox handler is Active, 256 MB memory, 20 second timeout	Verified
SageMaker	Real-time endpoint is InService with one production variant	Instance type and expected uptime still required
CloudWatch	Logs and metrics exist through AWS and Kubernetes services	Retention and ingestion volume still required

Cost-estimation methodology

1. Use AWS Pricing Calculator for ap-southeast-1.
2. Enter the observed deployed resources from the evidence folder.
3. Add missing runtime inputs: node instance type, EBS size, Redis node type/count, SageMaker instance type, monthly runtime hours, data transfer, request volume, and CloudWatch log retention.
4. Export the Pricing Calculator estimate as the forecast.
5. Compare the forecast with Cost Explorer after the environment has run for a representative period.

Service	Monthly estimate method	Current evidence status
Amazon EKS	Cluster hourly price times monthly runtime	Deployed; Week 8 cost summary reports \$5.64
EC2 worker nodes	Node hourly price times two nodes, plus EBS storage	Nodes verified; EC2 Compute reports \$12.42, EC2 - Other reports \$7.68
Amazon RDS PostgreSQL	db.t4g.micro hourly price, 20 GiB storage per instance, backup/I/O	Two private encrypted instances verified; Week 8 cost summary reports \$29.69
Amazon ElastiCache / Valkey	Node hourly price times node count, plus data transfer	Replication group verified; node type/count needed
Application Load Balancer	ALB hourly charge plus LCU usage	Active ALB verified; included in networking/runtime cost checks
Amazon CloudFront	Requests, transfer out, and invalidations	Monitoring screenshot available; behavior/origin export still required
Amazon S3	Storage GB, PUT/GET requests, lifecycle transitions	Frontend bucket object screenshot available
Amazon DynamoDB	On-demand read/write requests plus storage	Table status screenshot available
Amazon SQS	Standard queue requests and payload volume	Queue and DLQ screenshot available
AWS Lambda	Requests plus GB-seconds	Function exists; invocation cost and trigger health still pending
Amazon SES	Email send count and attachments if any	Used by notification path
Amazon ECR	Image storage and transfer if applicable	Not separated in the Week 8 top-driver table
Amazon SageMaker	Endpoint instance hours and invocation/data charges	Endpoint verified InService; Week 8 cost summary reports \$23.45
Amazon CloudWatch	Log ingestion, storage retention, metrics, alarms	Retention and ingestion volume still required
Amazon VPC and data transfer	CloudFront egress, ALB traffic, NAT processing, cross-AZ traffic	Week 8 cost summary reports Amazon VPC at \$14.11; traffic breakdown still needed

Service	Monthly estimate method	Current evidence status
Total	Pricing Calculator forecast, then billing comparison	Week 8 July 1-28 spend reports \$94.92; finalized bill and steady-state estimate pending

The largest current and future cost drivers are RDS, SageMaker endpoint uptime, VPC/NAT-related networking, EC2 worker nodes and EBS, EKS control plane, Redis, ALB, CloudFront/data transfer, and CloudWatch logs. If the SageMaker endpoint remains online continuously, AI inference can become one of the dominant monthly costs.

Evidence hygiene for cost materials

No raw screenshots or logs should expose account IDs, access keys, secret keys, database passwords, connection strings, Kubernetes Secret values, GitHub tokens, presigned URLs, or login-related logs. The pod log evidence must be summarized or sanitized before publication because the raw `02-eks/pod-logs-tail.txt` file contains a Redis connection URL with an embedded credential.

Cost Optimization

- Run the production environment only when demoing or testing.
- Use instance types that match the actual workload.
- Limit the number of worker nodes.
- Stop or delete SageMaker endpoints when not in use.
- Configure suitable log retention.
- Delete old images in ECR.
- Apply S3 lifecycle policies when appropriate.
- Use DynamoDB on-demand while traffic is still unstable.
- Configure AWS Budget and Billing Alarm.
- Delete unused Load Balancers, public IPv4 addresses, snapshots, and volumes.
- Confirm whether NAT Gateway is required before leaving it running.

Risk Assessment

Risk Matrix

Risk	Likelihood	Impact	Severity
Secret or AWS credential exposure	Low	Very high	High
RDS or services become public unnecessarily	Medium	High	High
CV and personal data are accessed without permission	Low	Very high	High
AI returns inaccurate results	Medium	Medium	Medium
AI service runs out of resources or times out	Medium	High	High
Realtime chat disconnects when scaled	Medium	Medium	Medium
Worker processes duplicate jobs	Medium	High	High
Database updates encounter race conditions	Medium	High	High
Kubernetes pod or node failure	Medium	Medium	Medium
AWS cost exceeds expectations	Medium	High	High
CI/CD deployment fails	Medium	Medium	Medium
Third-party package has a vulnerability	Medium	High	High

Mitigation Measures

Security

- Use IAM Roles and least privilege.
- Do not store AWS Access Keys in source code.
- Use GitHub Actions OIDC.
- Keep .env, private keys, and secrets out of the repository.
- Keep S3 buckets private.
- Use presigned URLs with expiration.
- Do not log full CV content or tokens.
- Check authorization and object ownership in the backend.

Reliability

- Use readiness and liveness probes.
- Use HPA to support scaling.
- Use PDB to reduce disruption during maintenance.
- Use limited retry.
- Apply processing-job leases.
- Use idempotency keys.
- Apply optimistic concurrency.
- Use transactional outbox for events.

AI

- Validate output with schemas.
- Set timeouts.
- Limit retries.
- Allow human review before recruitment decisions.
- Do not use AI scores as the only hiring decision.
- Record reasons or contributing factors when possible.

Cost

- Configure AWS Budget.
- Monitor Cost Explorer.
- Delete resources after the workshop.
- Limit AI endpoint runtime.
- Use suitable log retention.
- Check for remaining EBS volumes, Elastic IPs, Load Balancers, and snapshots.

Contingency Plan

- If EKS is not ready, use Docker Compose or local Kubernetes for the demo.
- If the AI model is unavailable, use mock responses or deterministic scoring to continue testing the business flow.
- If Redis is unavailable, run a single chat-service instance for the demo environment.
- If S3 cannot be accessed, use local storage in development.
- If a worker fails, processing jobs are retried with lease and attempt limits.
- If a new deployment fails, roll back to the previous container image or Kubernetes revision.
- If cost exceeds the limit, stop non-essential workloads and delete high-cost resources.

Expected Outcomes

After completion, the system is expected to achieve the following outcomes:

- Provide a centralized platform for Candidates and HR.
- Manage companies, jobs, applications, and documents.
- Store CVs and documents securely.

- Support realtime chat.
- Support AI analysis for CVs and Job Descriptions.
- Process long-running tasks with workers.
- Control repeated requests and concurrent updates.
- Run with Docker Compose and Kubernetes.
- Provide a CI/CD pipeline.
- Provide metrics, logs, traces, and alerts.
- Provide an AWS deployment architecture.
- Provide documentation for deployment, testing, and cleanup.

Long-Term Value

The project can continue evolving into a more complete system with:

- Email or notification when application status changes.
- Interview scheduling.
- AI-generated cover letters.
- Interview questions generated from CVs and Job Descriptions.
- Job recommendations based on Candidate profiles.
- Recruitment analytics dashboard.
- Calendar and email integration.
- Improved explainability for AI matching.
- Mobile application support.
- Expansion to multiple universities and companies.

Beyond product value, the project also provides a reference for building a Cloud-native multi-service system with AI integration, realtime communication, asynchronous processing, Kubernetes, observability, and CI/CD.

3 Blogs Posted

This section lists and introduces the blogs I have posted or prepared for the [AWS Study Group](#). The blog entries are organized into three small sections, following the same structure as the workshop sample.

Blog 1 - WHY DOES AWS STILL CHARGE AFTER STOPPING EC2?

This blog explains why AWS may still generate charges even after an EC2 instance has been stopped, focusing on related billable resources that may continue to exist outside the running instance itself.

Published on: July 22, 2026

Platform: AWS Study Groups

Status: Approved

Public link: [Blog 1](#)

Blog 2 - BUILDING A SCALABLE REAL-TIME CHAT SYSTEM WITH SOCKET.IO, REDIS, AND DYNAMODB

This blog introduces the design of a scalable real-time chat system using Socket.IO, Redis, and DynamoDB. It focuses on how real-time communication, distributed state, and persistent storage can work together in a larger system.

Published on: July 23, 2026

Platform: AWS Study Groups

Status: Pending

Blog 3 - A Few Takeaways from My Software Engineering Journey: From Writing Code to Breaking the System and Fixing It Myself

This article reflects on the journey from being someone who mainly focused on writing code to becoming a Software Engineer with a broader understanding of how systems operate, fail, and recover. Through working with backend, frontend, Docker, Kubernetes, CI/CD, IAM, databases, message queues, and AWS, the author realized that code running successfully in a local environment is not enough. A stable system also depends on configuration, networking, permissions, data, infrastructure, and failure-handling mechanisms. The main lesson is to debug systematically, identify the actual point of failure instead of making random changes, and design systems so that the same problems are less likely to happen again.

Published date: July 28, 2026

Platform: AWS Study Groups

Status: Pending

Public link: [Blog 3](#)

Blog 4 - AWS Networking Advanced: 3 Techniques for Building an Enterprise-Grade VPC

This article presents three techniques for upgrading a basic VPC architecture into a design that is more suitable for enterprise systems. It focuses on using VPC Endpoints to reduce traffic through NAT Gateways, deploying infrastructure across multiple Availability Zones to improve availability and reduce unnecessary cross-AZ data transfer, and using AWS Transit Gateway to manage connections between multiple VPCs through a centralized network model. The article demonstrates how network architecture optimization can improve security, scalability, operational control, and AWS cost efficiency.

Published date: July 29, 2026

Platform: AWS Study Groups

Status: Published

Public link: [Blog 4](#)

3.1 Blog 1

WHY DOES AWS STILL CHARGE AFTER STOPPING EC2?

A common misunderstanding when learning AWS is: *EC2 is stopped = AWS stops charging*.

However, through posts and messages in the group, I have seen many people still receive bills even when their instance is in the *Stopped* state, or even after it has been *Terminated*.

One real case was a friend of mine who accidentally left a NAT Gateway running after doing a lab and received a bill of nearly \$250.

The issue is that EC2 is only one part of the system. The resources created together with it do not always disappear automatically.

Stopping EC2 only stops virtual machine compute charges

When an EC2 instance is in the Stopped state, AWS stops charging for the compute part. However, the EBS disk is still kept so the machine can be started again later, so the storage capacity is still charged.

A simple way to understand it:

- Stop = Turn off the machine but keep the disk.
- Terminate = Delete the machine, but it is still necessary to check whether the EBS volume is configured to be deleted with the instance.

NAT Gateway still charges even when EC2 is stopped

NAT Gateway is a separate resource in a VPC, so it does not stop automatically with EC2.

AWS charges based on both:

- The number of hours the NAT Gateway exists.
- The amount of data passing through it.

For example, in AWS pricing for some regions, a NAT Gateway may cost around \$0.045 per hour (reference cost), equivalent to more than \$32 per month, even with almost no traffic.

If data passes through it, the cost increases further.

Elastic IP is not free either

If an Elastic IP has been allocated but not released, that address may continue to generate cost.

AWS currently charges for public IPv4 addresses whether they are in use or idle. One address may only cost a few dollars per month, but if many addresses are forgotten across multiple regions, the cost can add up significantly.

AWS Budget is not a hard spending limit

By default, AWS Budget is mainly used to send alerts. It does not automatically lock the entire account when spending reaches the configured amount.

If the system needs to respond automatically, Budget Actions must be configured, for example:

- Stop a specific EC2 or RDS resource.
- Attach an IAM policy to prevent creating more resources.
- Send alerts through email or SNS.

Reference checklist after each AWS lab

- Open Cost Explorer and group by Service and Usage Type.
- Check EC2 → Volumes and delete EBS volumes that are no longer needed.
- Check VPC → NAT Gateways.
- Release Elastic IPs that are no longer used.
- Check the correct region because resources in other regions can still generate cost.
- Enable AWS Budget and Cost Anomaly Detection from the beginning.

The most important lesson I learned is: when deleting a service on AWS, do not only ask: “Have I turned it off?” Also ask:

“What supporting resources did this service create, and are they still generating cost?”

#AWS #EC2 #CloudComputing #AWSCostOptimization

3.2 Blog 2

BUILDING A SCALABLE REAL-TIME CHAT SYSTEM WITH SOCKET.IO, REDIS, AND DYNAMODB

Realtime chat appears in many systems such as e-commerce, recruitment platforms, customer support, social networks, and collaboration tools.

At a basic level, a single server can receive and send messages. However, when the number of users grows, the system needs to answer more difficult questions:

- How can messages appear immediately?
- What happens when the sender and receiver are connected to different servers?

- Where should messages be stored so they are not lost when a server restarts?
- How can the number of servers increase without interrupting users?

A common architecture for solving these problems combines Socket.IO, Redis, and Amazon DynamoDB.

1. Why REST API is not enough for realtime chat

REST API follows a request-response model:

```
Client sends request
v
Server processes
v
Server returns response
```

If REST API is used for chat, the frontend usually needs to keep calling an API to check for new messages:

```
GET /messages
Wait a few seconds
GET /messages
Wait a few seconds
GET /messages
```

This approach is called polling.

Polling is simple, but it has several limitations:

- Message latency depends on the polling interval.
- The client sends requests even when there are no new messages.
- The backend has to process many unnecessary requests.
- System load increases quickly as the number of users grows.

For systems that require near-instant responses, WebSocket or a realtime library such as Socket.IO is usually more suitable.

2. Socket.IO for realtime communication

Socket.IO allows a two-way connection to be maintained between the client and the server.

After the connection is established, the client and server can send events to each other without creating a new HTTP request for every exchange.

A simple message flow can be understood as:

```
User A sends a message
v
Socket.IO server receives the event
v
Server processes the message
v
Server sends a new event
v
User B receives the message
```

Common events include:

- `conversation:join`
- `message:send`
- `message:new`
- `user:typing`
- `message:read`

Socket.IO also supports:

- Automatic reconnection when the network is interrupted.
- Grouping users into rooms.
- Sending events to one user or a group of users.
- Fallback when WebSocket is not available.

However, Socket.IO works best when the system has only one server. When the application runs multiple servers, an additional synchronization mechanism is required.

3. The issue when the chat service runs multiple instances

With one chat server, both users connect to the same server:

```
User A --+
        +-- Chat server
User B --+
```

When User A sends a message, the server can send it directly to User B's connection.

As traffic grows, the system may need multiple servers:

```
Chat server 1
Chat server 2
Chat server 3
```

A load balancer may distribute users like this:

```
User A $\rightarrow$ Chat server 1
User B $\rightarrow$ Chat server 2
```

At this point, Chat server 1 does not directly manage User B's connection.

Without communication between servers, User A's message may be saved successfully, but User B may not receive a realtime notification.

This is the problem Redis can solve.

4. Redis synchronizes events between chat servers

Redis is a high-speed in-memory data store. In realtime chat architecture, Redis is commonly used for publish/subscribe.

The flow works like this:

```
User A
v
Chat server 1
v Publish
Redis
v Subscribe
Chat server 2
v
User B
```

When Chat server 1 receives a message:

- The server processes and stores the message.
- The server publishes an event to Redis.
- Other chat servers receive the event.
- The server that holds the receiver's connection sends the message to the client.

Redis works like a shared broadcast channel between chat servers.

Redis is suitable for this role because:

- It has high processing speed.
- It supports publish/subscribe.
- It is suitable for short-lived data and events.
- It allows servers to operate independently while still exchanging events.

Redis does not have to be the long-term storage location for chat history. That task should be handled by a more durable database.

5. DynamoDB stores chat data

Amazon DynamoDB is a managed NoSQL database service from AWS.

Chat data often has these characteristics:

- The number of messages continuously increases.
- Write throughput can be high.
- Messages are usually queried by conversation.
- Complex joins are rarely needed.
- Usage can spike suddenly.

DynamoDB fits this model because it scales well and does not require users to manage database servers. A simple message table design can use:

- Partition key: `conversation_id`
- Sort key: `timestamp` or `message_id`

In this design:

- `conversation_id` groups messages from the same conversation.
- `timestamp` sorts messages by time.

When chat history is needed, the system queries records with the same `conversation_id`. The roles of Redis and DynamoDB can be separated clearly:

Redis

\$\rightarrow\$ Transmits realtime events between servers

DynamoDB

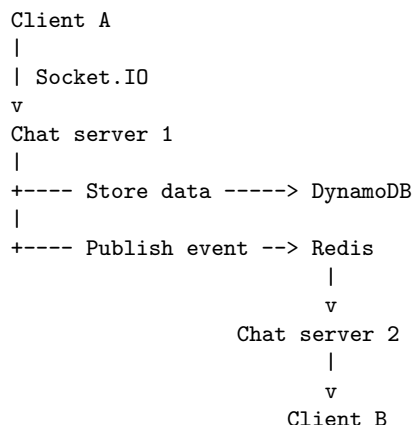
\$\rightarrow\$ Stores long-term chat history

6. Complete message sending flow

A message can pass through the system in these steps:

1. The user enters a message.
2. The client sends an event through Socket.IO.
3. The chat server authenticates the user.
4. The server checks whether the user can access the conversation.
5. The message is stored in DynamoDB.
6. The server publishes an event through Redis.
7. The chat server holding the receiver's connection receives the event.
8. The server sends the new message to the receiver's client.

General architecture:



This architecture achieves two goals:

- Realtime responses through Socket.IO and Redis.
- Durable message history through DynamoDB.

7. How Kubernetes supports scaling chat services

When the application is packaged as containers, Kubernetes can run multiple replicas of the chat service.

Chat service

```
+--- Pod 1
+--- Pod 2
+--- Pod 3
+--- Pod N
```

A Kubernetes Service or load balancer distributes connections to the pods.

Kubernetes also supports:

- Restarting containers automatically when failures occur.
- Rolling updates for new versions.
- Scaling the number of pods up or down.
- Health checks through liveness and readiness probes.
- Traffic distribution across multiple pods.
- Horizontal Pod Autoscaler based on CPU, memory, or custom metrics.

However, Kubernetes only runs and manages multiple instances. Realtime synchronization between instances still requires Redis or a suitable message broker.

8. Sticky session and Redis solve different problems

When Socket.IO uses HTTP long polling, a client can create multiple consecutive requests in the same session.

If these requests are routed to different servers, the connection session may fail.

Sticky session keeps requests from the same client on the same server:

```
Client A $\rightarrow$ Chat server 1
Client A $\rightarrow$ Chat server 1
Client A $\rightarrow$ Chat server 1
```

However, sticky session does not replace Redis.

The difference is:

- Sticky session keeps one client's connection stable on one server.
- Redis transmits events between different servers.

Even when sticky session is enabled, the sender and receiver may still be connected to different servers.

9. Deployment on AWS

On AWS, the architecture can use:

```
Application Load Balancer
v
Amazon EKS or container service
v
Multiple chat service instances
+--- Amazon DynamoDB
+--- Amazon ElastiCache/Valkey
```

Service roles:

- Amazon EKS: runs and manages chat service containers.
- Application Load Balancer: receives and distributes user connections.
- Amazon DynamoDB: stores chat data.
- Amazon ElastiCache or Valkey: provides a Redis-compatible service.
- Amazon CloudWatch: stores logs, metrics, and alarms.

For small projects or test environments, Redis and DynamoDB Local can be run with Docker before using real AWS services.

10. Common mistakes when building realtime chat

Storing messages only in server memory

If the server restarts, all data may be lost.

Server restart $\rightarrow$ Message history is lost

Messages should be stored in a durable database.

Running multiple servers without a Redis adapter

Each server only knows the clients directly connected to it. Users on another server may not receive events.

Using Redis as the only storage layer

Redis is suitable for cache and event transmission, but it is not always suitable as the only source of chat history.

Not authenticating Socket.IO connections

WebSocket or Socket.IO connections still need to be authenticated with JWT, session, or another appropriate mechanism.

Not checking conversation access

A logged-in user is not automatically allowed to join every chat room.

Not handling duplicate messages

When the network is unstable, the client may resend the same message. A client-generated message ID or idempotency key can help reduce duplicate records.

Not limiting message size and sending frequency

The system should include:

- Message length limits.
- Rate limiting.
- Spam prevention.
- File type validation if attachments are supported.

11. Cost optimization ideas

Realtime chat on AWS can generate cost from compute, load balancers, databases, Redis, and logs.

Some optimization ideas:

- Use DynamoDB On-Demand when traffic is low or unpredictable.
- Choose a suitable ElastiCache node instead of over-provisioning.
- Use Auto Scaling to avoid running too many instances during low traffic.
- Set retention periods for CloudWatch Logs instead of keeping logs forever.
- Use DynamoDB TTL for temporary data when the business rules allow it.
- Delete Load Balancers, ElastiCache clusters, or test environments after labs are completed.
- Run the system with Docker and DynamoDB Local during development.

Managed services reduce operational work, but resources still need to be monitored to avoid unexpected cost.

12. Reference checklist

- Use Socket.IO or WebSocket for realtime communication.
- Use a Redis adapter when running multiple chat servers.
- Store message history in a durable database.
- Design an appropriate DynamoDB partition key.
- Do not store important state only in server memory.
- Configure sticky session if using long polling.
- Authenticate realtime connections.
- Check conversation access permission.
- Add rate limiting and message size limits.
- Configure liveness and readiness probes.
- Monitor connection count, message rate, latency, and error rate.
- Configure Auto Scaling and resource limits properly.
- Check AWS resource cost after testing.

The most important lesson is that realtime chat is not only about setting up a WebSocket connection.

When the system starts to scale, three responsibilities should be separated clearly:

Socket.IO

\$\rightarrow\$ Realtime communication between client and server

Redis

\$\rightarrow\$ Event synchronization between multiple servers

DynamoDB

\$\rightarrow\$ Long-term chat data storage

Choosing the right tool for each responsibility helps the system respond quickly while remaining scalable as the number of users increases.

#AWS #SocketIO #Redis #DynamoDB #AmazonEKS #RealtimeChat #NodeJS #Kubernetes
#CloudComputing #AWSStudyGroup

3.3 Blog 3

A Few Takeaways from My Software Engineering Journey: From Writing Code to Breaking the System and Fixing It Myself

Introduction

I once believed that the main responsibility of a Software Engineer was to receive requirements, write code, complete features, and push the source code to a repository. However, after working directly on a relatively complete system, I realized that writing functional code is only the first step.

The more difficult challenge is ensuring that the code operates reliably within a real system containing multiple interconnected components, including a frontend, backend APIs, databases, message queues, containers, Kubernetes, CI/CD pipelines, and AWS infrastructure.

While developing the AI-Powered Internship Application Tracker, I worked with FastAPI, React, PostgreSQL, Redis, DynamoDB, Amazon SQS, Docker, Kubernetes, GitHub Actions, and several AWS services. This experience did not simply introduce me to more technologies. It also changed how I understand software development, deployment, failure, and recovery.

Running Locally Does Not Mean Running Successfully in Production

In a local environment, system components are usually close together and relatively easy to control. The database may run inside Docker Compose, Redis may share the same network, and environment variables may be stored in a local `.env` file.

When the system is moved to Kubernetes or AWS, communication becomes more complex. A service must use the correct DNS name, namespace, port, and network configuration to reach another service. A pod connecting to a database requires the correct connection string, security group, and network route. Applications in private subnets also need an appropriate outbound path to access external services.

This experience helped me understand that an application does not operate based on source code alone. Its reliability also depends on the environment surrounding it.

Docker and the Runtime Environment

Docker helps standardize the runtime environment across different machines, but containerization also introduces new types of problems.

A container may be running while its health check is failing. An application may start before its database becomes available. Environment variables may exist on the host machine but may not be passed correctly into the container. Port conflicts, missing volumes, Docker Desktop settings, and WSL integration can also prevent the system from operating even when the code has not changed.

These situations encouraged me to examine more than the application code:

- Which environment variables does the container actually receive?
- Are the required dependencies ready?
- Are the services connected to the correct network?
- Are ports exposed and mapped properly?
- What behavior is the health check validating?

CI/CD Is Not a Magical YAML File

GitHub Actions can automate linting, testing, image building, security scanning, and deployment. However, a pipeline works only when all of its assumptions are correct.

A workflow may fail because a secret is missing, a branch condition is incorrect, an image tag is inconsistent, or the runner does not contain the required tools. A deployment job may also be skipped because an environment variable or execution condition has not been configured correctly.

Through troubleshooting failed workflows, I learned that CI/CD is not simply about creating a configuration file. Developers need to understand what each step does, what data is passed between jobs, and what state the destination system must be in before deployment can proceed.

IAM and the Importance of Verifying Permissions

AWS IAM was one of the areas that required the most troubleshooting. Initially, I assumed that a missing permission could be fixed by adding a policy containing the required action.

In practice, AWS permissions also depend on trust policies, principals, conditions, repositories, branches, resource ARNs, and whether a particular action supports resource-level permissions.

A policy may appear secure and carefully restricted but still fail to operate. Some actions require the resource to be set to "*", while others can be limited to a specific ARN.

Instead of continuing to guess, I began using policy simulation tools to verify each action. The main lesson was straightforward:

Permissions should not be guessed. Each required action should be verified to determine whether it is allowed or denied.

Kubernetes: Running Does Not Always Mean Healthy

Kubernetes may report a pod as Running, but that status only confirms that the container currently exists. It does not guarantee that the application inside it is functioning correctly.

A running pod may still fail to connect to Redis, wait for an unfinished migration, return no API response, or lack outbound network access. Therefore, system validation should include more than pod status:

- Application logs.
- Readiness and liveness probes.
- Kubernetes Services and Ingress.
- Database migrations.
- Network connectivity.
- Dependent services.

This taught me not to rely only on infrastructure status. The actual behavior of the application must also be tested through endpoints and real user flows.

The NAT Gateway Incident and Network Architecture

One of the most memorable incidents occurred when I disabled the NAT Gateway to reduce operating costs. Afterward, pods in private subnets could no longer access the Internet, some connections to AWS services were interrupted, and deployment processes began to fail.

Because the source code, IAM configuration, and security groups had not changed, the cause was initially difficult to identify. After reviewing the network architecture, I discovered that the outbound route had disappeared when the NAT Gateway was disabled.

This incident gave me a much clearer understanding of public subnets, private subnets, Internet Gateways, NAT Gateways, route tables, and VPC Endpoints. Concepts that had previously existed only in architecture diagrams became practical knowledge after I experienced and resolved the failure myself.

Database Concurrency and Idempotency

Many backend problems do not appear when only one user is testing the system. They become visible when two or more requests arrive at nearly the same time.

For example, two requests may both check that a record does not exist and then attempt to insert it. Without proper constraints and transaction handling, duplicate data may be created.

To reduce these risks, the system can use:

- Database unique constraints.
- Transactions and rollback handling.
- HTTP 409 responses for conflicts.

- Optimistic locking.
- Idempotency keys for important operations.

An internship application should not be created twice simply because the user clicked a button multiple times or because the client retried the request.

Message Queues and the Transactional Outbox Pattern

Another challenge appears when a system must update a database and publish a message to a queue as part of the same logical operation.

If the database commit succeeds but message delivery fails, the data exists while the event is lost. Sending the message first is also unsafe because the database transaction may later fail.

The transactional outbox pattern addresses this issue by storing both the main data and its event within the same database transaction. A separate dispatcher then reads the outbox records and sends them to Amazon SQS. Failed deliveries can be retried, while consumers should support deduplication to prevent the same event from being processed repeatedly.

This is an example of an engineering effort that users may never see directly but that significantly improves system reliability.

A Systematic Debugging Approach

The most valuable lesson was not memorizing a large number of commands. It was learning how to divide a problem into smaller layers.

When an error occurs, I now begin with questions such as:

1. Is the issue located in the application or the infrastructure?
2. Did it occur before or after deployment?
3. Are the container and pod running?
4. Is the service exposing the correct port?
5. Can DNS resolve the required hostname?
6. Does the network provide the necessary route?
7. Does IAM allow the required action?
8. Are the database and other dependencies ready?
9. What was the last step that completed successfully?

Instead of changing several components at once, I try to identify the first point of failure in the processing chain. This approach makes troubleshooting more structured and reduces the time spent making random changes.

Conclusion

This journey significantly changed how I understand Software Engineering.

Previously, my first response to a system failure was to modify the code. I now understand that a failure may originate from configuration, environment variables, containers, networking, databases, permissions, infrastructure, or CI/CD.

I also consider questions beyond whether a feature simply works:

- What happens when a request is submitted twice?
- What happens if the database commits but the queue fails?
- How are simultaneous updates handled?
- How does the system recover when a pod restarts?
- Can a failed deployment be rolled back safely?

A Software Engineer is not someone who never causes a system failure. More importantly, a Software Engineer learns from each failure, understands the system more deeply, resolves issues systematically, and designs the system so that the same failure is less likely to occur again.

Publication Information

- **Topic:** Practical lessons from developing, deploying, and operating a software system.
- **Published date:** July 28, 2026.
- **Platform:** AWS Study Groups.
- **Status:** Pending.
- **Public link:** [Blog 3 on AWS Study Groups](#)

3.4 Blog 4

AWS Networking Advanced: 3 Techniques for Building an Enterprise-Grade VPC

Introduction

A basic Amazon VPC architecture consisting of public subnets, private subnets, and a NAT Gateway is a suitable foundation for deploying applications on AWS. This architecture separates resources that require direct Internet access from internal components such as application servers, databases, and caches.

However, when a system expands to include Development, Staging, and Production environments or is divided across multiple microservices and VPCs, the basic architecture may begin to present three major limitations:

- Increasing NAT Gateway costs caused by frequent access from private subnets to Amazon S3, Amazon DynamoDB, and other AWS services.
- A potential single point of failure when all outbound traffic depends on one NAT Gateway in one Availability Zone.
- Complex network management when many VPCs are connected through a full-mesh VPC Peering model.

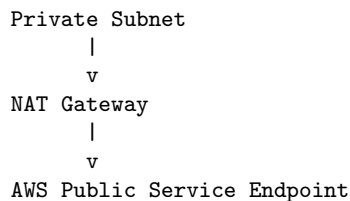
This article presents three techniques for upgrading a VPC architecture for enterprise environments: VPC Endpoints, Multi-AZ networking, and AWS Transit Gateway.

1. Reducing Costs and Improving Security with VPC Endpoints

The NAT Gateway Traffic Problem

EC2 instances, containers, and application servers running in private subnets usually do not have public IP addresses. When these resources need to access Amazon S3, DynamoDB, or other AWS services, their traffic may pass through a NAT Gateway.

A basic traffic flow may look like this:



This design may introduce hourly NAT Gateway charges and data-processing costs. Traffic may also pass through a public service endpoint even though both the application and destination service operate within AWS infrastructure.

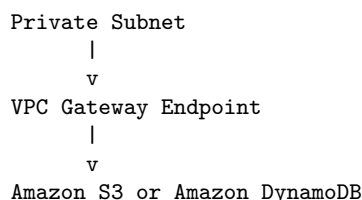
Gateway Endpoints

Gateway Endpoints are available for two primary services:

- Amazon S3.
- Amazon DynamoDB.

After a Gateway Endpoint is created and associated with the route tables of private subnets, traffic to S3 or DynamoDB can travel directly through the AWS network without using a NAT Gateway.

The updated traffic flow can be represented as follows:



Gateway Endpoints provide several benefits:

- A NAT Gateway is no longer required for S3 and DynamoDB traffic.
- Traffic remains within the AWS network.
- NAT Gateway data-processing costs can be reduced.

- Access can be controlled through endpoint policies.
- Workloads become less dependent on outbound Internet connectivity.

Gateway Endpoints for S3 and DynamoDB do not have hourly endpoint charges, which makes them one of the first options to consider when optimizing network costs.

Interface Endpoints and AWS PrivateLink

For AWS services that do not support Gateway Endpoints, an Interface Endpoint can be used.

An Interface Endpoint creates Elastic Network Interfaces inside selected subnets and assigns private IP addresses to them. Applications can then access the target AWS service through private IP addresses or private DNS without using the public Internet.

Services commonly accessed through Interface Endpoints include:

- Amazon CloudWatch.
- AWS Secrets Manager.
- Amazon ECR.
- AWS Systems Manager.
- AWS Key Management Service.
- Amazon Kinesis.

Interface Endpoints use AWS PrivateLink to keep service traffic inside the AWS network.

Compared with NAT Gateway access, this design can:

- Reduce outbound Internet traffic.
- Control access through Security Groups.
- Reduce the attack surface.
- Support a clearer private-networking model.
- Allow private workloads to access AWS services through private IP addresses.

Interface Endpoints still have hourly and data-processing charges. Their cost should therefore be compared with NAT Gateway costs based on the actual traffic patterns of the system.

2. Deploying Across Multiple Availability Zones and Avoiding Unnecessary Cross-AZ Costs

Why Multiple Availability Zones Matter

An Availability Zone can experience an independent failure. If all application servers, databases, and outbound networking resources are placed in one AZ, the entire system may become unavailable when that AZ is affected.

For higher availability, production architectures are commonly deployed across at least two Availability Zones.

For example:

```
Availability Zone A
- Public Subnet A
- Private Subnet A
- NAT Gateway A
- Application instances A
```

```
Availability Zone B
- Public Subnet B
- Private Subnet B
- NAT Gateway B
- Application instances B
```

In this model, each Availability Zone has an independent outbound path. If one AZ becomes unavailable, workloads in the remaining AZ can continue using the NAT Gateway and resources available in that AZ.

Avoiding a Single Point of Failure

If the system uses only one NAT Gateway in AZ-A, workloads in AZ-B must send their outbound traffic across Availability Zone boundaries before reaching the gateway.

This creates two issues:

1. The NAT Gateway in AZ-A becomes a shared point of dependency.
2. Traffic from AZ-B to AZ-A may generate Cross-AZ data-transfer charges.

A more resilient architecture deploys one NAT Gateway in every active Availability Zone. Each private subnet route table is configured to use the NAT Gateway located in the same AZ.

For example:

Private Subnet A -> NAT Gateway A
Private Subnet B -> NAT Gateway B

If AZ-A becomes unavailable, resources in AZ-B still have an independent outbound path and do not depend on NAT Gateway A.

AZ-Aware Routing

AZ-aware routing prioritizes communication between workloads and dependencies located in the same Availability Zone.

Examples include:

- An application instance in AZ-A using NAT Gateway A.
- A workload in AZ-B connecting to a cache node in AZ-B.
- A load balancer distributing requests to suitable targets.
- Database replicas or cache nodes being distributed across multiple AZs.
- Service discovery preferring endpoints located closer to the workload when supported.

This design can:

- Reduce unnecessary Cross-AZ traffic.
- Reduce network latency.
- Limit inter-AZ data-transfer costs.
- Allow each AZ to operate more independently.

However, AZ locality must still be balanced with High Availability requirements. An application should not depend entirely on one local resource if that dependency does not provide a failover mechanism.

3. Managing Multi-VPC Connectivity with AWS Transit Gateway

Limitations of VPC Peering

VPC Peering works well when only a small number of VPCs need to communicate. Two VPCs can be connected directly and exchange traffic through private IP addresses.

However, VPC Peering does not support transitive routing.

For example:

VPC-A <-> VPC-B
VPC-B <-> VPC-C

The fact that VPC-A connects to VPC-B and VPC-B connects to VPC-C does not automatically allow VPC-A to communicate with VPC-C.

When many VPCs require direct connections, the number of VPC Peering connections can grow according to the following formula:

$$N(N - 1) / 2$$

With 10 VPCs, a full-mesh design may require up to 45 peering connections.

At this scale, managing route tables, CIDR ranges, Security Groups, and network policies becomes increasingly difficult. Adding one new VPC may also require multiple new connections and route-table updates.

Transit Gateway as a Cloud Router

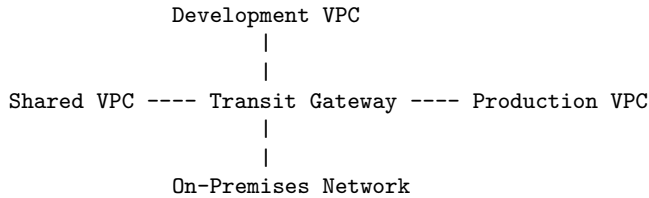
AWS Transit Gateway acts as a centralized router for multiple VPCs and external networks.

Resources that can connect to a Transit Gateway include:

- Amazon VPCs.
- Site-to-Site VPN connections.
- AWS Direct Connect through a Direct Connect Gateway.
- VPCs from multiple AWS accounts.
- Development, Staging, and Production environments.

- Shared Services VPCs.
- On-Premises data center networks.

Instead of creating direct connections between every pair of VPCs, each VPC creates a single attachment to the Transit Gateway.



This hub-and-spoke model simplifies the network topology and centralizes routing management.

Network Segmentation with Transit Gateway Route Tables

Transit Gateway can use multiple route tables to control which VPCs and networks are allowed to communicate. For example:

- The Development VPC can access the Shared Services VPC.
- The Production VPC can access the Shared Services VPC.
- The Development VPC cannot directly access the Production VPC.
- The On-Premises Network can access only selected subnets.
- A Security VPC can inspect traffic before forwarding it to other VPCs.

This segmentation can:

- Reduce lateral movement between environments.
- Separate Development, Staging, and Production more clearly.
- Centralize network-policy management.
- Simplify the addition of new VPCs.
- Support multi-account architectures with AWS Organizations.

Comparing Basic and Enterprise VPC Architectures

Category	Basic Architecture	Enterprise Architecture
Access to S3 and DynamoDB	Through a NAT Gateway or public endpoint	Through a Gateway Endpoint
Access to other AWS services	Commonly through a NAT Gateway	Through Interface Endpoints and AWS PrivateLink when appropriate
High Availability	May depend on one NAT Gateway	Independent NAT Gateway in each Availability Zone
Cross-AZ traffic	May increase because of inefficient routing	Reduced through AZ-aware routing
Multi-VPC connectivity	VPC Peering configured manually between pairs	Centrally managed through Transit Gateway
Development and Production separation	Depends on multiple routes and peering connections	Controlled through Transit Gateway route tables
Scalability	Suitable for smaller systems	More suitable for multi-VPC and multi-account systems
Management	Simple initially but difficult to control at scale	More complex initially but easier to manage centrally

When Should Each Technique Be Used?

Use VPC Endpoints when:

- Workloads in private subnets frequently access Amazon S3 or DynamoDB.

- Applications need to access AWS services without using the public Internet.
- NAT Gateway traffic and processing costs need to be reduced.
- The system has stronger data-security requirements.
- Access needs to be limited through endpoint policies.

Use Multi-AZ NAT Gateways when:

- The system requires High Availability.
- Workloads operate across multiple Availability Zones.
- A single NAT Gateway should not become a single point of failure.
- Unnecessary Cross-AZ traffic should be reduced.
- Outbound connectivity must remain available when one AZ fails.

Use AWS Transit Gateway when:

- The organization operates many VPCs or AWS accounts.
- On-Premises networks must connect to AWS.
- VPC Peering has become difficult to manage.
- Development, Staging, and Production require clear separation.
- Routing needs to be controlled from a central location.
- Shared Services or centralized Security Inspection architectures are required.

Cost Considerations

An advanced network architecture does not mean that every component becomes less expensive.

Gateway Endpoints for S3 and DynamoDB can reduce traffic through NAT Gateways. However, Interface Endpoints, NAT Gateways, and Transit Gateway each have their own pricing models.

Before implementation, the architecture should consider:

- Total monthly data-transfer volume.
- AWS services frequently accessed by private workloads.
- Number of Availability Zones.
- Number of VPCs and Transit Gateway attachments.
- Frequency of Cross-AZ communication.
- Internet outbound traffic.
- Security and availability requirements.
- Operational cost and management complexity.

The objective is not simply to minimize cost. It is to find an appropriate balance between cost, security, performance, scalability, and High Availability.

Conclusion

The combination of public subnets, private subnets, and a NAT Gateway provides a strong starting point for building applications on AWS. As the system grows across multiple environments, VPCs, or AWS accounts, the network architecture must also evolve.

Three important techniques are:

1. Use VPC Endpoints to keep AWS service traffic inside the AWS network and reduce dependence on NAT Gateways.
2. Deploy independent NAT Gateways across multiple Availability Zones to improve availability and reduce unnecessary Cross-AZ traffic.
3. Use AWS Transit Gateway to manage connectivity between multiple VPCs through a centralized model.

VPC optimization is not only a routing problem. It also involves balancing security, scalability, availability, performance, and cost.

A strong Enterprise VPC Architecture should be based on the actual requirements of the system. Additional services should not be introduced only to make the architecture appear more complex without providing clear value.

Publication Information

- **Topic:** Upgrading Amazon VPC architecture for enterprise systems.
- **Published date:** July 29, 2026.
- **Platform:** AWS Study Groups.
- **Status:** Published.
- **Public link:** [Blog 4 on AWS Study Groups](#)

4 Events Participated

During my internship, I followed three events related to AWS Cloud, enterprise architectures, system operations, and security, Agentic AI. These events helped me connect the project work with practical examples of how Cloud and AI technologies are designed, deployed, and operated in real environments.

No.	Event	Time	Participation Format
1	FCAJ Community Day – June 2026: Data Driven, AI Risen	June 27, 2026	Online
2	AWS Study Tour: Enterprise Cloud Architectures and Industry Applications	May 23, 2026	Online
3	FCAJ x Agentic AI Build Week 2026	July 26, 2026	Offline

Event 1 - FCAJ Community Day: Data Driven, AI Risen

Date: June 27, 2026

Participation: Online

The event introduced practical applications of data-driven systems and generative AI in enterprise environments. The main topics included AI Agents, Amazon Q, Voice AI, DevOps automation, and security considerations for Model Context Protocol integrations. I learned how AI capabilities should be connected to reliable data sources, controlled tools, and appropriate permission boundaries instead of being treated as isolated demonstrations.

Event 2 - AWS Study Tour: Enterprise Cloud Architectures and Industry Applications

Date: May 23, 2026

Participation: Online

This study tour presented common patterns used to design production workloads on AWS. The discussion covered network segmentation, managed compute and storage, high availability, monitoring, security, and cost awareness. The event helped me relate the Internship Application Platform to enterprise architecture principles, especially the separation of public and private resources, least-privilege access, operational visibility, and failure recovery.

Event 3 - FCAJ x Agentic AI Build Week 2026

Date: July 26, 2026

Participation: Offline

The build week focused on developing Agentic AI applications through rapid prototyping and teamwork. Core topics included Amazon Bedrock, tool-using agents, workflow orchestration, prompt and context design, and responsible access to external systems. The hands-on format strengthened my understanding of how an AI feature moves from an initial idea to a working application that can be tested, demonstrated, and improved from user feedback.

The three events provided me with a more comprehensive understanding of the process of developing Cloud and AI products, from problem identification and architecture design to Agentic AI development, deployment, monitoring, and system security.

Through the presented topics, I recognized that technical knowledge represents only one part of product development. An effective solution must also address actual needs, operate reliably, control costs, protect data, and deliver practical value to users.

The knowledge gained from these events will help me improve my internship project in a more structured and secure manner, particularly in areas related to AWS architecture, AI Agents, monitoring, data management, and system security.

4.1 Event 1: FCAJ Community Day – June 2026

Event Name

FCAJ Community Day – June 2026: Data Driven, AI Risen

This community event focused on AWS Cloud and AI. It included presentations related to Cloud careers, AI Agents, Voice AI, DevOps Automation, Amazon Q, and MCP connection security.

Time

9:00 AM, Saturday, June 27, 2026.

Location

Floors 26 and 36, Bitexco Financial Tower, 02 Hai Trieu Street, Ho Chi Minh City.

For this report, I attended the event online through the AWS Study Group livestream on Youtube.

Role

Online participant.

I followed the presentations, recorded the main technical points, and analyzed how the introduced solutions could be applied to Cloud and AI projects.

Main Content

The event covered several topics related to the adoption of AI and Cloud technologies in enterprise environments.

Changes in Cloud Careers in the AI Era

The first presentation discussed the impact of AI on the work of Cloud Engineers and Solution Architects.

AI can assist with coding, log analysis, and the automation of operational tasks. However, these capabilities also require engineers to develop a deeper understanding of system architecture, security, costs, and incident response.

AI can improve productivity, but engineers remain responsible for validating the results and making appropriate technical decisions.

Building Voice AI Agents for the Vietnamese Market

The speakers introduced the general Voice AI Agent workflow:

Speech-to-Text -> Large Language Model -> Text-to-Speech

Important challenges included:

- Reducing system latency.
- Accurately recognizing Vietnamese speech.
- Handling regional accents and pronunciation differences.
- Detecting when users interrupt the Agent.
- Maintaining conversational context.
- Controlling costs and maintaining system stability.

Voice AI Agents can also be connected to business tools to perform practical tasks instead of only answering questions.

Automating Incident Response with AWS DevOps Agent

AWS DevOps Agent was introduced as a solution that can assist in collecting logs, analyzing alerts, identifying root causes, and recommending remediation steps.

This solution may reduce incident-resolution time compared with manually reviewing multiple data sources. However, engineers must still verify the identified cause and evaluate the safety of the proposed actions.

Applying Amazon Q to Recruitment

One use case demonstrated how Amazon Q could read multiple CVs, compare candidate skills with job descriptions, and generate evaluation reports.

This solution could reduce the time required for initial CV screening. It also demonstrated that enterprise AI must be deployed in a controlled environment to protect personal and candidate data.

Securing MCP Connections

The final presentation discussed the risks of exposing an MCP Server directly to the Internet. Potential risks include unauthorized access, denial-of-service attacks, and data theft during transmission.

The proposed solution was to place the MCP Server inside a private network by using a VPC, Private subnet, private endpoint, and internal DNS resolution.

This approach allows data exchanged between an AI Assistant and enterprise tools to remain outside the public Internet, thereby reducing the attack surface and improving access control.

Participation Evidence

Personal participation evidence:

Lessons Learned

After the event, I recognized that AI creates meaningful value only when it is integrated with reliable data and business processes.

A conventional chatbot may answer questions, but an enterprise AI system must also be able to retrieve trusted data, use tools, enforce access control, and record its activities.

The Voice AI presentation helped me understand that combining STT, LLM, and TTS components can be relatively straightforward at the prototype level. However, production deployment requires teams to address latency, user interruption, regional accents, cost, and system stability.

The AWS DevOps Agent session demonstrated that AI can provide considerable support in log analysis and incident resolution. Nevertheless, engineers must understand the system architecture to validate the identified cause and the recommended remediation plan.

The most important lesson for me was that security must be considered from the design stage. When AI is connected to email, databases, project-management systems, or internal tools, a single configuration error may expose significant amounts of sensitive data.

Therefore, systems should apply the principle of least privilege, use private networking when appropriate, and maintain control over every action performed by an AI Agent.

Personal Contribution

My personal contribution was to record and classify the event content into three main categories:

- AI and Agent applications.
- Cloud operations.
- System and connection security.

I also related these concepts to my internship project, particularly the use of AI for CV analysis, candidate-job matching, and the protection of personal data stored in the system.

Through this review, I identified several requirements that should be considered when developing AI features, including data-access permissions, activity tracking, user-confirmation mechanisms, and the protection of sensitive information.

4.2 Event 2: Enterprise Cloud Architectures and Industry Applications

Event Name

AWS Study Tour: Enterprise Cloud Architectures and Industry Applications featuring Cloud Kinetics and Renova Cloud

This program provided a practical learning opportunity at AWS and included professional presentations from representatives of AWS, Cloud Kinetics, and Renova Cloud.

Time

May 23, 2026.

Location

Amazon Web Services Vietnam Office, Bitexco Financial Tower, Ho Chi Minh City.

The event was also livestreamed and recorded on the AWS Study Group YouTube channel, allowing participants to attend remotely or review the sessions later.

Role

Online participant.

I joined the event as a learner, followed the presentations delivered by industry professionals, and studied how Cloud architectures are designed, deployed, and operated in enterprise environments.

Main Content

The event introduced knowledge related to enterprise Cloud architectures and demonstrated how AWS can be applied to practical business problems.

One important topic was the difference between a basic Cloud system and a system that meets enterprise requirements. An enterprise architecture must not only function correctly but also satisfy criteria such as:

- Scalability as the number of users or the volume of data increases.
- High availability and reduced service interruption.
- Data security and access control.
- Backup and disaster-recovery capabilities.

- Monitoring of performance, logs, and resource costs.
- Compliance with internal requirements and industry-specific regulations.

The speakers also discussed the role of a Solution Architect in translating business requirements into technical architectures. A Solution Architect must understand customer needs, identify functional and non-functional requirements, and select appropriate AWS services.

Another notable topic was the transition from a development environment to production. A production system requires automated deployment, monitoring and alerting, change-control mechanisms, and a recovery plan in case a new release fails.

The presentations from Cloud Kinetics and Renova Cloud helped me better understand how enterprises use Cloud technologies to modernize systems, improve scalability, and support digital transformation. The event also allowed students to learn from Solution Architects currently working in the Cloud industry.

Participation Evidence

Personal participation evidence:

Lessons Learned

The first lesson I learned was that Cloud architecture design should not focus only on making an application run. Developers must also consider security, performance, scalability, monitoring, disaster recovery, and operational costs.

The second lesson was that an AWS service should only be used when it is appropriate for the system requirements. Using more services does not automatically result in a better architecture. A suitable architecture should be simple, manageable, scalable, and aligned with actual needs.

The third lesson was that production environments must be prepared and operated differently from development environments. Sensitive information should not be stored directly in source code; data should be backed up; and the system should provide logs, metrics, alarms, and incident-response procedures.

Personal Contribution

The knowledge gained from the event helped me better understand the roles of components such as API Gateway, backend servers, databases, storage systems, and monitoring services in my internship project.

I summarized the enterprise architecture principles introduced during the event and compared them with the architecture of the project currently being developed. Through this comparison, I identified areas that could be improved, including monitoring, backup, secret management, scalability, and cost control.

I also applied the principles of system layering, restricted access permissions, and resource monitoring when evaluating the project's security and operational readiness.

4.3 Event 3: FCAJ x Agentic AI Build Week 2026

Event Name

FCAJ x Agentic AI Build Week 2026: Show Up. Build. Pitch. WIN!

This program presented a recap of the notable activities, products, and project presentations from Agentic AI Build Week 2026. The main program was organized over five days, from July 8 to July 12, 2026, and included workshops, product development activities, a hackathon, and AI solution pitches before a panel of judges.

Time

- Agentic AI Build Week: July 8–12, 2026.
- Team offline participation: July 26, 2026.

Location

- Participation format: Offline.
- Location of the main program: Ho Chi Minh City, Vietnam.

Role

Offline participant.

During the event, our team focused on how the participating teams identified real-world problems, developed Agentic AI solutions, selected appropriate AWS architectures, and presented the value of their products to the judges.

Main Content

The event focused on the development of AI Agent and Agentic AI systems capable of completing multi-step tasks, using external tools, and supporting the resolution of practical business problems.

Unlike conventional chatbots that mainly respond to questions, AI Agents can analyze requests, create plans, select appropriate tools, process data, and perform actions based on predefined objectives.

Several notable topics and projects were introduced during the program:

- Developing a multi-channel food-ordering chatbot through Zalo and Messenger.
- Building a Multi-Agent system to collect and analyze signals related to changes in corporate strategies.
- Automating AWS architecture design, system diagram generation, and deployment cost estimation.
- Combining Computer Vision and Agentic AI to monitor crowd density, predict congestion, and support crowd coordination.
- Using multiple AI Sub-agent to assist in detecting and investigating suspicious financial transactions.

Through the presentations, I learned that an AI product requires more than a capable model. It must also address the correct user problem. The teams had to balance technical architecture, development time, operational cost, security, and business value.

Another important topic was the role of Human-in-the-loop mechanisms. In sensitive areas such as finance, banking, and customer service, AI can collect information, analyze data, and provide recommendations, but a human should still review the results and make the final decision.

Participation Evidence

Team offline participation evidence:

Lessons Learned

After watching the event, I identified several important lessons.

First, an AI project should begin with a real pain point rather than with the selection of a trending technology. A complex architecture provides limited value if the product does not address a specific user need.

Second, development teams should control the project scope carefully when working under time constraints. A stable MVP that clearly demonstrates its value is often more effective than a system with many incomplete features.

Third, when developing Agentic AI systems, teams must consider accuracy, security, tool permissions, model usage costs, and the ability to track the actions performed by the Agent. AI should not be allowed to execute critical actions without appropriate control and confirmation mechanisms.

Personal Contribution

My personal contribution was to actively follow the presentations, record the main points, and summarize the Agentic AI application patterns introduced during the event.

I also analyzed how these concepts could be applied to my internship project, particularly in using AI Agents to process data, support users, and automate repetitive tasks. These notes provided additional references for evaluating feasibility, project scope, and security requirements when integrating AI into a real system.

5 Workshop

Overview

This workshop presents the complete deployment process of the Internship Application Tracker on AWS.

The workshop covers source code preparation, cloud infrastructure provisioning, database deployment, application deployment, AI processing, event-driven notification, monitoring, testing, troubleshooting, and resource clean-up.

The final architecture uses:

- Amazon CloudFront and Amazon S3 for frontend delivery.
- Application Load Balancer for public API and realtime traffic.
- Amazon EKS for backend, chat, worker and AI adapter workloads.
- Amazon RDS PostgreSQL for transactional business data.
- Amazon ElastiCache Redis for realtime Socket.IO pub/sub.
- Amazon DynamoDB for chat persistence and Lambda event deduplication.
- Amazon SQS for asynchronous event delivery.
- AWS Lambda and Amazon SES for notifications.
- Amazon SageMaker for AI inference.
- GitHub Actions and AWS OIDC for continuous deployment.

Learning objectives

After completing this workshop, the reader should be able to:

1. Prepare a multi-service application for AWS deployment.
2. Deploy containerized services to Amazon EKS.
3. Deploy a static React frontend through Amazon S3 and CloudFront.
4. Connect EKS workloads to RDS, Redis, DynamoDB, SQS and SageMaker.
5. Implement asynchronous processing with workers and transactional outbox.
6. Process SQS events with AWS Lambda.
7. Implement idempotent Lambda processing using DynamoDB.
8. Configure CI/CD using GitHub Actions and AWS OIDC.
9. Validate the complete system through end-to-end testing.
10. Diagnose common networking, IAM and deployment failures.

Target audience

This workshop is intended for:

- Cloud engineering interns.
- Backend and DevOps students.
- Developers learning Amazon EKS.
- Developers implementing event-driven AWS architectures.
- Students preparing an AWS internship or graduation project report.

Architecture summary

The browser connects to Amazon CloudFront through HTTPS.

CloudFront routes:

- static frontend requests to Amazon S3
- `/api/*` requests to the backend through the Application Load Balancer
- `/chat/*` requests to the chat service
- `/socket.io/*` requests to the realtime Socket.IO service

The backend, chat service, processing worker, AI service and outbox dispatcher run inside Amazon EKS.

PostgreSQL stores transactional business data and processing jobs. DynamoDB stores chat data. Redis provides pub/sub between chat pods.

The processing worker calls the AI service, which invokes the SageMaker endpoint.

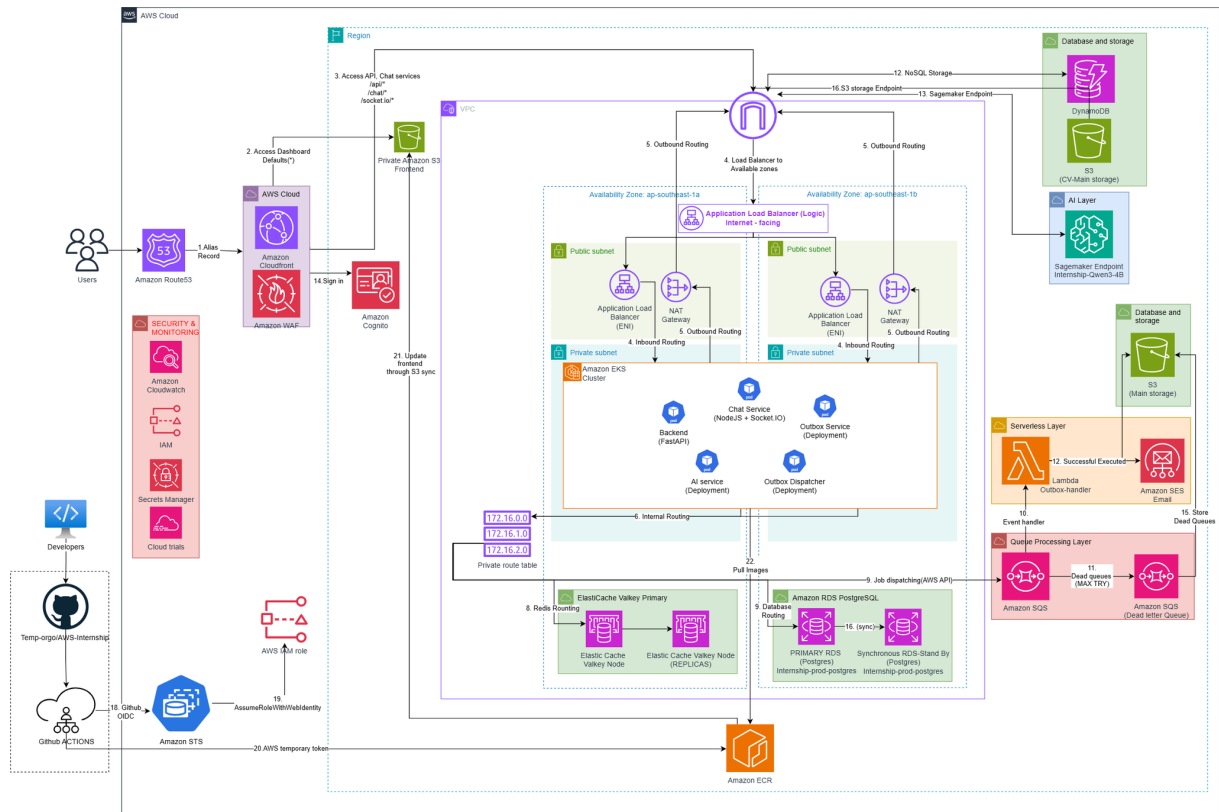


Figure 17: *Internship Application Tracker architecture on AWS*

The outbox dispatcher publishes business events to Amazon SQS. AWS Lambda consumes these events, performs deduplication in DynamoDB, archives events to Amazon S3 and sends notifications through Amazon SES.

Chapters

Chapter	Title	Description	Status
5.1	Overview	Workshop objectives, scope and final architecture	Complete
5.2	Architecture	Detailed AWS architecture and request flows	Complete
5.3	Prerequisites	Required accounts, tools and local configuration	Complete
5.4	Source Code Preparation	Repository structure, environment variables and validation	Complete
5.5	AWS Infrastructure	Networking, IAM, EKS, ECR, ALB and CloudFront	Complete
5.6	Database Deployment	PostgreSQL, Redis and DynamoDB deployment	Complete
5.7	Backend Deployment	Backend, chat, dispatcher, AI service and worker deployment	Complete
5.8	Frontend Deployment	React build, S3 upload and CloudFront distribution	Complete
5.9	Monitoring and Logging	Logs, health checks, metrics and alarms	Complete
5.10	End-to-End Testing	Application, chat, AI and Lambda verification	Complete

Chapter	Title	Description	Status
5.11	Security and Cost	IAM, encryption, OIDC and cost assumptions	Complete
5.12	Troubleshooting	Resolved deployment and runtime failures	Complete
5.13	Clean-up	Safe resource deletion procedure	Complete

Final outcome

At the end of the workshop, the Internship Application Tracker is available through CloudFront and supports:

- static frontend delivery
- backend APIs
- realtime chat
- persistent data storage
- asynchronous AI processing
- transactional outbox publishing
- Lambda-based event notifications
- automated deployment through GitHub Actions

5.1 Overview

Overview

Objective

This workshop explains how the Internship Application Tracker is prepared, deployed, verified, monitored, troubleshot, and safely cleaned up on AWS.

The target production environment is:

Item	Value
Project	Internship Application Tracker
AWS account	Project AWS account, ID redacted
Region	ap-southeast-1
Environment	production
EKS cluster	internship-prod
Kubernetes namespace	internship
Public frontend	https://dhm2rz5nmsibj.cloudfront.net
Application repository	https://github.com/Temp-orgo/AWS-Internship

Scope

The workshop covers the current production architecture only. The frontend is hosted on Amazon S3 and CloudFront. It is not documented as an EKS Deployment because the old Kubernetes frontend Deployment, Service, HPA, and PDB were removed.

The EKS cluster hosts these long-running workloads:

- backend
- chat-service
- backend-outbox-dispatcher
- backend-processing-worker
- ai-service

The managed AWS services connected to the application are RDS PostgreSQL, DynamoDB, ElastiCache Redis, S3, SQS, Lambda, SES, SageMaker, CloudFront, ALB, ECR, IAM, and CloudWatch.

Architecture context

The browser reaches CloudFront first. CloudFront routes static frontend requests to a private S3 bucket and routes dynamic paths to the Application Load Balancer:

- Default * -> S3 frontend bucket
- /api/* -> ALB -> FastAPI backend
- /chat/* -> ALB -> chat service
- /socket.io/* -> ALB -> chat service

PostgreSQL stores transactional business data and worker queues. DynamoDB stores chat records and Lambda dedupe records. Redis provides Socket.IO pub/sub between chat pods. SQS decouples committed business events from Lambda processing. Lambda archives events to S3 and sends email through SES. The processing worker calls ai-service, which invokes the SageMaker endpoint internship-qwen3-4b.

Target users

This workshop is written for:

- cloud engineering interns who need to explain a production AWS deployment
- backend and DevOps students deploying a multi-service application
- reviewers who need evidence-based architecture, operations, and cleanup steps
- future maintainers of the Internship Application Tracker

Functional scope

The application supports:

- candidate and HR authentication
- job posting and job search
- candidate application submission
- application workflow management
- CV and document upload
- asynchronous CV parsing, job parsing, and matching
- realtime chat between users
- transactional outbox event publication
- email notification through Lambda and SES

Technical scope

The technical implementation includes:

- React and Vite frontend
- FastAPI backend
- Node.js, Express, and Socket.IO chat service
- PostgreSQL with Alembic migrations
- DynamoDB chat tables
- Redis Socket.IO adapter
- EKS Deployments, Services, HPA, PDB, ServiceAccount, ConfigMap, Secret, Jobs, and Ingress
- S3 buckets for frontend assets, uploads, and event archives
- CloudFront distribution with S3 and ALB origins
- SQS Standard queue and DLQ
- Lambda event consumer with DynamoDB idempotency
- SageMaker inference endpoint behind an internal AI adapter
- GitHub Actions CI/CD using AWS OIDC

Prerequisites

The reader should have authorized access to the AWS account, GitHub repository, EKS cluster, and production deployment settings. Exact tool versions must be verified in the target environment unless documented by the repository workflow.

Implementation flow

1. Review the source repository and deployment scripts.

2. Confirm AWS account, region, IAM, and GitHub OIDC configuration.
3. Provision or verify AWS infrastructure.
4. Deploy RDS, Redis, DynamoDB, S3, SQS, Lambda, SageMaker, and CloudFront resources.
5. Build and push backend, chat, and AI images to ECR.
6. Deploy backend, chat, dispatcher, worker, and AI adapter to EKS.
7. Build the Vite frontend and publish it to S3.
8. Invalidate CloudFront.
9. Run health checks and end-to-end acceptance tests.
10. Monitor logs, metrics, and alarms.
11. Troubleshoot incidents using evidence.
12. Clean up resources in dependency order when the environment is no longer needed.

Expected result

After a successful deployment:

- CloudFront serves the frontend.
- `/api/health/ready` reaches the FastAPI backend.
- `/chat/health/ready` reaches the chat service.
- Socket.IO connects through `/socket.io`.
- PostgreSQL, DynamoDB, Redis, SQS, Lambda, SES, S3, and SageMaker are integrated.
- GitHub Actions can run validation and deployment workflows through OIDC.
- The system can process business events and archive Lambda output.

Verification

Use these high-level checks after deployment:

```
aws sts get-caller-identity
aws eks update-kubeconfig --name internship-prod --region ap-southeast-1
kubectl get deployments,pods,svc,hpa,pdb -n internship
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Common errors

Symptom	Likely cause	First diagnostic command
CloudFront returns frontend but API fails	ALB origin or behavior issue	<code>aws cloudfront get-distribution --id EQIGYNECXDL8</code>
Backend pod not ready	PostgreSQL or secret configuration failure	<code>kubectl logs deployment/backend -n internship --tail=100</code>
Chat readiness returns 503	Redis or DynamoDB unavailable	<code>kubectl logs deployment/chat-service -n internship --tail=100</code>
Worker stays disabled	AI deployment or SageMaker endpoint not ready	<code>aws sagemaker describe-endpoint --endpoint-name internship-qwen3-4b --region ap-southeast-1</code>
Lambda receives duplicates	Expected SQS at-least-once delivery	Check <code>InternshipLambdaEventDedupe</code>

Outcome

This overview establishes the boundary for the rest of the workshop: frontend on S3 and CloudFront, long-running services on EKS, transactional data in PostgreSQL, chat data in DynamoDB, Redis for realtime pub/sub, SQS and Lambda for event processing, and SageMaker for AI inference.

5.2 Architecture

Architecture

Objective

This chapter explains the final AWS architecture and the main runtime flows of the Internship Application Tracker.

Scope

This architecture is based on the final production evidence, the project documentation notes, and the current application repository. I treat runtime evidence as the source of truth when it differs from older manifests. The most important current correction is that the frontend is hosted by S3 and CloudFront, not by EKS.

Architecture context

Layer	Main components
Edge	CloudFront distribution EQIGYNECXDY8
Public routing	Application Load Balancer k8s-internship-public-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com
Kubernetes Workloads	EKS cluster internship-prod, namespace internship-backend, chat-service, backend-outbox-dispatcher, backend-processing-worker, ai-service
Data	RDS PostgreSQL, DynamoDB, ElastiCache Redis, S3
Async and event	PostgreSQL processing jobs, PostgreSQL outbox, SQS, Lambda, SES
AI	ai-service adapter and SageMaker endpoint internship-qwen3-4b
CI/CD	GitHub Actions, AWS OIDC, ECR, kubectl, AWS CLI

Diagram 1: Overall AWS architecture

Flow summary. Users enter through CloudFront, which serves the private S3 frontend or forwards dynamic requests to the ALB and EKS. EKS workloads connect to RDS, DynamoDB, Redis, S3, SageMaker, and the SQS/Lambda notification path, with a DLQ and DynamoDB deduplication table for resilient event handling.

Diagram 2: CloudFront request-routing flow

Flow summary. CloudFront evaluates each HTTPS path. The default behavior serves the S3 frontend; /api/* is forwarded to the backend service; /chat/* and /socket.io/* are forwarded to the chat service through the ALB.

The frontend bucket remains private. CloudFront handles HTTPS and SPA fallback. The ALB Ingress rewrites /api and /chat prefixes before forwarding to the backend and chat services.

Diagram 3: CV upload and AI processing sequence

Flow summary. A candidate submits a CV through the frontend, and the backend stores the document reference plus application, idempotency, outbox, and processing records. The worker claims the queued job, calls the AI adapter and SageMaker, saves the normalized result in PostgreSQL, and exposes the result for frontend polling.

Diagram 4: Realtime chat sequence

Flow summary. Socket.IO traffic passes from CloudFront and the ALB to a chat-service pod. The receiving pod persists the message in DynamoDB, publishes the room event through Redis so other pods can notify subscribed users, and acknowledges the sender only after persistence.

DynamoDB is the permanent chat store. Redis is only the cross-pod pub/sub layer and does not replace persistent storage.

Diagram 5: Transactional outbox, SQS, and Lambda sequence

Flow summary. The backend writes the business mutation and outbox event in one PostgreSQL transaction. The dispatcher publishes the event to SQS, after which Lambda uses a conditional DynamoDB write for deduplication; first-time events are archived in S3 and may trigger SES, while duplicates are treated as already processed.

SQS delivery is at least once. Duplicate delivery is expected, so Lambda idempotency is required. The successful smoke-test event was `lambda-smoke-fixed-1785220478`, archived under `outbox-archive/2026/07/28/lambda-smoke-fixed-1785220478.json`.

Diagram 6: Source-code change and CI/CD deployment flow

Flow summary. A manually triggered GitHub Actions workflow validates the repository and restores compute when required, then builds SHA-tagged ECR images. It deploys the EKS application and public ALB, ensures the private S3/CloudFront configuration, publishes the frontend, invalidates CloudFront, and produces a deployment summary.

The workflow supports `validate`, `deploy`, `rollout`, `restore-compute`, `deploy-app`, `deploy-public`, `deploy-frontend`, and `full`. GitHub Actions assumes an AWS role through OIDC; long-lived AWS access keys are not part of the deployment design.

Component descriptions

Component	Responsibility	Verification command
CloudFront	Public HTTPS entry point and routing	<code>aws cloudfront get-distribution --id EQIGYNECXDYL8</code>
S3 frontend bucket	Static frontend assets	<code>aws s3 ls s3://internship-prod-frontend-<AWS_ACCOUNT_ID>/</code>
ALB	Routes API/chat/socket traffic to EKS services	<code>kubectl get ingress internship-public -n internship</code>
Backend	FastAPI business API and health endpoints	<code>kubectl rollout status deployment/backend -n internship</code>
Chat service	Socket.IO and chat REST API	<code>kubectl rollout status deployment/chat-service -n internship</code>
Processing worker	Async AI/document jobs	<code>kubectl get deployment backend-processing-worker -n internship</code>
Outbox dispatcher	Publishes PostgreSQL events to SQS	<code>kubectl logs deployment/backend-outbox-dispatcher -n internship</code>
AI service	SageMaker adapter with stable worker routes	<code>kubectl port-forward service/ai-service 8010:8010 -n internship</code>
RDS PostgreSQL	Transactional data and queues	<code>aws rds describe-db-instances --db-instance-identifier internship-prod-postgres --region ap-southeast-1</code>
DynamoDB	Chat and Lambda dedupe tables	<code>aws dynamodb describe-table --table-name ChatMessages --region ap-southeast-1</code>
Redis	Socket.IO pub/sub	<code>aws elasticache describe-replication-groups --replication-group-id internship-prod-redis --region ap-southeast-1</code>
SQS	Outbox event transport	<code>aws sqs get-queue-attributes --queue-url <OUTBOX_QUEUE_URL> --attribute-names All</code>

Public and private resources

Public-facing	Private or internal
CloudFront domain	RDS PostgreSQL
ALB DNS origin	ElastiCache Redis

Public-facing	Private or internal
CloudFront HTTPS routes	DynamoDB tables
Browser-accessible static assets through CloudFront	EKS pod-to-service traffic
SES email delivery	SQS queue and DLQ
	S3 buckets without direct public bucket access

Security boundaries

- GitHub Actions uses OIDC to assume `internship-github-deploy`.
- EKS workloads use `internship-app` ServiceAccount and IRSA role mapping.
- Secrets are injected through GitHub environment secrets and Kubernetes `internship-secrets`.
- S3 frontend bucket should remain private and readable through CloudFront only.
- Backend and chat pods expose only service ports through the ALB path rules.
- AI traffic stays inside the cluster from worker to `ai-service`; SageMaker is invoked by the adapter.
- SQS/Lambda processing is idempotent through DynamoDB conditional writes.

Expected result

The final architecture separates static delivery, public request routing, long-running service execution, transactional storage, realtime synchronization, asynchronous event delivery, short event-driven notification, and AI inference. Each component has a clear responsibility and an independent verification path.

Common errors

Error	Cause	Resolution
Frontend shown as running in EKS	Old architecture description	Use current S3 and CloudFront architecture only
S3 described behind ALB	Incorrect CloudFront origin model	Default CloudFront behavior must route directly to S3
Redis described as chat database	Misunderstood chat design	DynamoDB stores chat records; Redis is pub/sub
Outbox described as exactly-once	SQS Standard semantics	Document at-least-once plus idempotent consumer
Worker enabled before AI readiness	SageMaker dependency not ready	Keep worker disabled until AI and endpoint are ready

Outcome

This architecture chapter provides the reference model for the remaining workshop steps. All later deployment, testing, monitoring, security, cost, troubleshooting, and cleanup sections should match these diagrams.

5.3 Prerequisites

Prerequisites

Objective

Prepare the accounts, permissions, tools, repository access, quotas, and environment variables required to deploy and verify the Internship Application Tracker.

Architecture context

This project uses GitHub Actions and AWS OIDC for production deployment. Local tools are still needed for inspection, emergency verification, and manual recovery. Do not use long-lived AWS access keys in the CI/CD pipeline.

Required access



Access area	Requirement
AWS account	Authorized access to the project AWS account; account ID redacted
AWS region	ap-southeast-1
GitHub repository	Access to https://github.com/Temp-orgo/AWS-Internship
GitHub Actions environment	Production environment secrets and variables
EKS	Permission to update kubeconfig and inspect namespace <code>internship</code>
CloudFront and S3	Permission to inspect distribution, bucket policy, objects, and invalidations
RDS, Redis, DynamoDB, SQS, Lambda, SES, SageMaker	Read access for validation; deployment role needs scoped write actions

Required tools

Tool	Required version or configuration	Evidence
Git	Version must be verified in the target environment.	Local shell
AWS CLI	Version must be verified in the target environment.	Deployment scripts call <code>aws</code>
kubect1	Version must be verified in the target environment.	GitHub Actions uses <code>azure/setup-kubect1@v5</code>
Helm	Version must be verified in the target environment.	Local Kubernetes runbook uses Helm
Docker	Version must be verified in the target environment.	CI builds Docker images
Node.js	22 in <code>.github/workflows/cicd.yml</code> ; local version must be verified.	Workflow env <code>NODE_VERSION</code>
npm	Comes with the local or CI Node.js runtime.	<code>npm ci</code> , <code>npm run build</code> , <code>npm run test</code>
Python	3.12 in <code>.github/workflows/cicd.yml</code> ; local version must be verified.	Workflow env <code>PYTHON_VERSION</code>
pip	Required for backend dependencies.	<code>backend/requirements.txt</code>
jq	Required by CloudFront helper script.	<code>scripts/aws/ensure-cloudfront.sh</code>
Hugo Extended	0.134.3 in the report GitHub Actions workflow; local version must be verified.	Report workflow
Terraform	Not found as an implementation dependency in the inspected repository.	Infrastructure uses YAML and scripts

AWS resources that must already exist or be created

Resource	Required value or status
EKS cluster	<code>internship-prod</code>
Kubernetes namespace	<code>internship</code>
Frontend bucket	<code>internship-prod-frontend-<AWS_ACCOUNT_ID></code>
Upload/archive bucket	<code>internship-prod-uploads-<AWS_ACCOUNT_ID></code>
CloudFront distribution	<code>EQIGYNECXDYL8</code>
ALB DNS	<code>k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com</code>
RDS PostgreSQL	<code>internship-prod-postgres</code>
ElastiCache Redis	<code>internship-prod-redis</code>



Resource	Required value or status
DynamoDB chat tables	ChatUsers, ChatGroups, ChatMessages
DynamoDB Lambda dedupe table	InternshipLambdaEventDedupe
SQS queue	internship-prod-outbox
SQS DLQ	internship-prod-outbox-dlq
SageMaker endpoint	internship-qwen3-4b
Lambda function	internship-outbox-handler

AWS quotas to check

Service	Quota to check	Why it matters
EKS	Cluster and managed node group quotas	Required for workload scheduling
EC2	vCPU, Elastic IP, NAT Gateway, ENI quotas	EKS nodes, ALB targets, and NAT depend on these
ALB	Load balancer and target group quotas	Ingress controller creates ALB resources
RDS	DB instances and storage	PostgreSQL deployment
ElastiCache	Node quota	Redis replication group
DynamoDB	Table and account throughput quotas	Chat and Lambda dedupe tables
SQS	Queue and message throughput	Outbox event delivery
Lambda	Concurrency	SQS event consumer
SES	Sending sandbox or production sending limit	Email notification smoke tests
SageMaker	Endpoint instance quota, especially GPU quota if used	AI endpoint uptime and capacity
CloudFront	Distribution and invalidation quotas	Frontend delivery and deployments

GitHub Actions variables and secrets

The workflow reads deployment values from GitHub environment variables and secrets. Values below are names only and must not be published with secret values.

Required or commonly used variables

Variable	Purpose
AWS_REGION	Target region, default <code>ap-southeast-1</code>
AWS_ACCOUNT_ID	Target account ID, set from the project AWS account
EKS_CLUSTER_NAME	EKS cluster, default <code>internship-prod</code>
K8S_NAMESPACE	Namespace, default <code>internship</code>
EKS_DEPLOY_ENABLED	Must be <code>true</code> for app deployment
ENABLE_ALB_INGRESS	Controls public ALB Ingress deployment
FRONTEND_DEPLOY_ENABLED	Must be <code>true</code> for frontend deployment
CREATE_CLOUDFRONT	Allows CloudFront creation when distribution ID is absent
FRONTEND_BUCKET	Frontend bucket
CLOUDFRONT_DISTRIBUTION_ID	CloudFront distribution ID
ALB_DNS	Existing ALB origin when needed
ECR_REPOSITORY_BACKEND	Backend ECR repository, default <code>internship-backend</code>
ECR_REPOSITORY_CHAT	Chat ECR repository, default <code>internship-chat</code>
ECR_REPOSITORY_AI	AI ECR repository, default <code>internship-ai</code>
AI_DEPLOY_ENABLED	Controls AI adapter deployment
PROCESSING_WORKER_ENABLED	Controls processing worker scale-up
SAGEMAKER_ENDPOINT_NAME	SageMaker endpoint, default <code>internship-qwen3-4b</code>
IRSA_ROLE_ARN	Runtime IAM role annotation

Required secrets

Secret	Purpose
AWS_ROLE_TO_ASSUME	GitHub OIDC deployment role
SECRET_KEY	Backend JWT/application secret
DATABASE_URL	PostgreSQL connection URL
REDIS_URL	Redis connection URL
OUTBOX_QUEUE_URL	SQS main queue URL
AI_SERVICE_API_KEY	Optional shared key for worker-to-AI calls

Verification commands

Run these commands only with authorized credentials:

```
aws sts get-caller-identity
aws configure get region
aws eks update-kubeconfig --name internship-prod --region ap-southeast-1
kubectl get namespace internship
kubectl get nodes -o wide
kubectl get deployments -n internship
```

Check tool versions:

```
git --version
aws --version
kubectl version --client
helm version
docker version
node --version
npm --version
python --version
jq --version
hugo version
```

Expected result

- The AWS account ID is the project account ID and should be redacted from published screenshots or logs.
- The active region is ap-southeast-1.
- kubectl can inspect the internship namespace.
- GitHub Actions can assume the deploy role through OIDC.
- Required secrets are stored in GitHub environment secrets, not in source control.

Common errors

Symptom	Cause	Resolution
AccessDenied during deployment	Missing IAM action on the OIDC role	Add least-privilege permission for the specific API call
kubectl connects to the wrong cluster	Active kube context is not production	Run <code>kubectl config current-context</code> and update kubeconfig
Frontend deployment skipped	FRONTEND_DEPLOY_ENABLED is false or only configured as an environment variable unavailable to a job-level if	Set the required flag as a GitHub repository or environment variable as used by the workflow
Worker deployment fails	PROCESSING_WORKER_ENABLED=true without AI readiness	Enable worker only after AI service image and SageMaker endpoint are ready
jq is required	jq missing from local environment	Install jq before running CloudFront helper locally

Outcome

The environment is ready for source-code preparation and deployment only after account access, tools, OIDC variables, secrets, quotas, and Kubernetes access have been verified.

5.4 Source Code Preparation

Source Code Preparation

Objective

Prepare the application repository for repeatable validation and deployment without exposing secrets or modifying production resources accidentally.

Architecture context

The repository is a multi-service application. CI/CD builds separate backend, chat, and optional AI images, deploys Kubernetes manifests, and builds the frontend separately for S3 and CloudFront.

Repository structure

Path	Purpose
backend/	FastAPI application, SQLAlchemy models, Alembic migrations, worker code, tests
frontend/	React/Vite application, API clients, route components, frontend tests
chat-service/	Node.js, Express, Socket.IO, DynamoDB repositories, Redis adapter
ai_service/	FastAPI SageMaker adapter and AI parsing/scoring code
k8s/app/	Base Kubernetes namespace, services, deployments, jobs, HPA, PDB
k8s/eks/	EKS-specific ConfigMap, ServiceAccount, Ingress, secret example
k8s/observability/	ServiceMonitor and PrometheusRule manifests
scripts/k8s/	Local and EKS deployment scripts
scripts/ci/	GitHub Actions helper scripts
scripts/aws/	AWS resource helper scripts for CloudFront, ALB, SQS, rollout, node group
.github/workflows/cicd.yml	Validation and production deployment workflow
docs/architecture/	ADRs, concurrency policy, event consumer contract
Dockerfile.ai-service	AI adapter image build file
docker-compose.yml	Local dependency stack

Obtain authorized project source

Use authorized repository access only:

```
git clone https://github.com/Temp-orgo/AWS-Internship.git
cd AWS-Internship
git status --short --branch
```

I do not publish private tokens, clone credentials, or secret-bearing remote URLs in the documentation.

Review branch and workflow state

```
git remote -v
git status --short --branch
git log --oneline -5
```

Expected result:

- The remote points to the authorized project repository.
- The working tree is understood before any deployment action.

- No unrelated local edits are overwritten.

Identify service dependencies

Backend

```
cd backend
python -m pip install -r requirements.txt
python -m ruff format --check .
python -m ruff check .
python -m mypy app tests
python -m pytest -m "not postgres"
```

The backend uses Python 3.12 in CI, FastAPI, SQLAlchemy, Alembic, Pydantic settings, boto3, Prometheus client, and OpenTelemetry libraries.

Chat service

```
cd chat-service
npm ci
npm run check
npm run lint
npm run format:check
npm run test:ci
```

The chat service uses Node.js, Express, Socket.IO, DynamoDB, Redis adapter, OpenTelemetry, and Prometheus metrics.

Frontend

```
cd frontend
npm ci
npm run test
npm run build
```

The frontend uses React 18, Vite, Tailwind CSS, Socket.IO client, Axios, and route-level API clients.

AI service

```
docker build -f Dockerfile.ai-service -t internship-ai:ci .
```

The AI adapter exposes /health/ready, /parse-job, /parse-cv, /match-applications, /rerank, /cv-job/group-score, and related scoring routes. In production, it calls SageMaker Runtime instead of loading a local model inside the worker.

Environment templates

Use example files as references only:

File	Purpose
k8s/app/secret.local.example.yaml	Local Kubernetes secret example
k8s/app/secret.local.yaml.example	Local secret template
k8s/eks/secret.example.yaml	EKS secret key shape without real values
backend/requirements.txt	Backend runtime dependencies
frontend/package.json	Frontend scripts and dependencies
chat-service/package.json	Chat scripts and dependencies

Do not commit real .env, .env.production, kubeconfig, database URL, JWT secret, AWS key, GitHub token, private key, or SES credential.

Local validation commands

The CI workflow performs these major checks:



```
bash -n $(git ls-files '*.sh')
python scripts/ci/infrastructure.py
python -m ruff format --check backend
python -m ruff check backend
python -m mypy backend/app backend/tests
python -m pytest -m "not postgres" backend
npm run test --prefix frontend
npm run build --prefix frontend
npm run test:ci --prefix chat-service
docker build -t internship-backend:ci ./backend
docker build -t internship-chat:ci ./chat-service
docker build -f Dockerfile.ai-service -t internship-ai:ci .
git diff --check
```

If Docker is unavailable locally, report Docker build validation as not run instead of claiming success.

Deployment preparation

Before running production workflows:

1. Confirm GitHub production variables and secrets are configured.
2. Confirm OIDC role `internship-github-deploy` can assume the intended AWS account.
3. Confirm ECR repositories exist for backend, chat, and AI if AI deployment is enabled.
4. Confirm RDS, Redis, DynamoDB, SQS, S3, CloudFront, ALB, Lambda, SES, and SageMaker readiness.
5. Confirm `PROCESSING_WORKER_ENABLED` is false unless AI dependencies are ready.
6. Use `deploy-app` for EKS application deployment and `deploy-frontend` for frontend deployment.

GitHub Actions workflow modes

The inspected workflow supports:

Mode	Purpose
<code>validate</code>	Run validation without production deployment
<code>deploy</code>	Legacy application deploy path still accepted by the workflow
<code>rollout</code>	Restart existing EKS workloads without building images
<code>restore-compute</code>	Ensure EKS managed node group capacity
<code>deploy-app</code>	Build/push images and deploy EKS workloads
<code>deploy-public</code>	Apply public ALB Ingress
<code>deploy-frontend</code>	Build frontend, sync to S3, invalidate CloudFront
<code>full</code>	Run the full production sequence

Expected result

The source tree is ready when:

- service dependencies install successfully
- backend, frontend, and chat tests pass in the target validation environment
- Docker images can be built
- Kubernetes manifests pass repository infrastructure checks
- required GitHub environment variables and secrets exist
- deployment is split correctly between EKS app workloads and S3/CloudFront frontend

Common errors

Symptom	Cause	Resolution
Cannot find <code>build-and-push-image.sh</code>	Helper script path not found	Use repository <code>scripts/build-and-push-image.sh</code> or set <code>BUILD_IMAGE_SCRIPT</code>

Symptom	Cause	Resolution
Node header download failure during chat CI	Native dependency build needs local headers	CI sets <code>npm_config_nodedir</code> to verified toolcache headers
Frontend build points at localhost	Missing <code>VITE_API_BASE_URL=/api</code> for production	Set Vite variables during production build
Docker build unavailable	Docker daemon not running	Start Docker or report build validation as not run
Secrets appear in diff	<code>.env</code> or generated secret file accidentally staged	Remove secret-bearing files from the commit and rotate exposed values

Outcome

The repository is prepared for validated deployment. Static frontend build and Kubernetes workloads are treated as separate delivery paths, while GitHub Actions and OIDC provide the controlled production deployment surface.

5.5 AWS Infrastructure

AWS Infrastructure

Objective

Document the AWS infrastructure required by the production deployment and provide safe verification commands for each major resource.

Scope

This chapter documents implemented resources where evidence is available, identifies fields that still require exported AWS CLI evidence, and lists optional hardening separately.

Architecture context

The infrastructure supports a hybrid edge and container architecture:

- CloudFront and S3 serve the frontend.
- The ALB routes dynamic traffic to EKS.
- EKS runs long-running application workloads.
- Managed databases, messaging, object storage, Lambda, SES, and SageMaker provide platform services.
- GitHub Actions deploys through OIDC, not through long-lived AWS access keys.

Implemented resources

Area	Resource	Status
Account and region	Project AWS account, ap-southeast-1	Account ID redacted
EKS	internship-prod	Verified from Week 8 evidence and workflow
Namespace	internship	Verified from manifests
ALB	k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com	Verified from Week 8 evidence
Frontend bucket	internship-prod-frontend-<AWS_ACCOUNT_ID>	Account suffix redacted
Upload/archive bucket	internship-prod-uploads-<AWS_ACCOUNT_ID>	Account suffix redacted
CloudFront	Distribution EQIGYNECXDYLS	Verified from Week 8 evidence
RDS	internship-prod-postgres	Verified from Week 8 evidence
Redis	internship-prod-redis, status available	Verified from Week 8 evidence
DynamoDB	ChatUsers, ChatGroups, ChatMessages, InternshipLambdaEventDedupe	Verified from Week 8 evidence

Area	Resource	Status
SQS	internship-prod-outbox, internship-prod-outbox-dlq	Verified from Week 8 evidence
SageMaker	Endpoint internship-qwen3-4b	Verified from Week 8 evidence
Lambda	internship-outbox-handler	Verified from Week 8 evidence
IAM roles	internship-github-deploy, internship-eks-runtime, internship-sagemaker-execution, internship-eks-node-role, internship-lambda-outbox-role, AmazonEKSLoadBalancerControllerRole	Verified from deployment configuration and Week 8 evidence

Network foundation

The production EKS cluster requires a VPC with public and private subnets:

- Public subnets host internet-facing load balancer resources and NAT Gateway resources.
- Private subnets host EKS nodes and internal managed service access.
- An Internet Gateway supports public ingress and NAT egress.
- A NAT Gateway gives private nodes outbound access for image pulls, AWS APIs, package downloads, and control-plane integration.

Evidence required:

- VPC ID, except the troubleshooting incident confirms vpc-0cfee519122ae18b4 was used by the AWS Load Balancer Controller.
- Public subnet IDs.
- Private subnet IDs.
- Route table IDs.
- Security group IDs and rules.
- NAT Gateway ID and Elastic IP allocation ID.

Verification:

```
aws ec2 describe-vpcs --region ap-southeast-1
aws ec2 describe-subnets --region ap-southeast-1
aws ec2 describe-route-tables --region ap-southeast-1
aws ec2 describe-nat-gateways --region ap-southeast-1
aws ec2 describe-security-groups --region ap-southeast-1
```

IAM and OIDC

GitHub Actions uses AWS OIDC. The workflow configures:

```
permissions:
  id-token: write
  contents: read
```

Deployment jobs call `aws-actions/configure-aws-credentials@v6` with `role-to-assume: ${secrets.AWS_ROLE_TO_ASSUME}}` and `allowed-account-ids: ${env.AWS_ACCOUNT_ID}`.

Runtime workloads use Kubernetes ServiceAccount `internship-app`. When `IRSA_ROLE_ARN` is provided, `scripts/k8s/deploy-eks.sh` renders `k8s/eks/serviceaccount.yaml` and applies the role annotation.

Verification:

```
aws iam get-role --role-name internship-github-deploy
aws iam get-role --role-name internship-eks-runtime
aws iam get-role --role-name internship-lambda-outbox-role
kubectl get serviceaccount internship-app -n internship -o yaml
```

ECR

The workflow builds SHA-tagged images and verifies backend and chat tags before deployment:

Image	Default ECR repository
Backend	internship-backend
Chat	internship-chat
AI adapter	internship-ai

Verification:

```
aws ecr describe-repositories --region ap-southeast-1
aws ecr describe-images --region ap-southeast-1 --repository-name internship-backend
→ --image-ids imageTag=<GITHUB_SHA>
aws ecr describe-images --region ap-southeast-1 --repository-name internship-chat --image-ids
→ imageTag=<GITHUB_SHA>
```

EKS cluster and workloads

The EKS cluster hosts:

Workload	Type	Notes
backend	Deployment and Service	FastAPI on port 8000, 2 replicas
chat-service	Deployment and Service	Node.js/Socket.IO on port 3000, 2 replicas
backend-outbox-dispatcher	Deployment and Service	Metrics port 9101, 1 replica
backend-processing-worker	Deployment and Service	Metrics port 9102, can be scaled to zero when AI is not ready
ai-service	Deployment and Service	FastAPI adapter on port 8010, 1 replica
backend-migrate	Job	Runs alembic upgrade head
chat-init	Job	Initializes DynamoDB chat tables

Verification:

```
aws eks describe-cluster --name internship-prod --region ap-southeast-1
aws eks list-nodegroups --cluster-name internship-prod --region ap-southeast-1
kubectl get deployments,pods,svc,hpa,pdb,jobs -n internship -o wide
```

AWS Load Balancer Controller and ALB

The production Ingress is internship-public and uses AWS Load Balancer Controller annotations:

- scheme: internet-facing
- target type: ip
- listener: HTTP 80
- health check path: /health/ready
- idle timeout: 3600
- target group stickiness: enabled for 86400 seconds
- paths: /api, /chat, /socket.io

Verification:

```
kubectl get ingress internship-public -n internship -o yaml
kubectl get deployment,pods -n kube-system -l
→ app.kubernetes.io/name=aws-load-balancer-controller -o wide
aws elbv2 describe-load-balancers --region ap-southeast-1
aws elbv2 describe-target-groups --region ap-southeast-1
```

S3 and CloudFront

Frontend deployment uses:

- private bucket internship-prod-frontend-<AWS_ACCOUNT_ID>
- CloudFront distribution EQIGYNECXDYL8
- CloudFront Origin Access Control
- default behavior to S3
- /api/*, /chat/*, /socket.io/* behaviors to ALB
- SPA fallback for 403 and 404 to /index.html

Verification:

```
aws s3api get-public-access-block --bucket internship-prod-frontend-<AWS_ACCOUNT_ID>
aws cloudfront get-distribution --id EQIGYNECXDYL8
aws cloudfront list-invalidations --distribution-id EQIGYNECXDYL8
```

Messaging and event-driven resources

Implemented current runtime:

Resource	Configuration
SQS main queue	internship-prod-outbox, visibility timeout 120 seconds, retention 4 days, SSE enabled
SQS DLQ	internship-prod-outbox-dlq, retention 14 days, maxReceiveCount 5
Lambda	internship-outbox-handler, SQS trigger
Dedupe table	InternshipLambdaEventDedupe
Archive location	s3://internship-prod-uploads-<AWS_ACCOUNT_ID>/outbox-archive/...
Email service	Amazon SES

Verification:

```
aws sqs get-queue-url --queue-name internship-prod-outbox --region ap-southeast-1
aws sqs get-queue-attributes --queue-url <OUTBOX_QUEUE_URL> --attribute-names All --region
→ ap-southeast-1
aws lambda get-function --function-name internship-outbox-handler --region ap-southeast-1
aws lambda list-event-source-mappings --function-name internship-outbox-handler --region
→ ap-southeast-1
aws dynamodb describe-table --table-name InternshipLambdaEventDedupe --region ap-southeast-1
```

I recorded one naming conflict: repository scripts scripts/aws/provision-outbox-sqs.sh and .ps1 default to internship-outbox-events with visibility timeout 60, while the current runtime queue is internship-prod-outbox with visibility timeout 120. I use runtime evidence for the deployed resource and would update the scripts only in the application repository as a separate implementation task.

SageMaker resources

The worker should call the internal ai-service, and ai-service calls SageMaker endpoint internship-qwen3-4b. The deployment script requires the SageMaker endpoint to be InService before enabling PROCESSING_WORKER_ENABLED=true.

Verification:

```
aws sagemaker describe-endpoint --endpoint-name internship-qwen3-4b --region ap-southeast-1
kubectl get deployment,svc ai-service -n internship
```

Proposed hardening

Area	Hardening
ALB	Restrict direct ALB access so users enter through CloudFront where practical

Area	Hardening
CloudFront	Add custom domain and ACM certificate if a production domain is required
S3	Confirm bucket policies allow only intended CloudFront and workload access
WAF	Add AWS WAF to CloudFront for common web protections
Logs	Set explicit CloudWatch retention for all Lambda and application log groups
Backups	Verify RDS backup retention and PITR requirements
Alarms	Add DLQ, Lambda Errors, RDS, ALB, and SageMaker cost/health alarms
Cost	Schedule or delete SageMaker endpoint when not in use

Expected result

The infrastructure is correctly prepared when the network is healthy, EKS nodes are Ready, ALB routes health checks, S3 and CloudFront serve the frontend, managed data services are reachable, SQS and Lambda process events, and SageMaker is ready before AI workers are enabled.

Common errors

Symptom	Root cause	Resolution
EKS nodes cannot join	Private route points to deleted NAT Gateway	Restore NAT and recreate/recover node group
ALB controller cannot infer VPC	Metadata lookup timeout	Configure <code>--aws-region=ap-southeast-1</code> and <code>--aws-vpc-id=vpc-0cfee519122ae18b4</code>
CloudFront API routes fail	ALB DNS missing or wrong behavior	Verify CloudFront origins and cache behaviors
SQS query returns queue not found	Wrong queue name	Use <code>internship-prod-outbox</code>
Worker fails at deploy	SageMaker endpoint not InService	Keep worker disabled until endpoint readiness is verified

Outcome

The infrastructure chapter gives the production resource map and verification commands needed before database, backend, frontend, monitoring, testing, and cleanup work can proceed.

5.6 Database Deployment

Objective

Deploy and verify the data layer used by the Internship Application Tracker: RDS PostgreSQL, ElastiCache Redis, DynamoDB chat tables, and DynamoDB Lambda idempotency table.

Architecture context

Different data stores are used for different access patterns:

Store	Purpose
RDS PostgreSQL	Users, companies, jobs, applications, workflow history, idempotency records, async processing jobs, transactional outbox
ElastiCache Redis	Socket.IO pub/sub between multiple chat pods
DynamoDB chat tables	Permanent chat users, groups, and messages
DynamoDB dedupe table	Lambda consumer idempotency for SQS events
S3	File storage and event archives, covered in infrastructure/frontend chapters

PostgreSQL deployment

The production RDS identifier is `internship-prod-postgres`. PostgreSQL stores transactional business data and all database-backed worker queues.

Evidence required:

- DB instance class
- allocated storage
- storage encryption status
- backup retention
- Multi-AZ setting
- subnet group name
- security group IDs
- deletion protection setting

Verification:

```
aws rds describe-db-instances \
  --db-instance-identifier internship-prod-postgres \
  --region ap-southeast-1
```

Check subnet groups:

```
aws rds describe-db-subnet-groups --region ap-southeast-1
```

Alembic migrations

The backend uses Alembic migrations in `backend/alembic/versions/`. Important migrations include:

Migration	Purpose
0004_idempotency_records.py	Idempotency records
0005_idempotency_status_check.py	Bounded idempotency statuses
0006_job_application_versions.py	Optimistic version fields
0007_outbox_events.py	Transactional outbox
0008_async_processing_jobs.py	Async processing queue
1065bcf66cb9_0007_add_rerank_run_tables.py	Rerank run tables

The EKS deployment creates a Kubernetes Job named `backend-migrate` that runs:

```
alembic upgrade head
```

Manual verification from the repository root:

```
cd backend
alembic current
alembic heads
alembic history --verbose
```

Do not run migrations against production unless the deployment window and rollback plan are approved.

PostgreSQL connection configuration

The production deployment provides `DATABASE_URL` through the Kubernetes Secret `internship-secrets`. The actual connection string is secret, so I do not print it in the documentation.

Kubernetes verification without exposing the value:

```
kubectl get secret internship-secrets -n internship
kubectl describe secret internship-secrets -n internship
```

Backend readiness verifies PostgreSQL by executing `SELECT 1`:

```
kubectl port-forward service/backend 18080:8000 -n internship
curl -fsS http://127.0.0.1:18080/health/ready
```

Expected response:

```
{"status":"ready","service":"internship-api","dependencies":{"postgres":true}}
```

ElastiCache Redis

The production Redis replication group is `internship-prod-redis`, and I verified it as available in the Week 8 evidence. Redis is used by the Socket.IO adapter so chat events can be broadcast between multiple chat pods.

Verification:

```
aws elasticache describe-replication-groups \
  --replication-group-id internship-prod-redis \
  --region ap-southeast-1
```

The chat service reads `REDIS_URL` from `internship-secrets`. In production, Redis must be reachable from chat pods and should not be exposed publicly.

DynamoDB chat tables

The chat service uses three DynamoDB tables:

- ChatUsers
- ChatGroups
- ChatMessages

I verified all three chat tables as `ACTIVE` in the Week 8 evidence. The chat service readiness endpoint depends on both DynamoDB and Redis:

```
kubect1 port-forward service/chat-service 18081:3000 -n internship
curl -fsS http://127.0.0.1:18081/health/ready
```

Expected response when dependencies are ready:

```
{"status":"ready","dependencies":{"redis":true,"dynamodb":true}}
```

DynamoDB verification:

```
aws dynamodb describe-table --table-name ChatUsers --region ap-southeast-1
aws dynamodb describe-table --table-name ChatGroups --region ap-southeast-1
aws dynamodb describe-table --table-name ChatMessages --region ap-southeast-1
```

DynamoDB Lambda dedupe table

The Lambda SQS consumer uses `InternshipLambdaEventDedupe` to store processed `eventId` values. This protects the Lambda side from duplicate SQS delivery.

Verification:

```
aws dynamodb describe-table \
  --table-name InternshipLambdaEventDedupe \
  --region ap-southeast-1
```

For the successful smoke-test event:

```
aws dynamodb get-item \
  --table-name InternshipLambdaEventDedupe \
  --key '{"eventId":{"S":"lambda-smoke-fixed-1785220478"}}' \
  --region ap-southeast-1
```

The report should not claim this command was run unless the output is exported as evidence.

Data consistency design

Concern	Mechanism
Duplicate registration	PostgreSQL unique constraints and 409 handling
Duplicate candidate apply	Unique (job_posting_id, candidate_user_id) and Idempotency-Key
Stale HR updates	expectedVersion and conditional updates
Long AI jobs	async_processing_jobs table with leases and retries
Lost event publication	outbox_events committed with business mutation
Duplicate SQS delivery	Lambda DynamoDB conditional write by eventId
Duplicate chat message retry	DynamoDB conditional create with clientMessageId

Commands

Check Kubernetes data-related configuration:

```
kubectl get configmap internship-config -n internship -o yaml
kubectl get secret internship-secrets -n internship
kubectl get job backend-migrate chat-init -n internship
kubectl logs job/backend-migrate -n internship
kubectl logs job/chat-init -n internship
```

Check application tables through application health:

```
kubectl rollout status deployment/backend -n internship --timeout=300s
kubectl rollout status deployment/chat-service -n internship --timeout=300s
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Expected result

- backend-migrate completes successfully.
- Backend readiness returns PostgreSQL dependency true.
- Chat readiness returns Redis and DynamoDB dependencies true.
- DynamoDB chat tables are ACTIVE.
- Redis replication group is available.
- Lambda dedupe table exists and supports conditional writes.

Common errors

Symptom	Root cause	Resolution
Backend readiness fails	Wrong DATABASE_URL, security group issue, migration failure	Inspect backend logs and RDS security groups
Alembic fails to parse URL	Shell interpolation or secret formatting problem	Store URL as a secret and avoid printing it
Chat readiness returns not_ready	Redis adapter or DynamoDB initialization failure	Inspect chat-service logs and chat-init job
Duplicate Lambda event sends email twice	Dedupe table write is missing or not conditional	Use DynamoDB conditional write keyed by eventId
Outbox rows stay pending	Dispatcher cannot publish to SQS	Check OUTBOX_QUEUE_URL, IRSA SQS policy, dispatcher logs

Troubleshooting

Use these commands during incidents:

```
kubectl logs deployment/backend -n internship --tail=200
kubectl logs deployment/chat-service -n internship --tail=200
kubectl logs deployment/backend-outbox-dispatcher -n internship --tail=200
kubectl logs deployment/backend-processing-worker -n internship --tail=200
```

```
aws rds describe-db-instances --db-instance-identifier internship-prod-postgres --region  
→ ap-southeast-1  
aws elasticache describe-replication-groups --replication-group-id internship-prod-redis  
→ --region ap-southeast-1  
aws dynamodb describe-table --table-name ChatMessages --region ap-southeast-1
```

Outcome

The database layer is ready when PostgreSQL migrations have completed, Redis is available to chat pods, DynamoDB chat tables are active, Lambda dedupe is configured, and application readiness checks pass through the deployed routing path.

5.7 Backend Deployment

Objective

Deploy and verify the EKS application workloads: FastAPI backend, chat service, outbox dispatcher, processing worker, and AI service adapter.

Architecture context

The frontend is not deployed to EKS. EKS hosts the long-running services and workers behind an ALB origin used by CloudFront.

Deployment inputs

The production workflow computes image URIs from the AWS account, region, ECR repository, and `github.sha`:

Workload	Image variable	Default ECR repository
Backend and workers	BACKEND_IMAGE	internship-backend
Chat service	CHAT_IMAGE	internship-chat
AI adapter	AI_IMAGE	internship-ai

The deployment script requires:

- BACKEND_IMAGE
- CHAT_IMAGE
- SECRET_KEY
- DATABASE_URL
- REDIS_URL
- AWS_REGION
- OUTBOX_QUEUE_URL

Optional but important:

- AI_DEPLOY_ENABLED
- AI_IMAGE
- PROCESSING_WORKER_ENABLED
- SAGEMAKER_ENDPOINT_NAME
- AI_SERVICE_BASE_URL
- IRSA_ROLE_ARN
- S3_BUCKET
- AI_SERVICE_API_KEY

Image build and ECR push

The CI helper scripts/`ci/deploy-eks-pipeline.sh` builds and pushes images:

```
bash scripts/ci/deploy-eks-pipeline.sh
```

The workflow verifies pushed backend and chat tags:

```
aws ecr describe-images \
  --region ap-southeast-1 \
  --repository-name internship-backend \
  --image-ids imageTag=<GITHUB_SHA>
```

```
aws ecr describe-images \
  --region ap-southeast-1 \
  --repository-name internship-chat \
  --image-ids imageTag=<GITHUB_SHA>
```

If `AI_DEPLOY_ENABLED=true`, the AI adapter image is built with:

```
docker build --file Dockerfile.ai-service --tag <AI_IMAGE> .
docker push <AI_IMAGE>
```

Kubernetes namespace and ServiceAccount

The deployment applies:

```
kubectl apply -f k8s/app/namespace.yaml
kubectl apply -f k8s/app/serviceaccount.yaml
```

When `IRSA_ROLE_ARN` is set, the EKS-specific ServiceAccount manifest is rendered and applied from `k8s/eks/serviceaccount.yaml`.

Verification:

```
kubectl get namespace internship
kubectl get serviceaccount internship-app -n internship -o yaml
```

ConfigMap and Secret usage

The production ConfigMap comes from `k8s/eks/configmap.yaml`. Important values include:

Key	Value or purpose
APP_ENV	eks
OUTBOX_PUBLISHER	sqs
AI_SERVICE_BASE_URL	http://ai-service:8010
AI_SERVICE_PARSE_TIMEOUT_SECONDS	120
AI_SERVICE_MATCH_TIMEOUT_SECONDS	600
STORAGE_BACKEND	s3
AWS_REGION	ap-southeast-1
DYNAMODB_USERS_TABLE	ChatUsers
DYNAMODB_GROUPS_TABLE	ChatGroups
DYNAMODB_MESSAGES_TABLE	ChatMessages

The deployment creates or updates Kubernetes Secret `internship-secrets` from GitHub secrets. Do not print secret values.

Verification:

```
kubectl get configmap internship-config -n internship -o yaml
kubectl describe secret internship-secrets -n internship
```

Migration and chat initialization jobs

The deployment deletes old jobs, applies fresh jobs, and waits for completion:

```
kubectl delete job backend-migrate chat-init -n internship --ignore-not-found
kubectl apply -f <rendered-backend-migration-job.yaml>
kubectl apply -f <rendered-chat-init-job.yaml>
kubectl wait --for=condition=complete job/backend-migrate -n internship --timeout=300s
kubectl wait --for=condition=complete job/chat-init -n internship --timeout=300s
```

backend-migrate runs `alembic upgrade head`. chat-init initializes the DynamoDB chat tables.

Workload deployments

Deployment	Container command	Port	Replicas	Purpose
backend	<code>uvicorn app.main:app --host 0.0.0.0 --port 8000 --proxy-headers</code>	8000	2	FastAPI REST API
chat-service	<code>node server.js</code> through image start command	3000	2	Chat REST and Socket.IO
backend-outbox-dispatcher	<code>python -m app.workers.outbox_dispatcher</code>	9101 metrics	1	Publishes outbox events to SQS
backend-processing-worker	<code>python -m app.workers.processing_worker</code>	9102 metrics	2 in manifest, can be rendered to 0	Async AI/document jobs
ai-service	AI image command	8010	1	SageMaker adapter

Apply and verify:

```
bash scripts/k8s/deploy-eks.sh
kubectl rollout status deployment/backend -n internship --timeout=300s
kubectl rollout status deployment/chat-service -n internship --timeout=300s
kubectl rollout status deployment/backend-outbox-dispatcher -n internship --timeout=300s
kubectl rollout status deployment/backend-processing-worker -n internship --timeout=300s
kubectl rollout status deployment/ai-service -n internship --timeout=900s
```

Only run the ai-service rollout command when AI deployment is enabled.

AI and processing worker gate

The deployment script blocks unsafe worker enablement:

- `PROCESSING_WORKER_ENABLED=true` requires `AI_DEPLOY_ENABLED=true`.
- `PROCESSING_WORKER_ENABLED=true` requires `AI_IMAGE`.
- SageMaker endpoint `internship-qwen3-4b` must be `InService`.

Check the endpoint before enabling the worker:

```
aws sagemaker describe-endpoint \
  --endpoint-name internship-qwen3-4b \
  --region ap-southeast-1 \
  --query EndpointStatus \
  --output text
```

Expected result:

InService

Health probes

Workload	Startup probe	Readiness probe	Liveness probe
backend	<code>/health/live</code>	<code>/health/ready</code>	<code>/health/live</code>
chat-service	<code>/health/live</code>	<code>/health/ready</code>	<code>/health/live</code>
ai-service	Not configured separately	<code>/health/ready</code>	<code>/health/ready</code>



Workload	Startup probe	Readiness probe	Liveness probe
backend-outbox-dispatcher	TCP 9101	TCP 9101	TCP 9101
backend-processing-worker	TCP 9102	TCP 9102	TCP 9102

HPA and PDB

The manifest `k8s/app/autoscaling.yaml` defines:

Resource	HPA	PDB
backend	min 2, max 5, CPU target 70%	min available 1
chat-service	min 2, max 5, CPU target 70%	min available 1
backend-processing-worker	min 2, max 5, CPU target 70%	min available 1 when worker enabled

When the processing worker is disabled, deployment scripts delete its HPA and PDB and scale it to zero.
Verification:

```
kubectl get hpa,pdb -n internship
kubectl describe hpa backend -n internship
kubectl describe hpa chat-service -n internship
```

Ingress and ALB routing

The public ALB Ingress is `k8s/eks/ingress-alb-no-domain.yaml`. It routes:

- `/api` to Service backend
- `/chat` to Service chat-service
- `/socket.io` to Service chat-service

Deploy public ALB routing with the workflow mode `deploy-public` or the helper script:

```
ENABLE_ALB_INGRESS=true bash scripts/aws/deploy-public-ingress.sh
```

Verification:

```
kubectl get ingress internship-public -n internship
curl -fsS http://k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com/api/health/ready
curl -fsS http://k8s-internshippublic-48101b50ad-85486086.ap-southeast-1.elb.amazonaws.com/chat/health/ready
```

Port-forward smoke checks

The deployment script uses local port-forwards for health smoke tests:

```
kubectl port-forward service/backend 18080:8000 -n internship
curl -fsS http://127.0.0.1:18080/health/ready

kubectl port-forward service/chat-service 18081:3000 -n internship
curl -fsS http://127.0.0.1:18081/health/ready

kubectl port-forward service/ai-service 18082:8010 -n internship
curl -fsS http://127.0.0.1:18082/health/ready
```

Expected result

- Migration and chat init jobs complete.
- Backend, chat, and dispatcher deployments roll out successfully.
- Processing worker is either Ready or intentionally scaled to zero.

- AI service is Ready when AI deployment is enabled.
- Services have endpoints.
- ALB health checks pass for backend and chat.
- CloudFront can reach ALB origin for /api, /chat, and /socket.io.

Common errors

Symptom	Cause	Resolution
Missing required environment variable	Required deployment secret or variable absent	Add the missing GitHub secret or variable
Unable to compute backend/chat image URIs	Workflow variable or SHA-based image construction issue	Use account, region, repository, and <code>github.sha</code>
ECR image not found	Build/push job did not publish the SHA tag	Inspect <code>build-images</code> job and ECR repository
PROCESSING_WORKER_ENABLED=true fails	AI dependencies not ready	Enable AI image and verify SageMaker InService first
Chat readiness fails	Redis or DynamoDB dependency unavailable	Inspect chat logs and DynamoDB/Redis status
Backend readiness fails	PostgreSQL unavailable or migration failed	Inspect backend logs and RDS connectivity

Outcome

The EKS backend deployment is complete when backend, chat, dispatcher, optional worker, and optional AI adapter are running with correct probes, services, endpoints, scaling controls, and ALB routing.

5.8 Frontend Deployment

Objective

Build the React/Vite frontend, publish it to a private S3 bucket, and serve it through CloudFront.

Architecture context

The frontend is currently deployed as static files, not as an EKS workload. The old frontend Kubernetes Deployment, Service, HPA, PDB, and frontend ALB Ingress were removed so future EKS app deployments cannot recreate them.

Current public entry point:

`https://dhm2rz5nmsibj.cloudfront.net`

Current CloudFront distribution:

`EQIGYNECXDYL8`

Vite environment configuration

The frontend production build uses:

Variable	Production value	Purpose
<code>VITE_API_BASE_URL</code>	<code>/api</code>	Makes backend API calls go through CloudFront and ALB
<code>VITE_CHAT_API_BASE_URL</code>	empty string	Allows browser runtime to use /chat outside localhost

The frontend API client defaults to local development when not configured:

- API default: `http://localhost:8001`
- Chat default in local browser: `http://localhost:3000`

- Chat default outside localhost: /chat
- Socket.IO path: /socket.io

Build commands

From the application repository:

```
cd frontend
npm ci
VITE_API_BASE_URL=/api VITE_CHAT_API_BASE_URL= npm run build
```

Expected output directory:

frontend/dist

The deployment script detects frontend/dist or frontend/build, but the current Vite output is frontend/dist.

S3 publishing

The production helper is:

```
bash scripts/ci/deploy-frontend.sh
```

Required variables:

Variable	Purpose
FRONTEND_DEPLOY_ENABLED	Must be true
FRONTEND_BUCKET	Target bucket, for example internship-prod-frontend-<AWS_ACCOUNT_ID>
CLOUDFRONT_DISTRIBUTION_ID	Distribution to invalidate
VITE_API_BASE_URL	/api
VITE_CHAT_API_BASE_URL	empty string

The script keeps the bucket private:

```
aws s3api put-public-access-block \
  --bucket internship-prod-frontend-<AWS_ACCOUNT_ID> \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Static assets are uploaded with long cache headers:

```
aws s3 sync frontend/dist s3://internship-prod-frontend-<AWS_ACCOUNT_ID> \
  --delete \
  --exclude index.html \
  --cache-control public,max-age=31536000,immutable
```

index.html is uploaded separately with no-cache semantics:

```
aws s3 cp frontend/dist/index.html s3://internship-prod-frontend-<AWS_ACCOUNT_ID>/index.html \
  --cache-control no-cache \
  --content-type text/html
```

CloudFront configuration

The helper scripts/aws/ensure-cloudfront.sh validates or creates the distribution. The current design uses:

Behavior	Origin	Notes
Default *	S3 frontend bucket	Static frontend and SPA assets

Behavior	Origin	Notes
/api/*	ALB	Backend API
/chat/*	ALB	Chat REST API
/socket.io/*	ALB	Socket.IO traffic

CloudFront also configures SPA fallback:

Error	Response path	Response code
403	/index.html	200
404	/index.html	200

CloudFront invalidation:

```
aws cloudfront create-invalidation \  
  --distribution-id EQIGYNECXDYL8 \  
  --paths "/*"
```

GitHub Actions frontend deployment

Use workflow dispatch mode:

GitHub -> Actions -> CI/CD -> Run workflow -> mode: deploy-frontend

The deploy-frontend job:

1. Configures AWS credentials through OIDC.
2. Sets Node.js from workflow NODE_VERSION.
3. Verifies ALB health at /api/health/ready and /chat/health/ready.
4. Runs scripts/ci/deploy-frontend.sh.
5. Uploads frontend files to S3.
6. Creates a CloudFront invalidation.

Verification

Check CloudFront:

```
curl -I https://dhm2rz5nmsibj.cloudfront.net/  
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready  
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Check S3 bucket privacy:

```
aws s3api get-public-access-block \  
  --bucket internship-prod-frontend-<AWS_ACCOUNT_ID>
```

Check distribution:

```
aws cloudfront get-distribution --id EQIGYNECXDYL8  
aws cloudfront list-invalidations --distribution-id EQIGYNECXDYL8
```

Expected result

- The frontend loads through CloudFront.
- Direct frontend S3 bucket public access remains blocked.
- /api/*, /chat/*, and /socket.io/* routes use the ALB origin.
- SPA deep links return index.html.
- A new CloudFront invalidation is created after deployment.

Common errors

Symptom	Cause	Resolution
Frontend job skipped	FRONTEND_DEPLOY_ENABLED not true in the context used by the job	Configure the flag in GitHub variables as required by the workflow
API calls go to localhost in production	Missing VITE_API_BASE_URL=/api	Rebuild with production Vite variables
CloudFront returns old UI	Cached assets or missing invalidation	Check invalidation status and index.html cache control
S3 access denied through CloudFront	Missing OAC bucket policy	Run <code>ensure-cloudfront.sh</code> with correct account and distribution
Direct S3 website endpoint fails	Expected current behavior	Frontend bucket is private and should be accessed through CloudFront
Socket.IO fails	Missing /socket.io/* CloudFront behavior or ALB route	Verify CloudFront behavior and Ingress path

Removal of old Kubernetes frontend resources

The current architecture intentionally removes frontend Kubernetes resources. Do not recreate:

- Deployment/frontend
- Service/frontend
- frontend HPA
- frontend PDB
- frontend-specific ALB Ingress
- FRONTEND_IMAGE deployment path for EKS app deploy

The frontend is built once and served as static files from S3 and CloudFront.

Outcome

Frontend deployment is complete when the Vite build is in S3, CloudFront has invalidated cached objects, public users can load the application through `https://dhm2rz5nmsibj.cloudfront.net`, and dynamic routes reach backend/chat through CloudFront and the ALB.

5.9 Monitoring and Logging

Objective

Monitor application health, deployment state, logs, metrics, queues, Lambda processing, and AWS managed services.

Architecture context

The repository contains Kubernetes ServiceMonitor and PrometheusRule manifests for application metrics. AWS-side CloudWatch metrics and alarms must be verified from the target account before being marked as deployed.

Kubernetes resource inspection

Use `kubectl get` for fast status checks:

```
kubectl get deployments,pods,svc,endpoints,hpa,pdb,jobs -n internship -o wide
kubectl get ingress internship-public -n internship
kubectl get events -n internship --sort-by=.lastTimestamp
```

Expected result:

- backend and chat-service have Ready replicas.
- backend-outbox-dispatcher is Ready.
- backend-processing-worker is either Ready or intentionally scaled to zero.
- ai-service is Ready when AI deployment is enabled.
- backend-migrate and chat-init have completed.

Rollout status

```
kubectl rollout status deployment/backend -n internship --timeout=300s
kubectl rollout status deployment/chat-service -n internship --timeout=300s
kubectl rollout status deployment/backend-outbox-dispatcher -n internship --timeout=300s
kubectl rollout status deployment/backend-processing-worker -n internship --timeout=300s
kubectl rollout status deployment/ai-service -n internship --timeout=900s
```

Only check ai-service when AI deployment is enabled.

Application logs

```
kubectl logs deployment/backend -n internship --tail=200
kubectl logs deployment/chat-service -n internship --tail=200
kubectl logs deployment/backend-outbox-dispatcher -n internship --tail=200
kubectl logs deployment/backend-processing-worker -n internship --tail=200
kubectl logs deployment/ai-service -n internship --tail=200
```

Use `--previous` when a container has restarted:

```
kubectl logs deployment/backend -n internship --previous --tail=200
```

Describe commands

Use `kubectl describe` when readiness, scheduling, probes, or Ingress reconciliation fail:

```
kubectl describe pod -n internship -l app=backend
kubectl describe pod -n internship -l app=chat-service
kubectl describe hpa backend -n internship
kubectl describe ingress internship-public -n internship
```

Health endpoints

Backend readiness checks PostgreSQL:

```
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
```

Expected response:

```
{"status": "ready", "service": "internship-api", "dependencies": {"postgres": true}}
```

Chat readiness checks Redis and DynamoDB:

```
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

Expected response:

```
{"status": "ready", "dependencies": {"redis": true, "dynamodb": true}}
```

AI readiness checks that SAGEMAKER_ENDPOINT_NAME is configured:

```
kubectl port-forward service/ai-service 18082:8010 -n internship
curl -fsS http://127.0.0.1:18082/health/ready
```

Prometheus and Grafana

The repository defines ServiceMonitor resources for:

- backend
- chat-service
- backend-outbox-dispatcher
- backend-processing-worker

Apply when the Prometheus Operator CRDs are installed:

```
kubectl apply -f k8s/observability/service-monitors.yaml
kubectl apply -f k8s/observability/prometheus-rules.yaml
```



Verify:

```
kubectl get servicemonitor,prometheusrule -n monitoring
kubectl port-forward service/kube-prometheus-stack-prometheus 9092:9090 -n monitoring
curl -fsS "http://127.0.0.1:9092/api/v1/targets?state=active"
```

Expected application targets:

- backend
- chat-service
- backend-outbox-dispatcher
- backend-processing-worker

Repository-defined alerts

The repository includes Prometheus alerts for:

Alert	Purpose
InternshipBackendDown	Backend metrics target unavailable
InternshipChatDown	Chat metrics target unavailable
InternshipChatDependencyDown	Chat Redis or DynamoDB dependency unavailable
InternshipOutboxDispatcherDown	Dispatcher metrics target unavailable
InternshipProcessingWorkerDown	Processing worker metrics target unavailable
InternshipBackend5xxHigh	Elevated backend 5xx rate
InternshipBackendP95LatencyHigh	High backend p95 latency
InternshipChatMessageFailureRatioHigh	High chat message failure rate
InternshipPodCrashLooping	Pod CrashLoopBackOff
InternshipOutboxDeadEventsPresent	Dead outbox events need operator action
InternshipOutboxPendingTooOld	Pending outbox event age too high
InternshipOutboxTransportErrors	SQS transport errors
InternshipProcessingQueueOld	Processing queue age too high
InternshipProcessingJobsFailing	Processing jobs failing
InternshipAIProviderFailures	AI provider failures

Status: implemented in repository manifests. Evidence required to prove these were applied in production.

CloudWatch Logs

Check Lambda logs:

```
aws logs describe-log-groups \
  --log-group-name-prefix /aws/lambda/internship-outbox-handler \
  --region ap-southeast-1

aws logs tail /aws/lambda/internship-outbox-handler \
  --since 1h \
  --region ap-southeast-1
```

Check EKS-related logs only if container log shipping is configured. Evidence required: CloudWatch log group names for backend, chat, worker, dispatcher, and AI service.

AWS managed-service metrics

Service	Useful metrics
ALB	HTTPCode_Target_5XX_Count, TargetResponseTime, healthy hosts, LCUs
CloudFront	requests, 4xx/5xx rate, origin latency, bytes downloaded
RDS	CPU, free storage, connections, read/write IOPS, replica lag if any

Service	Useful metrics
ElastiCache Redis	CPU, memory, evictions, current connections
DynamoDB	throttled requests, consumed read/write capacity, system errors
SQS	approximate visible messages, age of oldest message, DLQ visible messages
Lambda	errors, duration, throttles, iterator age, concurrent executions
SageMaker	endpoint invocation errors, model latency, overhead latency, instance utilization

Proposed CloudWatch alarms

These alarms are recommended. Evidence required before marking them deployed:

Alarm	Recommended condition
DLQ messages visible	<code>ApproximateNumberOfMessagesVisible > 0</code> on <code>internship-prod-outbox-dlq</code>
Main queue age high	Oldest SQS message age exceeds operational threshold
Lambda errors	<code>Errors > 0</code> for <code>internship-outbox-handler</code>
Lambda throttles	<code>Throttles > 0</code>
ALB target 5xx high	Target 5xx count/rate above threshold
RDS CPU high	CPU above sustained threshold
RDS free storage low	Free storage below threshold
Redis memory high	Memory or evictions above threshold
SageMaker errors	Invocation/model errors above threshold
CloudFront 5xx high	5xx error rate above threshold

Expected result

- Kubernetes deployments are Ready.
- Health endpoints pass through CloudFront.
- Logs are available for each EKS workload and Lambda.
- Prometheus can scrape application metrics when observability is installed.
- SQS, Lambda, RDS, Redis, DynamoDB, ALB, CloudFront, and SageMaker metrics are visible in CloudWatch.
- Proposed alarms are either deployed and evidenced or clearly listed as proposed.

Common errors

Symptom	Cause	Resolution
kubect1 top has no data	Metrics server unavailable or not scraped yet	Check metrics-server rollout
Prometheus target down	ServiceMonitor selector or namespace mismatch	Inspect Service labels and ServiceMonitor selectors
Chat health not ready	Redis or DynamoDB dependency down	Check chat logs and AWS service status
Lambda logs absent	Function not invoked or wrong region	Confirm event-source mapping and region
DLQ growing	Lambda or downstream processing failure	Inspect DLQ messages and Lambda logs

Outcome

Monitoring is complete when operational staff can inspect Kubernetes state, logs, health endpoints, Prometheus metrics, CloudWatch metrics, Lambda logs, SQS/DLQ state, and alarms without exposing secrets.

5.10 End-to-End Testing

Objective

Define an acceptance test set for the deployed Internship Application Tracker and record which results are verified, implemented but still requiring runtime evidence, proposed, or not completed.

Architecture context

End-to-end tests must cover the full public path:

Browser -> CloudFront -> S3 or ALB -> EKS workloads -> managed AWS services

They must also cover asynchronous paths:

Backend -> PostgreSQL outbox -> dispatcher -> SQS -> Lambda -> DynamoDB dedupe -> S3 archive -> SES

Test status definitions

Status	Meaning
Verified	Runtime evidence from Week 8 or exported production evidence confirms the result
Implemented, verification required	Code or manifests implement the capability, but production runtime evidence is not included
Proposed	Recommended test or control, not proven implemented
Not completed	Not implemented or not completed in the evidence I have

Acceptance test table

ID	Test case	Test input or command	Expected result	Evidence status
TC01	CloudFront frontend	<code>curl -I https://dhm2rz5nmsibj.cloudfront.net/</code>	CloudFront returns frontend response	Implemented, verification required
TC02	Backend health through CloudFront	<code>curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready</code>	Backend readiness JSON with PostgreSQL dependency true	Implemented, verification required
TC03	Chat health through CloudFront	<code>curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready</code>	Chat readiness JSON with Redis and DynamoDB true	Implemented, verification required
TC04	Socket.IO handshake	Browser or Socket.IO client connects with path /socket.io	Socket connection established through CloudFront and ALB	Implemented, verification required
TC05	Registration	Create candidate or HR account	User created or duplicate email returns 409	Implemented, verification required
TC06	Login	Valid credentials	JWT session created and role-based route opens	Implemented, verification required
TC07	Job creation	HR creates job posting	Job stored in PostgreSQL with version	Implemented, verification required
TC08	Application submission	Candidate applies to published job	Application, documents, status history, outbox event and processing job are committed	Implemented, verification required

ID	Test case	Test input or command	Expected result	Evidence status
TC09	Duplicate application handling	Repeat apply for same job/candidate	409 duplicate application or idempotent replay	Implemented, verification required
TC10	CV upload	Apply with CV/document	File stored in S3 or configured storage backend and metadata stored in PostgreSQL	Implemented, verification required
TC11	Processing job creation	Submit CV/job parse request	<code>async_processing_jobs</code> row created and status visible	Implemented, verification required
TC12	AI completion	Worker calls <code>ai-service</code> and SageMaker endpoint	Parsed CV/job or matching result persisted	Implemented, verification required
TC13	Chat between two clients	Send direct or group message	Sender receives ACK; recipient receives realtime message	Implemented, verification required
TC14	Chat persistence	Reload chat history	DynamoDB returns persisted message	Implemented, verification required
TC15	Chat pod restart and reconnect	Restart one chat pod during active session	Socket reconnects and no duplicate persisted message for retry	Implemented, verification required
TC16	PostgreSQL connectivity	Backend <code>/health/ready</code>	<code>postgres: true</code>	Implemented, verification required
TC17	Redis status	Chat <code>/health/ready</code>	<code>redis: true</code>	Redis runtime status verified from Week 8 evidence; health endpoint output still useful
TC18	DynamoDB table status	<code>describe-table</code> for chat tables	ACTIVE for ChatUsers, ChatGroups, ChatMessages	Verified from Week 8 DynamoDB evidence
TC19	Outbox dispatcher	Inspect dispatcher deployment and logs	Dispatcher reads PostgreSQL outbox and publishes to SQS	Deployment verified from Week 8 EKS evidence; event publish verification required
TC20	SQS event	Inspect <code>internship-prod-outbox</code>	Message accepted with SSE, visibility timeout, retention	Queue and DLQ verified from Week 8 SQS evidence
TC21	Lambda processing	Smoke-test event <code>lambda-smoke-fixed-1785220478</code>	Lambda processed event and returned EMAIL_SENT	Function and log group partially verified; successful invocation evidence still required
TC22	SES notification	Same smoke-test event	Email send result recorded as EMAIL_SENT	Verification required; Week 8 evidence does not prove SES delivery
TC23	S3 event archive	Inspect archive key	<code>outbox-archive/2026/07/28/lambda-smoke-fixed-1785220478.json</code> exists	Verification required; Week 8 evidence does not prove archive object creation
TC24	DynamoDB idempotency	Process duplicate event ID	Duplicate is skipped by dedupe table	Implemented, verification required
TC25	Duplicate Lambda event	Re-send same event ID	No second side effect	Implemented, verification required

ID	Test case	Test input or command	Expected result	Evidence status
TC26	DLQ handling	Force consumer failure in controlled test	Message redriven to internship-prod-outbox-dlq after max receive count	Proposed
TC27	CloudFront routing correctness	Check distribution behaviors	Default to S3; /api, /chat, /socket.io to ALB	Partially verified from CloudFront monitoring; behavior export or browser smoke proof still required
TC28	Frontend no longer in EKS	kubectl get deployment frontend -n internship	Not found	Partially verified from current architecture and EKS workload evidence; explicit command output still useful

Manual command set

Run only with approved production access:

```
aws eks update-kubeconfig --name internship-prod --region ap-southeast-1
kubectl get deployments,pods,svc,endpoints,hpa,pdb -n internship -o wide
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

AWS service checks:

```
aws dynamodb describe-table --table-name ChatUsers --region ap-southeast-1
aws dynamodb describe-table --table-name ChatGroups --region ap-southeast-1
aws dynamodb describe-table --table-name ChatMessages --region ap-southeast-1
aws elasticache describe-replication-groups --replication-group-id internship-prod-redis
→ --region ap-southeast-1
aws sqs get-queue-url --queue-name internship-prod-outbox --region ap-southeast-1
aws lambda get-function --function-name internship-outbox-handler --region ap-southeast-1
aws sagemaker describe-endpoint --endpoint-name internship-qwen3-4b --region ap-southeast-1
```

Request examples

Backend health:

```
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/api/health/ready
```

Chat health:

```
curl -fsS https://dhm2rz5nmsibj.cloudfront.net/chat/health/ready
```

AI health through port-forward:

```
kubectl port-forward service/ai-service 18082:8010 -n internship
curl -fsS http://127.0.0.1:18082/health/ready
```

Database result checks

Do not publish database credentials or raw personal data. Use application APIs, migration logs, and aggregate checks where possible.

Useful production-safe checks:

```
kubectl logs job/backend-migrate -n internship --tail=100
kubectl logs deployment/backend-processing-worker -n internship --tail=100
kubectl logs deployment/backend-outbox-dispatcher -n internship --tail=100
```

Storage result checks

Frontend bucket:

```
aws s3 ls s3://internship-prod-frontend-<AWS_ACCOUNT_ID>/ --region ap-southeast-1
```

Event archive:

```
aws s3 ls s3://internship-prod-uploads-<AWS_ACCOUNT_ID>/outbox-archive/2026/07/28/ --region  
→ ap-southeast-1
```

CloudWatch log checks

Lambda:

```
aws logs tail /aws/lambda/internship-outbox-handler --since 1h --region ap-southeast-1
```

EKS workload CloudWatch log group names require evidence from log shipping configuration before being listed as deployed.

Expected result

The platform is accepted for demonstration when critical public health checks pass, EKS workloads are Ready, DynamoDB/Redis/SQS/Lambda/SageMaker resources are available, and the Lambda smoke-test path has exported proof for event processing, archive, and email result.

Common errors

Symptom	Likely cause	Resolution
Health check passes through ALB but fails through CloudFront	CloudFront behavior or origin configuration issue	Inspect CloudFront distribution behaviors
Chat test fails after scaling	Redis adapter or ALB stickiness issue	Verify Redis readiness and target group stickiness
Duplicate application creates two records	Missing DB unique constraint or idempotency bug	Run backend idempotency tests and inspect constraints
Worker jobs stay queued	Worker disabled or SageMaker not ready	Verify <code>PROCESSING_WORKER_ENABLED</code> , <code>ai-service</code> , and endpoint status
Lambda smoke test fails with DynamoDB reserved word	<code>result</code> used directly in <code>UpdateExpression</code>	Use <code>#result</code> and <code>ExpressionAttributeNames</code> mapping

Outcome

The E2E test plan is evidence-aware. It avoids marking tests passed solely because code exists and records which runtime checks still require exported evidence.

5.11 Security and Cost

Objective

Document the security controls and cost-estimation approach for the production AWS deployment.

Architecture context

The platform processes user accounts, CVs, job applications, chat messages, AI analysis, event notifications, and email. Security controls must protect identity, secrets, private data, deployment roles, storage, network paths, and event processing. Cost controls must pay special attention to always-on resources such as EKS, NAT Gateway, RDS, Redis, ALB, and SageMaker.

Security controls

Area	Implemented or required control	Status
GitHub OIDC	GitHub Actions assumes AWS role through OIDC with <code>id-token: write</code>	Implemented in workflow

Area	Implemented or required control	Status
Long-lived AWS keys	Do not store long-lived AWS access keys in GitHub or source code	Required control
IAM least privilege	Deployment, runtime, node, SageMaker, Lambda, and ALB controller roles should be scoped	Implemented concept; policy evidence required
IRSA	EKS ServiceAccount <code>internship-app</code> can be annotated with runtime role	Implemented in deployment script
Kubernetes secrets	<code>SECRET_KEY</code> , <code>DATABASE_URL</code> , <code>REDIS_URL</code> , <code>OUTBOX_QUEUE_URL</code> , and optional AI key injected as <code>internship-secrets</code>	Implemented in script
S3 private frontend bucket	Bucket public access blocked, CloudFront OAC used	Implemented in <code>ensure-cloudfront.sh</code> ; runtime evidence required
CloudFront HTTPS	CloudFront redirects viewers to HTTPS	Implemented in helper config
ALB route control	ALB routes only API/chat/socket paths to services	Implemented in Ingress
RDS private access	PostgreSQL should not be publicly exposed	Evidence required
Redis private access	ElastiCache should not be publicly exposed	Evidence required
SQS SSE	Main queue and DLQ use server-side encryption	Verified from Week 8 evidence
DynamoDB encryption	DynamoDB tables use AWS-managed encryption by default unless configured otherwise	Evidence required for final table settings
RDS encryption	Must be verified with <code>describe-db-instances</code>	Evidence required
Encryption in transit	CloudFront HTTPS; internal service and database TLS settings require evidence	Partially evidenced
Secret handling	No <code>.env</code> , token, key, password, or database URL should be committed	Required control
DLQ	SQS DLQ isolates failed consumer messages	Verified from Week 8 evidence
Idempotency	PostgreSQL idempotency, outbox, chat <code>clientId</code> , and Lambda DynamoDB dedupe	Implemented; Lambda smoke verified
Backup and PITR	RDS backup and DynamoDB PITR settings require exported evidence	Evidence required
Direct ALB access hardening	Restricting ALB origin to CloudFront is recommended	Optional hardening
AWS WAF	WAF on CloudFront is recommended for production	Optional hardening

Authentication and authorization

The backend uses JWT-based authentication and role-aware routes for candidate and HR workflows. The frontend stores the session under the `internship_tracker_` prefix and clears local/session storage on session expiration. Protected frontend routes redirect users according to role.

Security expectations:

- Passwords must be hashed.
- JWT secret must remain in secrets management, not in source code.
- Candidate and HR routes must enforce ownership and role checks on the backend.
- AI scores should assist review but not become the only hiring decision.

Data protection



Data type	Protection requirement
CVs and documents	Store in private S3, authorize access, avoid logging full content
Applications	Store in PostgreSQL with ownership checks
Chat messages	Store in DynamoDB and authorize access through chat service
Outbox events	Avoid future sensitive payload logging; archive only intended event content
AI prompts	Do not log full CV text; AI service logs prompt hash and character counts
Secrets	Keep in GitHub secrets and Kubernetes Secret; never publish values

Security verification commands

```
aws iam get-role --role-name internship-github-deploy
aws iam get-role --role-name internship-eks-runtime
kubectl get serviceaccount internship-app -n internship -o yaml
kubectl describe secret internship-secrets -n internship
aws s3api get-public-access-block --bucket <FRONTEND_BUCKET_NAME>
aws sqs get-queue-attributes --queue-url <OUTBOX_QUEUE_URL> --attribute-names All --region
→ ap-southeast-1
aws rds describe-db-instances --db-instance-identifier internship-prod-postgres --region
→ ap-southeast-1
```

I do not print or paste secret values from Kubernetes, GitHub, or AWS into the documentation.

Cost evidence source

I report cost data from the Week 8 AWS cost evidence. The evidence records July 1-28, 2026 month-to-date spend and a Billing and Cost Management credits screenshot. The finalized monthly bill and AWS Pricing Calculator estimate remain pending.

Cost evidence	Observed value	Interpretation
July 1-28, 2026 total spend	Current month-to-date cost	\$94.92 Source-of-truth Week 8 cost summary
Highest daily spend	July 28, 2026	\$31.83 Largest daily spend in the July 1-28 period
Credits total amount used	Billing credits	\$27.90 Billing credits screenshot
Credits total estimated amount used	Billing credits	\$140.65 Billing credits screenshot
Credits total amount remaining	Billing credits	\$172.10 Billing credits screenshot
Credits total estimated amount remaining	Billing credits	\$59.35 Billing credits screenshot

These values cover the July 1-28 period only. I do not treat them as the finalized monthly bill or steady-state production estimate until AWS Bills and AWS Pricing Calculator evidence are attached.

Observed Week 8 service cost

The Week 8 cost summary identifies the following top cost drivers for July 1-28, 2026.

Rank	Service	July 1-28 cost	Share of total
1	Amazon RDS	\$29.69	31.3%
2	Amazon SageMaker	\$23.45	24.7%
3	Amazon VPC	\$14.11	14.9%
4	Amazon EC2 - Compute	\$12.42	13.1%

Rank	Service	July 1-28 cost	Share of total
5	EC2 - Other	\$7.68	8.1%
6	Amazon EKS	\$5.64	5.9%

These six services account for approximately 98% of the July 1-28 reported spend. The July 28 daily cost spike indicates that new production infrastructure, SageMaker endpoint uptime, and VPC/NAT-related networking should be monitored closely.

Cost assumptions

Exact future monthly cost still requires AWS Pricing Calculator or a representative Cost Explorer period. The deployed configuration observed in the evidence folder is:

Cost driver	Observed configuration	Monthly estimate input still required
EKS control plane	One production cluster in ap-southeast-1	Cluster runtime hours and current regional EKS hourly price
EC2 worker nodes	Two Ready Kubernetes worker nodes	Node instance type, EBS size, runtime hours
RDS PostgreSQL	Two private, encrypted PostgreSQL db. t4g.micro instances, 20 GiB each, Single-AZ	Backup storage, additional I/O, representative runtime
ElastiCache / Valkey	One available replication group with encryption at rest and in transit	Node type, node count, data transfer
Application Load Balancer	One active internet-facing ALB	ALB runtime hours and LCU usage
CloudFront	Application distribution plus report distribution evidence	Request count, data transfer out, invalidations, behavior export
S3	Frontend bucket object evidence plus upload/archive/report bucket context	Storage GB, PUT/GET requests, lifecycle policy
DynamoDB	Chat and Lambda dedupe tables use on-demand billing in the Week 8 screenshot	Read/write request volume, storage, PITR if enabled
SQS	Main queue and DLQ had zero visible messages in the Week 8 screenshot	Request volume and payload size
Lambda	Outbox handler is Active, 256 MB memory, 20 second timeout	Invocations, average duration, errors/retries
SageMaker	Real-time endpoint is InService with one production variant	Instance type, endpoint uptime, data processed
CloudWatch	Logs and metrics are enabled through AWS and Kubernetes services	Log ingestion, retention, custom metrics, alarms
Data transfer	Cost Explorer shows data transfer adjustment in the captured period	CloudFront egress, ALB traffic, NAT processing, cross-AZ traffic

Cost estimation table

Service	Estimation method	Current evidence status
Amazon EKS	Cluster hourly charge multiplied by runtime hours	Deployed; Week 8 cost summary reports \$5.64
EC2 worker nodes	Node hourly price times two nodes plus EBS storage	Nodes verified; EC2 Compute reports \$12.42, EC2 - Other reports \$7.68
Amazon RDS PostgreSQL	db.t4g.micro hourly price, 20 GiB storage per instance, backup and I/O	Two private encrypted instances verified; Week 8 cost summary reports \$29.69
Amazon ElastiCache / Valkey	Node hourly price times node count plus data transfer	Replication group verified; node type/count still needed
Application Load Balancer	ALB hourly charge plus LCU usage	Active ALB verified; target health still needs final healthy evidence

Service	Estimation method	Current evidence status
Amazon CloudFront	Requests plus data transfer out and invalidations	Monitoring screenshot available; behavior/origin export still required
Amazon S3	Storage GB plus request volume and lifecycle transitions	Frontend bucket object screenshot available
Amazon DynamoDB	On-demand read/write requests plus storage	Table status screenshot available
Amazon SQS	Standard queue requests and payload volume	Queue and DLQ screenshot available
AWS Lambda	Request count plus GB-seconds	Function exists; invocation and trigger health still pending
Amazon SES	Email send count and attachments if any	Service planned through Lambda notification path
Amazon ECR	Image storage GB and data transfer if applicable	Not separated in the Week 8 top-driver table
Amazon SageMaker	Endpoint instance hours and invocation/data charges	Endpoint verified InService; Week 8 cost summary reports \$23.45
Amazon CloudWatch	Log ingestion, retention, metrics, alarms	Retention and ingestion volume still required
Amazon VPC and data transfer	CloudFront egress, ALB traffic, NAT, and cross-AZ traffic	Week 8 cost summary reports Amazon VPC at \$14.11; traffic breakdown still needed
Total	Use Pricing Calculator for forecast, then compare with representative billing evidence	Week 8 July 1-28 spend reports \$94.92; finalized bill and steady-state estimate pending

Evidence hygiene

Before I use any AWS evidence image or log in the documentation, I redact AWS account IDs, access keys, secret keys, database passwords, connection strings, Kubernetes Secret values, GitHub tokens, presigned URLs, and logs containing credentials.

My evidence scan did not find obvious access keys or GitHub tokens, but 02-eks/pod-logs-tail.txt contains a Redis connection URL with an embedded credential from chat-service startup logs. I do not upload or paste that raw file. I use only a summary or replace the whole value with <REDACTED_REDIS_CONNECTION_STRING>.

Largest cost drivers

The highest-risk future cost areas are:

- SageMaker real-time endpoint uptime, especially if the selected endpoint instance is GPU-backed.
- EKS control plane and EC2 worker nodes running continuously.
- RDS and Redis always-on managed capacity.
- ALB hourly and LCU usage.
- CloudFront and data transfer when traffic grows.
- CloudWatch log ingestion and retention if verbose logs are kept.

The Week 8 evidence shows that cost is no longer near zero. SageMaker, RDS, VPC/NAT-related networking, EC2 worker nodes, and EKS should be treated as active cost drivers. SageMaker should be stopped or deleted when demos finish if real-time inference is not required.

Cost optimization options

Option	Effect
Delete or schedule SageMaker endpoint when idle	Reduces likely largest AI cost driver
Keep processing worker disabled when AI testing is not active	Avoids unnecessary retries and worker runtime
Right-size EKS node group	Reduces EC2 and EBS cost
Scale non-production workloads down after demos	Reduces always-on compute

Option	Effect
Confirm whether NAT Gateway is required	Avoids NAT hourly and data-processing charges if private egress is not needed
Add VPC endpoints where appropriate	Reduces NAT data processing for AWS service traffic
S3 lifecycle rules	Move old uploads/archives to cheaper storage or expire test artifacts
ECR lifecycle policy	Delete old SHA images after the retention window
CloudWatch retention	Avoid indefinite high-volume log storage
DynamoDB on-demand during unstable traffic	Avoids over-provisioning early
AWS Budget and billing alert	Detects unexpected charges before they become large

Expected result

I consider the security and cost section complete when it identifies implemented controls, marks missing evidence clearly, avoids secret exposure, and gives a concrete cost-estimation method without inventing live AWS prices.

Common errors

Symptom	Cause	Resolution
Long-lived AWS key in CI	Static credentials used instead of OIDC	Remove key, rotate it, use OIDC role
Direct S3 public access	Bucket policy or public access block misconfigured	Restore public access block and CloudFront OAC policy
Lambda duplicate side effects	Missing event dedupe	Use DynamoDB conditional writes by <code>eventId</code>
Unexpected high bill	SageMaker/NAT/EKS left running	Check Cost Explorer and stop/delete idle high-cost resources
Cost table has fake totals	Pricing not exported	Use <code>Pricing evidence required</code> until AWS Pricing Calculator is attached

Outcome

The deployment follows an evidence-based security model and a practical cost model. Remaining security and cost facts require AWS CLI, Cost Explorer, or AWS Pricing Calculator exports before final numeric claims are made.

5.12 Troubleshooting

Objective

Record production troubleshooting cases, root causes, diagnostic commands, resolutions, and prevention steps.

Architecture context

Most failures in this project happened at integration boundaries: VPC egress, EKS node readiness, AWS Load Balancer Controller, GitHub Actions variables, CloudFront routing, queue naming, DynamoDB expressions, IAM, and AI dependency readiness.

Troubleshooting table

Incident	Symptom	Root cause	Diagnostic command	Resolution	Prevention
NAT Gateway blackhole	Managed EKS nodes could not join cluster	Private subnet default route pointed to a deleted NAT Gateway and route state became blackhole	<code>aws ec2 describe-route-tables --region ap-southeast-1</code>	Created or restored NAT Gateway, updated private route tables, recreated or recovered node group	Check private subnet default routes before node group changes

Incident	Symptom	Root cause	Diagnostic command	Resolution	Prevention
AWS Load Balancer Controller crash	Controller could not create ALB resources	Controller could not obtain VPC ID through metadata; error included failed to get VPC ID from instance metadata and context deadline exceeded	<code>kubectl logs -n kube-system deployment/aws-load-balancer-controller -tail=200</code>	Added <code>--aws-region=ap-southeast-1</code> and <code>--aws-vpc-id=vpc-0cfee519122ae18b4</code>	Configure region and VPC ID explicitly in constrained metadata environments
GitHub Actions frontend job skipped	Frontend deployment did not run	A job-level <code>if</code> used GitHub environment variables before environment values were loaded	Inspect workflow job conditions and run summary	Required flags were also configured as repository variables; frontend deployment was separated from app deployment	Avoid environment-only variables in early job-level conditions
CloudFront architecture correction	API/chat routing was misunderstood	S3 was incorrectly described as behind ALB	<code>aws cloudfront get-distribution --id EQIGYNECXDYLS</code>	Corrected design: default behavior to S3, <code>/api/*</code> , <code>/chat/*</code> , <code>/socket.io/*</code> to ALB	Treat CloudFront distribution behaviors as source of truth
SQS queue mismatch	AWS CLI query returned no queue or wrong queue	CloudShell <code>OUTBOX_QUEUE_URL</code> was empty and wrong queue name <code>internship-outbox</code> was queried	<code>aws sqs list-queues --region ap-southeast-1</code>	Used current queue <code>internship-prod-outbox</code>	Store queue URL in deployment secrets and verify names before testing
SQS script/runtime mismatch	Provisioning script output did not match current runtime queue settings	Repository scripts default to <code>internship-outbox-events</code> and visibility timeout 60; current runtime evidence shows <code>internship-prod-outbox</code> and visibility timeout 120	Inspect <code>scripts/aws/provision-outbox-sqs.sh</code> and AWS queue attributes	I use runtime evidence for the deployed report state; script updates belong in a separate application change task	Keep infrastructure scripts synchronized with production names
Lambda DynamoDB reserved keyword	Lambda failed with DynamoDB <code>ValidationException</code>	<code>UpdateExpression</code> used reserved attribute name <code>result</code> directly	Inspect Lambda CloudWatch logs	Replaced <code>result</code> with <code>#result</code> and mapped <code>"#result": "result"</code>	Use expression attribute names for reserved words
IAM <code>PassRole</code> denial	Deployment or SageMaker operation failed	Role lacked <code>iam:PassRole</code> for the intended SageMaker execution role	Inspect GitHub Actions logs and CloudTrail	Grant scoped <code>iam:PassRole</code> only for required role and service	Add IAM preflight checks before deploy

Incident	Symptom	Root cause	Diagnostic command	Resolution	Prevention
ECR authorization or DescribeImages denial	Deploy job could not verify image tag	GitHub OIDC role lacked required ECR read action	<code>aws ecr describe-images --repository-name internship-backend -image-ids imageTag=<GITHUB_SHA></code>	Add scoped ECR permissions for repository/image verification	Keep deploy role policy aligned with workflow checks
GitHub OIDC AssumeRoleWithWebIdentity failure	AWS credential step failed	OIDC trust policy, audience, branch/environment condition, or role ARN mismatch	Inspect <code>aws-actions/configure-aws-credentials</code> error and role trust policy	Correct trust policy and <code>AWS_ROLE_TO_ASSUME</code>	Test OIDC assume-role before adding deploy steps
Alembic database URL interpolation failure	Migration failed before connecting	Database URL contained characters interpreted by shell or config interpolation	<code>kubect1 logs job/backend-migrate -n internship</code>	Store <code>DATABASE_URL</code> as secret and avoid rewriting it through unsafe interpolation	Treat database URLs as opaque secret values
Docker unavailable in WSL or local environment	CI/local validation could not build images	Docker daemon unavailable or Windows/WSL integration issue	<code>docker version</code>	Start Docker Desktop or use CI validation; report local Docker validation as not run if unavailable	Check Docker before image validation
ShellCheck SC2155	Shell lint failed	Variable declaration and assignment combined in a way ShellCheck flags	<code>shellcheck scripts/*.sh</code>	Split declaration and assignment	Run shell lint before workflow dispatch
SageMaker endpoint not found	AI deploy or worker enablement failed	Endpoint name not created or not InService	<code>aws sagemaker describe-endpoint -endpoint-name internship-qwen3-4b -region ap-southeast-1</code>	Create/restore endpoint or keep worker disabled	Gate worker on endpoint InService
Frontend environment misconfiguration	Frontend called localhost in production	Missing <code>VITE_API_BASE_URL=/api</code> or wrong chat base URL	Inspect built app environment and browser network tab	Build with <code>VITE_API_BASE_URL=/api</code> and empty <code>VITE_CHAT_API_BASE_URL</code>	Keep frontend deploy variables in workflow
ALB certificate/domain handling	Public endpoint used default ALB/CloudFront domain	Custom domain and ACM certificate not configured	<code>aws elbv2 describe-listeners</code> and CloudFront viewer certificate config	Use CloudFront default certificate for default domain or add ACM/custom domain later	Separate default-domain deployment from custom-domain hardening

Detailed incident notes

NAT Gateway blackhole

Private EKS worker nodes need outbound connectivity to reach AWS APIs and pull images. When the private subnet default route points to a deleted NAT Gateway, the route state becomes `blackhole`. Nodes fail to join or

remain unhealthy.

Useful commands:

```
aws ec2 describe-route-tables --region ap-southeast-1
aws ec2 describe-nat-gateways --region ap-southeast-1
aws eks describe-nodegroup --cluster-name internship-prod --nodegroup-name <NODEGROUP_NAME>
→ --region ap-southeast-1
kubectl get nodes -o wide
```

AWS Load Balancer Controller crash

The controller needed explicit region and VPC ID:

```
--aws-region=ap-southeast-1
--aws-vpc-id=vpc-0cfee519122ae18b4
```

Useful commands:

```
kubectl get deployment,pods -n kube-system -l
→ app.kubernetes.io/name=aws-load-balancer-controller -o wide
kubectl logs -n kube-system deployment/aws-load-balancer-controller --tail=200
kubectl describe ingress internship-public -n internship
```

CloudFront routing correction

Correct current design:

CloudFront default *	-> S3 frontend origin
CloudFront /api/*	-> ALB origin -> backend
CloudFront /chat/*	-> ALB origin -> chat-service
CloudFront /socket.io/*	-> ALB origin -> chat-service

Incorrect design to avoid:

S3 behind ALB
frontend running in EKS

Lambda DynamoDB reserved word

The failing expression used `result` directly. The fixed pattern uses an expression attribute name:

```
{
  "UpdateExpression": "SET #result = :result",
  "ExpressionAttributeNames": {
    "#result": "result"
  }
}
```

The successful smoke-test event was:

lambda-smoke-fixed-1785220478

Archive key:

outbox-archive/2026/07/28/lambda-smoke-fixed-1785220478.json

General diagnostic workflow

1. Confirm AWS account and region.
2. Confirm kube context and namespace.
3. Check CloudFront behavior and ALB Ingress status.
4. Check Kubernetes deployment rollouts and pod events.
5. Check application logs.
6. Check managed service health.
7. Check IAM AccessDenied errors and CloudTrail if available.

8. Check queue, Lambda, DLQ, and dedupe state for async failures.
9. Record exact timestamps, command outputs, and final resolution.

Commands:

```
aws sts get-caller-identity
kubectl config current-context
kubectl get deployments,pods,svc,endpoints,ingress -n internship -o wide
kubectl get events -n internship --sort-by=.lastTimestamp
kubectl logs deployment/backend -n internship --tail=200
kubectl logs deployment/chat-service -n internship --tail=200
kubectl logs deployment/backend-outbox-dispatcher -n internship --tail=200
aws logs tail /aws/lambda/internship-outbox-handler --since 1h --region ap-southeast-1
```

Expected result

Each incident should end with:

- root cause identified
- command evidence captured
- fix applied
- verification result recorded
- prevention step documented

Outcome

The troubleshooting record turns deployment failures into reusable operational knowledge and helps future maintainers diagnose networking, IAM, CloudFront, SQS, Lambda, and SageMaker issues faster.

5.13 Clean-up

Objective

Provide a safe, dependency-aware cleanup sequence for the AWS production environment after testing or demonstration is complete.

Warning

The commands in this chapter are destructive. Do not run them unless the project owner, AWS account owner, and team have approved cleanup. Export required evidence, logs, test results, cost reports, and final screenshots before deleting resources.

This chapter is documentation only. No cleanup commands were executed as part of preparing this report.

Architecture context

Cleanup must follow dependency order. Deleting shared networking or IAM first can leave load balancers, ENIs, node groups, endpoints, queues, buckets, or log groups stuck. The safest approach is to stop producers, stop consumers, delete high-cost compute, then remove networking and IAM last.

Pre-cleanup checklist

- ☐ Confirm the environment is no longer needed.
- ☐ Export CloudFront, ALB, EKS, RDS, Redis, DynamoDB, SQS, Lambda, SageMaker, and S3 evidence.
- ☐ Export AWS Cost Explorer or Pricing Calculator evidence.
- ☐ Export Lambda logs for the final smoke test.
- ☐ Export any required S3 event archive objects.
- ☐ Confirm no production users depend on the deployment.
- ☐ Confirm backups or snapshots are either retained or intentionally deleted.
- ☐ Confirm the active AWS account is the intended project account.
- ☐ Confirm the active region is ap-southeast-1.

Verification:

```
aws sts get-caller-identity
aws configure get region
kubectl config current-context
```

Dependency-aware cleanup order

1. Disable the processing worker

Stop long-running AI work before deleting SageMaker or database resources:

```
kubectrl scale deployment/backend-processing-worker --replicas=0 -n internship
kubectrl delete hpa backend-processing-worker -n internship --ignore-not-found
kubectrl delete pdb backend-processing-worker -n internship --ignore-not-found
kubectrl rollout status deployment/backend-processing-worker -n internship --timeout=120s
```

2. Disable Lambda SQS event-source mapping

List mappings:

```
aws lambda list-event-source-mappings \
  --function-name internship-outbox-handler \
  --region ap-southeast-1
```

Disable each mapping:

```
aws lambda update-event-source-mapping \
  --uuid <EVENT_SOURCE_MAPPING_UUID> \
  --enabled false \
  --region ap-southeast-1
```

3. Delete SageMaker endpoint

SageMaker endpoint uptime can be a major cost driver:

```
aws sagemaker delete-endpoint \
  --endpoint-name internship-qwen3-4b \
  --region ap-southeast-1
```

Then inspect endpoint config and model resources before deleting them:

```
aws sagemaker list-endpoint-configs --region ap-southeast-1
aws sagemaker list-models --region ap-southeast-1
```

4. Delete Lambda function

Delete after disabling the event-source mapping:

```
aws lambda delete-function \
  --function-name internship-outbox-handler \
  --region ap-southeast-1
```

5. Delete Lambda event-source mapping

```
aws lambda delete-event-source-mapping \
  --uuid <EVENT_SOURCE_MAPPING_UUID> \
  --region ap-southeast-1
```

6. Inspect or purge test messages

Check queue state:

```
aws sqs get-queue-attributes \
  --queue-url <OUTBOX_QUEUE_URL> \
  --attribute-names ApproximateNumberOfMessages ApproximateNumberOfMessagesNotVisible \
  --region ap-southeast-1
```

If approved, purge test messages:

```
aws sqs purge-queue --queue-url <OUTBOX_QUEUE_URL> --region ap-southeast-1
```

7. Delete SQS queues

```
aws sqs delete-queue --queue-url <OUTBOX_QUEUE_URL> --region ap-southeast-1
aws sqs delete-queue --queue-url <OUTBOX_DLQ_URL> --region ap-southeast-1
```

8. Delete Kubernetes Ingress

Deleting the Ingress lets AWS Load Balancer Controller remove the ALB:

```
kubect1 delete ingress internship-public -n internship --ignore-not-found
```

9. Wait for ALB deletion

```
aws elbv2 describe-load-balancers --region ap-southeast-1
aws elbv2 describe-target-groups --region ap-southeast-1
```

Wait until the internship ALB and target groups are gone.

10. Delete EKS workloads

```
kubect1 delete -f k8s/app/autoscaling.yaml --ignore-not-found
kubect1 delete -f k8s/app/ai-service.yaml --ignore-not-found
kubect1 delete -f k8s/app/backend-processing-worker.yaml --ignore-not-found
kubect1 delete -f k8s/app/backend-outbox-dispatcher.yaml --ignore-not-found
kubect1 delete -f k8s/app/chat.yaml --ignore-not-found
kubect1 delete -f k8s/app/backend.yaml --ignore-not-found
kubect1 delete job backend-migrate chat-init -n internship --ignore-not-found
```

Delete namespace only after confirming no resources remain:

```
kubect1 get all -n internship
kubect1 delete namespace internship
```

11. Delete EKS node group

```
aws eks list-nodegroups \
  --cluster-name internship-prod \
  --region ap-southeast-1

aws eks delete-nodegroup \
  --cluster-name internship-prod \
  --nodegroup-name <NODEGROUP_NAME> \
  --region ap-southeast-1
```

Wait:

```
aws eks wait nodegroup-deleted \
  --cluster-name internship-prod \
  --nodegroup-name <NODEGROUP_NAME> \
  --region ap-southeast-1
```

12. Delete EKS cluster

```
aws eks delete-cluster \
  --name internship-prod \
  --region ap-southeast-1
```

13. Delete RDS

Decide whether to keep a final snapshot:

```
aws rds delete-db-instance \
  --db-instance-identifier internship-prod-postgres \
  --final-db-snapshot-identifier internship-prod-postgres-final-<DATE> \
  --region ap-southeast-1
```

Only skip the snapshot when explicitly approved:

```
aws rds delete-db-instance \  
  --db-instance-identifier internship-prod-postgres \  
  --skip-final-snapshot \  
  --region ap-southeast-1
```

14. Delete ElastiCache

```
aws elasticache delete-replication-group \  
  --replication-group-id internship-prod-redis \  
  --region ap-southeast-1
```

15. Delete DynamoDB tables

Delete only after confirming no data retention is required:

```
aws dynamodb delete-table --table-name ChatUsers --region ap-southeast-1  
aws dynamodb delete-table --table-name ChatGroups --region ap-southeast-1  
aws dynamodb delete-table --table-name ChatMessages --region ap-southeast-1  
aws dynamodb delete-table --table-name InternshipLambdaEventDedupe --region ap-southeast-1
```

16. Empty and delete S3 buckets

Inspect first:

```
aws s3 ls s3://internship-prod-frontend-<AWS_ACCOUNT_ID> --recursive --summarize  
aws s3 ls s3://internship-prod-uploads-<AWS_ACCOUNT_ID> --recursive --summarize
```

Empty and delete after approval:

```
aws s3 rm s3://internship-prod-frontend-<AWS_ACCOUNT_ID> --recursive  
aws s3 rb s3://internship-prod-frontend-<AWS_ACCOUNT_ID>  
aws s3 rm s3://internship-prod-uploads-<AWS_ACCOUNT_ID> --recursive  
aws s3 rb s3://internship-prod-uploads-<AWS_ACCOUNT_ID>
```

If versioning is enabled, delete object versions and delete markers before removing the bucket.

17. Disable and delete CloudFront

CloudFront must be disabled before deletion:

```
aws cloudfront get-distribution-config --id EQIGYNECXDYL8
```

Update the distribution config with Enabled=false, keep the current ETag, then wait:

```
aws cloudfront wait distribution-deployed --id EQIGYNECXDYL8
```

Delete:

```
aws cloudfront delete-distribution \  
  --id EQIGYNECXDYL8 \  
  --if-match <ETAG>
```

18. Delete NAT Gateway

```
aws ec2 describe-nat-gateways --region ap-southeast-1  
aws ec2 delete-nat-gateway --nat-gateway-id <NAT_GATEWAY_ID> --region ap-southeast-1
```

19. Release Elastic IP

Release the NAT Gateway Elastic IP only after the NAT Gateway is deleted:

```
aws ec2 release-address --allocation-id <ALLOCATION_ID> --region ap-southeast-1
```

20. Delete IAM roles and policies

Delete IAM roles last, after workloads that use them are gone:

```
aws iam detach-role-policy --role-name internship-github-deploy --policy-arn <POLICY_ARN>
aws iam delete-role --role-name internship-github-deploy
```

Repeat for project-specific roles only after confirming no shared resource depends on them.

21. Delete remaining VPC resources

Inspect and remove:

- security groups
- route tables
- subnets
- Internet Gateway
- NAT routes
- VPC endpoints
- leftover ENIs

Commands:

```
aws ec2 describe-network-interfaces --region ap-southeast-1
aws ec2 describe-security-groups --region ap-southeast-1
aws ec2 describe-route-tables --region ap-southeast-1
aws ec2 describe-internet-gateways --region ap-southeast-1
aws ec2 describe-vpc-endpoints --region ap-southeast-1
```

22. Verify billing resources are gone

Final checks:

```
aws eks list-clusters --region ap-southeast-1
aws elbv2 describe-load-balancers --region ap-southeast-1
aws rds describe-db-instances --region ap-southeast-1
aws elasticache describe-replication-groups --region ap-southeast-1
aws sqs list-queues --region ap-southeast-1
aws lambda list-functions --region ap-southeast-1
aws sagemaker list-endpoints --region ap-southeast-1
aws cloudfront list-distributions
```

Review AWS Billing and Cost Explorer the next day because some usage records appear after a delay.

Common errors

Symptom	Cause	Resolution
VPC cannot be deleted	ENIs, ALB, NAT, endpoints, or security groups remain	Inspect ENIs and dependent resources
S3 bucket cannot be deleted	Versioned objects or delete markers remain	Delete all object versions and markers
CloudFront cannot be deleted	Distribution still enabled or ETag mismatch	Disable, wait for deployed state, use latest ETag
IAM role cannot be deleted	Policies still attached	Detach inline and managed policies first
SQS messages keep reappearing	Lambda event source still enabled	Disable/delete event-source mapping before queue deletion

Outcome

Cleanup is complete only when high-cost services, queues, buckets, databases, load balancers, endpoints, IAM roles, and VPC resources are removed or intentionally retained with written approval.

6 Self-Evaluation

Self-evaluation

During the development of the **Internship Application Platform**, I had the opportunity to apply knowledge in full-stack development, Cloud architecture, AI integration, realtime communication, containerization, Kubernetes, CI/CD, and observability. This self-evaluation summarizes my completion level, strengths, limitations, and improvement direction after the internship period.

Evaluation Criteria

No.	Criteria	Description	Good	Fair	Average
1	Professional knowledge & skills	Understanding of the field, applying knowledge in practice, proficiency with tools, and work quality	✓	☐	☐
2	Ability to learn	Ability to absorb new knowledge and learn quickly	✓	☐	☐
3	Proactiveness	Taking initiative and seeking out tasks without waiting for instructions	✓	☐	☐
4	Sense of responsibility	Completing tasks on time and ensuring quality	✓	☐	☐
5	Discipline	Adhering to schedules, rules, and work processes	☐	✓	☐
6	Progressive mindset	Willingness to receive feedback and improve oneself	✓	☐	☐
7	Communication	Presenting ideas and reporting work clearly	☐	✓	☐
8	Teamwork	Working effectively with colleagues and participating in teams	✓	☐	☐
9	Professional conduct	Respecting colleagues, partners, and the work environment	✓	☐	☐
10	Problem-solving skills	Identifying problems, proposing solutions, and showing creativity	✓	☐	☐
11	Contribution to project/team	Work effectiveness, technical contribution, and recognition from the team	✓	☐	☐

No.	Criteria	Description	Good	Fair	Average
12	Overall	General evaluation of the entire internship period	✓	☐	☐

Strengths

- I can approach problems from a system perspective instead of focusing only on isolated features. I considered authentication, storage, realtime chat, AI processing, deployment, and observability together.
- I proactively learned AWS services and Cloud-native tools to understand how an application can operate in a production-oriented environment.
- I connected academic knowledge with practical project requirements, such as using S3 presigned URLs for document security, Redis adapter for realtime chat, and workers for long-running tasks.
- I improved my technical writing skills, bilingual report structure, and ability to follow the FCAJ report template.
- I was open to feedback and willing to revise the content when something was not clear or consistent.

Areas for Improvement

- I need to improve time estimation for each work package, especially parts involving many components such as Kubernetes, observability, and CI/CD.
- I need more hands-on practice with production deployment on AWS, including networking, IAM, cost control, and real monitoring.
- I need to improve automated testing coverage for auth, applications, documents, chat, and AI flows.
- I need to improve UI/UX design skills so that the interface is easier to use for both Candidates and HR users.
- I need to keep improving how AI results are explained, so that matching scores are not treated as the only recruitment decision.

Future Learning Plan

- Review and practice the AWS Well-Architected Framework, especially security, reliability, and cost optimization.
- Continue learning Kubernetes, Helm, Ingress, HPA, PDB, probes, and deployment strategies.
- Study observability with Prometheus, Grafana, Loki, Tempo, and OpenTelemetry.
- Improve backend skills with FastAPI, SQLAlchemy, Alembic, PostgreSQL transactions, and security best practices.
- Practice CI/CD with GitHub Actions, OIDC, container scanning, secret scanning, and deployment automation.
- Learn more about AI application design, prompt engineering, schema validation, fallback strategy, and human-in-the-loop review.

Career Orientation

After the internship, I would like to continue developing toward **Cloud/Backend Engineering** combined with **AI integration**. This project helped me understand that a real system does not only need working features. It also needs security, scalability, observability, deployment automation, and cost control.

In the future, I want to keep building more practical projects, deploy them on AWS in a controlled way, and strengthen my system design mindset so that I can contribute better to Cloud-native projects in a professional environment.

7 Sharing and Feedback

Sharing and Feedback

This section summarizes my personal reflections after joining the **First Cloud AI Journey (FCAJ)** program and completing the **Internship Application Platform** project. The feedback focuses on my learning experience, mentor and community support, program strengths, and suggestions for future improvement.

Overall Evaluation

1. Learning and Practice Environment

The program provided a good environment for learning Cloud and AI topics through practical work. Instead of only studying theory, I had the opportunity to build a project with multiple components, including frontend, backend, realtime chat, AI service, databases, Docker, Kubernetes, CI/CD, and observability.

I especially appreciated that the program encouraged learners to research, implement, and record results independently. This approach helped me understand more deeply how a system is designed, operated, and deployed in a Cloud environment.

2. Support from Mentors and Team Admins

Mentors and team admins provided helpful support during the learning process. When I encountered issues, I could ask questions, receive direction, and continue solving the problem by myself instead of only receiving direct answers. This helped me improve debugging, documentation reading, and independent problem-solving skills.

Information about the program, events, workshops, and reference materials was also shared clearly, giving me suitable learning resources for each stage.

3. Alignment with My Major

The internship content was aligned with my Computer Science major and my development direction. The project involved backend engineering, databases, distributed systems, Cloud infrastructure, security, and AI integration.

Through the project, I not only strengthened programming knowledge but also learned more about non-functional requirements such as security, scalability, reliability, monitoring, and cost control.

4. Learning Opportunities and Skill Development

During the internship, I learned several important skills:

- Designing a multi-service system.
- Building REST APIs with FastAPI.
- Managing data with PostgreSQL, DynamoDB, Redis, and S3.
- Integrating realtime chat with Socket.IO.
- Integrating AI for CV and Job Description analysis.
- Containerizing applications with Docker and deploying with Kubernetes.
- Setting up CI/CD and observing systems with metrics, logs, and traces.
- Writing bilingual technical reports with a clear structure.

These skills helped me gain a more practical view of building a Cloud-native system instead of focusing only on individual features.

5. Learning Culture and Community Spirit

The FCAJ community felt open and encouraged knowledge sharing. Online events, workshops, and blog posts in the community gave me more perspectives on AWS Cloud, Agentic AI, enterprise architecture, DevOps, observability, and security.

I also valued the program's encouragement to share knowledge through blogs and reports. This is a good way to turn learned knowledge into content that can be presented and reviewed.

Additional Questions

What was I most satisfied with during the internship?

I was most satisfied with the opportunity to build a relatively complete project that covered business flows, AI integration, realtime communication, deployment direction, and observability. The project helped me understand the relationship between code, infrastructure, and operations.

What could the program improve for future interns?

The program could provide more checklists for each stage, such as backend, frontend, AWS deployment, monitoring, and cost cleanup checklists. This would help learners review their progress more easily and avoid missing important parts.

In addition, sample project review sessions or discussions about common AWS deployment issues would give learners more practical experience before working on their own projects.

Would I recommend the program to friends?

I would recommend the program to friends who are interested in Cloud, backend, DevOps, or AI applications. FCAJ is suitable for learners who want hands-on practice, product-oriented learning, and a better understanding of how AWS services can work together in a real system.

Suggestions and Expectations

- Provide project checklists for each milestone so learners can track progress more easily.
- Add more examples about cost optimization and cleanup to avoid unnecessary AWS charges.
- Share common issues when working with EKS, IAM, S3, CloudFront, RDS, and GitHub Actions.
- Provide more guidance on technical report writing and architecture diagram presentation.
- Organize short project review sessions so learners can receive feedback before finalizing reports.
- Provide more career guidance for Cloud Engineer, Backend Engineer, DevOps Engineer, and AI Engineer paths.

Career Impact

After the program, I have a clearer direction toward Cloud/Backend Engineering combined with AI integration. I realized that building a modern application is not only about writing code. It also includes architecture design, security, deployment, monitoring, cost management, and operational documentation.

The program helped me become more confident in reading technical documentation, analyzing requirements, selecting suitable services, and presenting solutions in a structured way.

Final Message

I would like to thank the First Cloud AI Journey program, mentors, team admins, and community for creating an environment where I could learn and complete the project during the internship. The knowledge and experience gained from this program are an important foundation for my continued development in Cloud, backend, DevOps, and AI application engineering.

Student

Company Representative
